Sitezy: Hybrid Local-LLM Routing, Deep-Learning Post-Generation Optimisation, and Agentic Backend Automation for Multi-Tenant Website Generation

ON

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Engineering

(Information Technology)

By

Manaswi Kabadi - Roll No (15)

Atharva Lotankar - Roll No (20)

Ronit Sahoo - Roll No (29)

Shakti Sankpal - Roll No (30)

Under the guidance of

Mrs. Kajal Joseph

Department of Information Technology

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY,

Chembur, Mumbai 400074

(An Autonomous Institute, Affiliated to University of Mumbai)

AY (2026-27)

VIVEKANAND EDUCATION SOCIETY
INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

Certificate

This is to certify that project entitled

"Sitezy: Hybrid Local-LLM Routing, Deep-Learning Post-Generation Optimisation, and Agentic Backend Automation for Multi-Tenant Website Generation"

Group Members Names

Manaswi Kabadi (Roll No. 15)
Atharva Lotankar (Roll No. 20)
Ronit Sahoo (Roll No. 29)
Shakti Sankpal (Roll No. 30)

In partial fulfilment of the Fourth Year (Sem. VI-Sem.VII), Bachelor of Engineering (B.E.) Degree in Information Technology at the University of Mumbai

Academic Year 2026-27

Mrs. Kajal Joseph
Project Mentor

External Examiner

Dr.(Mrs.) Shalu Chopra
H.O.D

Dr.(Mrs.) J.M. Nair
Principal

Date: / /2026
Place:
VESIT, Chembur

College Seal

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

- - - - - - - - - - - - - -
(Signature)
Manaswi Kabadi (Roll No. 15)

- - - - - - - - - - - - - -
(Signature)
Atharva Lotankar (Roll No. 20)

- - - - - - - - - - - - - -
(Signature)
Ronit Sahoo (Roll No. 29)

- - - - - - - - - - - - - -
(Signature)
Shakti Sankpal (Roll No. 30)

Date:
Place: VESIT, Chembur

Abstract

Small businesses need websites that look professional, rank in search engines, convert visitors and accept customer input, yet AI website builders send every generation request to a metered cloud language model, hand over pages that receive no measured optimisation, and leave forms, storage and hosting for the owner to wire. This project presents *Sitezy*, a multi-tenant software-as-a-service platform that generates complete multi-page websites from a plain-language brief and then improves, publishes and operates them.

Sitezy is built on a local-first, task-typed AI stack. Decisions with a bounded output space (block recommendation, SEO scoring, design auditing and conversion prediction) are made by small learned models in a Python service (Tier 1); only open-ended generation reaches a language model, served locally through Ollama (Qwen3.5-4B, Tier 2), with Google Gemini as an explicit, auditable cloud exception (Tier 3). Around this stack the platform provides a real-time collaborative visual builder, page versioning, a gated SEO rewrite loop, dynamic forms backed by Firestore, snapshot publishing with rollback, an agentic Firebase deployment pipeline with a diagnosing agent and circuit breaker, role-based multi-tenant isolation and Razorpay subscriptions.

Executing the deployed code on 20 benchmark sites and 66 real published pages showed that disabling the local model's reasoning mode is decisive: 59 of 60 generations were schema-valid, at about US$0.15 of GPU energy per 1,000 sites against US$13.43 for the 3.7 times faster cloud model. A GRU conversion model reached 99.4% of the Bayes-optimal AUC on synthetic sessions, the gated rewrite loop raised rubric SEO scores without lowering any page, and tenant-isolation tests found two access-control defects, which were fixed. The work was accepted for publication at ITI 2026 (Springer LNNS).

Keywords—*Website generation, Local LLM, Inference routing, Constrained decoding, Multi-tenant SaaS, GRU, Anomaly detection, SEO, Agentic deployment*

Contents

Chapter 1. Introduction 2

1.1 Introduction 2

1.2 Problem Definition 3

1.3 Objectives 3

1.4 Motivation 4

1.5 Scope of the Work 4

1.6 Feasibility Study 5

1.7 Organization of the report 5

Chapter 2. Literature Survey 8

2.1 Introduction 8

2.2 Review of Literature Survey 8

2.2.1 FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance 8

2.2.2 RouteLLM: Learning to Route LLMs with Preference Data 9

2.2.3 Efficient Guided Generation for Large Language Models 9

2.2.4 Design2Code: How Far Are We From Automating Front-End Engineering? 10

2.2.5 WebGen-Bench: Evaluating LLMs on Generating Interactive and Functional Websites from Scratch 10

2.2.6 Important Factors for Improving Google Search Rank 11

2.2.7 Session-based Recommendations with Recurrent Neural Networks 11

2.2.8 Isolation Forest 12

2.2.9 Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach 12

2.3 Research Gaps 13

2.4 Conclusion 14

Chapter 3. Proposed Methodology 16

3.1 Introduction 16

3.2 Requirement Gathering 16

3.2.1 Requirement Gathering Approach 16

3.2.2 Functional Requirements....17
3.2.3 Non-Functional Requirements....18
3.2.4 User Requirements....18
3.3 Proposed Algorithm....18
3.3.1 Task-Typed Routing with Deterministic Escalation....19
3.3.2 Constrained Local Generation....20
3.3.3 MLP SEO Scorer and Gated Rewrite Loop....21
3.3.4 GRU Conversion Model and Counterfactual Suggestions....21
3.3.5 Design Auditor....22
3.3.6 Zero-Shot Block Recommender....22
3.3.7 Agentic Deployment Orchestrator....22
3.4 Gantt Chart for Timeline....23
3.5 Architectural Diagrams....24
3.5.1 UML Diagrams....24
3.5.2 Block Diagram....27
3.5.3 Data Flow Diagram....28
3.5.4 Timeline Chart....29
3.6 Hardware Requirements....30
3.7 Software Requirements....30
3.8 Cost Estimation....30
3.9 Conclusion....31
Chapter 4. Implementation & Results....33
4.1 Introduction....33
4.2 Implementation....33
4.2.1 Project Structure and Technology Stack....33
4.2.2 Frontend....34
4.2.3 Backend Modules....34
4.2.4 Data Model....34
4.2.5 AI Generation Tier....35
4.2.6 Machine Learning Engine....35
4.2.7 Collaboration and Versioning....36

4.2.8 Publishing, Forms and Agentic Deployment 37
4.2.9 Security, Payments and Monitoring 37
4.3 Results of Implementation 37
4.4 Result Analysis 48
4.4.1 Generation Validity, Latency and Cost 48
4.4.2 SEO Engine 50
4.4.3 Conversion Model and Design Auditor 51
4.4.4 Recommender and Forms 51
4.4.5 Real Tenant Pages 52
4.4.6 Deployment Pipeline 52
4.4.7 End-to-End Ablation 53
4.4.8 Security and Tenant Isolation 53
4.4.9 Comparison with Objectives 54
4.5 Limitations 54
4.6 Observation/Remarks 55
4.7 Conclusion 55
Chapter 5. Conclusion 57
5.1 Limitations 57
5.2 Conclusion 57
5.3 Future Scope 58
5.4 Published Paper 58
References 59
Appendix 61
A.I. Publication Details 61
Camera-Ready Paper 61
Plagiarism Report 73
AI Writing Check Report 76
A.II. IPR: Copyright/Patent/etc. (if granted) 78
Acknowledgement Slip 79
Form XIV with Statement of Particulars 80

Supporting Document 85

No Objection Certificates.....88

List of Figures

Figure 3.1: Development methodology with the evaluation feedback loop 16
Figure 3.2: Three-tier architecture of Sitezy with the AI router and deployment orchestrator 19
Figure 3.3: Gantt chart of the project timeline (2026) 23
Figure 3.4: Use case diagram 24
Figure 3.5: Class diagram of the main entities 25
Figure 3.6: Sequence diagram of AI website generation 26
Figure 3.7: Activity diagram of the gated SEO auto-improve loop 27
Figure 3.8: Block diagram: Sitezy Workflow Diagram (copyright application AT-40971/2026-CO) 28
Figure 3.9: Data flow diagram, level 0 (context diagram) 29
Figure 3.10: Data flow diagram, level 1 29
Figure 3.11: Timeline chart of project milestones 30
Figure 4.1: Landing page of Sitezy 38
Figure 4.2: Workspace registration 38
Figure 4.3: Tenant dashboard with usage, plan and quick actions 39
Figure 4.4: Projects page listing the tenant's websites 39
Figure 4.5: AI playground with the business brief, settings and target website 40
Figure 4.6: Generation in progress with the local model 40
Figure 4.7: Generation result: three pages designed and saved 41
Figure 4.8: Visual builder with pages, section layers and live preview 41
Figure 4.9: SEO & Insights panel: SEO score with factor diagnostics 42
Figure 4.10: Engagement readiness and GRU reordering suggestions 43
Figure 4.11: Design health scan 43
Figure 4.12: Version history after creating a commit 44
Figure 4.13: Published coffee-shop website served to visitors 44
Figure 4.14: Project analytics: live visitors, total visitors, form submissions and AI-generated business insights 45
Figure 4.15: Form-submission inbox of the Pizzatly site 46
Figure 4.16: Workspace settings with Workspace Details, Team Members and Custom Domains tabs 46

Figure 4.17: Subscription plans 47
Figure 4.18: Deployment history 47
Figure 4.19: Activity log 48
Figure 4.20: Schema-valid generations by decoding configuration 49
Figure 4.21: Marginal cost and latency of local and cloud generation 49
Figure 4.22: Conversion-model AUC and design-auditor recall on unseen anomalies 51
Figure 4.23: End-to-end ablation: final SEO rubric score per arm (dashed line: full system) 53
Figure A.1: Copyright Office portal: list of submitted applications (Form XIV) 78
Figure A.2: Copyright Office portal: status of diary number AT-40971/2026-CO 78

List of Tables

Table 2.1: Summary of the literature survey 13

Table 2.2: Research gap analysis 13

Table 3.1: Functional requirements 17

Table 3.2: Non-functional requirements 18

Table 3.3: User requirements by user class 18

Table 3.4: Hardware requirements 30

Table 3.5: Software requirements 30

Table 3.6: Cost estimation 31

Table 4.1: Services of the implemented system 33

Table 4.2: MongoDB collections 34

Table 4.3: Synthetic training data of the Tier-1 engines 36

Table 4.4: Generation validity, latency and output length on the 20 production prompts 48

Table 4.5: SEO scorer against baselines (5 seeds, 5,000 test pages each; R^2 and MAE against the noise-free rubric) 50

Table 4.6: Deployment fault injection under three retry policies 52

Table 4.7: End-to-end ablation on the 20 benchmark sites (60 pages), one factor changed per arm 53

Table 4.8: Achievement of the objectives 54

Table A.1: Summary of the originality and AI-writing checks 73

Table A.2: Copyright application filed for the project 78

ACKNOWLEDGEMENT

The project report on “Sitezy: Hybrid Local-LLM Routing, Deep-Learning Post-Generation Optimisation, and Agentic Backend Automation for Multi-Tenant Website Generation” is the outcome of the guidance, moral support and devotion bestowed on our group throughout our work. For this we acknowledge and express our profound sense of gratitude to everybody who has been the source of inspiration throughout project preparation.

First and foremost we offer our sincere thanks and innate humility to Dr.(Mrs.) Shalu Chopra, Head of the Department of Information Technology, and to our project guide Mrs. Kajal Joseph, Department of Information Technology, for their valuable inputs and the consistent guidance and support they provided at every stage, from the first prototype to the evaluation and the research paper. We are grateful to Dr.(Mrs.) J.M. Nair, Principal, for her encouragement and support.

We must at the outset express our gratitude for the affectionate care of Vivekanand Education Society's Institute of Technology, which provided a stimulating atmosphere and a conducive work environment. We also thank the faculty and staff of the Department of Information Technology, the organisers of the SPIT Hackathon 2026, where the first prototype of Sitezy was built, and the reviewers of ITI 2026 whose comments strengthened the evaluation reported here.

- - - - - - - - - - - - - -
(Signature)
Manaswi Kabadi (Roll No. 15)

- - - - - - - - - - - - - -
(Signature)
Atharva Lotankar (Roll No. 20)

- - - - - - - - - - - - - -
(Signature)
Ronit Sahoo (Roll No. 29)

- - - - - - - - - - - - - -
(Signature)
Shakti Sankpal (Roll No. 30)

CHAPTER: 1

INTRODUCTION

Chapter 1
Introduction

1.1 Introduction

A website is now the first point of contact between a small business and its customers. A café, a clinic, a coaching institute or a co-working space needs a site that looks professional, explains what it offers, appears in search results, persuades visitors to act and collects their enquiries or orders. Building such a site couples four different skills: layout and visual design, copywriting, search-engine optimisation (SEO) and backend engineering for forms, storage and hosting. Most small businesses have none of these in-house.

Website builders such as Wix and Webflow [1, 2] made layout tractable through templates and drag-and-drop editors, and recent AI builders go further by generating a first draft of a site from a short description. In these systems, however, every content request is sent to a metered cloud language model, the tenant's business data leaves the platform, the generated page receives no measured optimisation once it exists, and the forms on the page stay inert until someone connects them to a database and a hosting service.

Sitezy is a multi-tenant software-as-a-service (SaaS) website platform developed as our capstone project. A team describes its business in plain language, in English, Hindi or Marathi, by typing or by voice; Sitezy generates a complete multi-page site with copy, sections, colours, fonts and images; the team refines it in a real-time collaborative visual builder; learned models score its SEO, conversion readiness and design health and propose fixes; and the site is published to a subdomain or a custom domain, or exported as a static site, with its forms already connected to a tenant-scoped database. The first working prototype was built at the SPIT Hackathon 2026 and was extended over the following semesters into the system described in this report.

The central design idea is a **local-first, task-typed AI stack**. Decisions with a bounded output space, such as choosing blocks, scoring SEO, auditing a design or predicting conversion, are answered by small learned models (Tier 1). Only open-ended generation reaches a language model, which is served locally on the platform's own GPU (Tier 2), and a cloud model (Tier 3) is used only when a tenant explicitly chooses it or the local model fails. The rest of the platform forms a loop around this stack: generate, score and audit, improve, publish, collect forms and visitor events, and learn.

1.2 Problem Definition

Existing AI website builders leave five problems unsolved for small businesses and for the platforms that serve them:

1. **Cost and data exposure of cloud-only generation.** Every generation is billed per token by a cloud provider and sends the tenant's business data off-platform, although most of the decisions a builder makes do not need a large language model at all.
2. **Unreliable structured output.** A generated site must be machine-readable layout JSON. Small local models with a reasoning mode can spend their output budget on hidden reasoning and return truncated, invalid output.
3. **No measured optimisation after generation.** Generated pages are not scored for SEO, conversion readiness or design quality, and automatic rewrites can make a page worse as easily as better.
4. **Backend left to the user.** Forms, form storage, hosting and recovery from deployment failures still require engineering effort the target users do not have.
5. **Multi-tenant isolation.** A shared platform must guarantee that one tenant can never read or modify another tenant's sites, pages, forms or deployments [3].

Problem statement: *To design, implement and evaluate a multi-tenant website generation platform that produces complete, schema-valid multi-page websites from a plain-language brief using a locally served language model, improves them with small learned models under a gate that never lowers quality, publishes them with working forms through a self-diagnosing deployment pipeline, and isolates every tenant's data, at a small fraction of the marginal cost of cloud-only generation.*

1.3 Objectives

1. To build a multi-tenant SaaS platform with tenant-scoped data, JWT authentication, four roles (Owner, Admin, Editor, Developer) and subscription plans that limit websites, AI usage and custom domains.
2. To generate complete multi-page websites (layout, copy, colours, fonts and images) from a natural-language brief, including multilingual and voice input.
3. To route AI work by task type, so that bounded tasks are answered by small learned models, open-ended generation by a local language model, and the cloud model only as an explicit exception.
4. To make local generation reliable through constrained decoding, schema validation, deterministic fallback and targeted regeneration of incomplete forms.

5. To develop learned post-generation engines: an MLP SEO scorer with a gated rewrite loop, a GRU conversion model with counterfactual suggestions, an autoencoder plus Isolation Forest design auditor and a zero-shot block recommender.
6. To provide a real-time collaborative visual builder with version history, multi-page management and exact static code export.
7. To automate the backend: publishing with immutable snapshots and rollback, form storage in Firestore and an agentic Firebase deployment pipeline with error diagnosis, safe retries and a circuit breaker.
8. To evaluate every component quantitatively by executing the deployed code, including ablations, cloud baselines, a cost model, real tenant pages, fault injection and tenant-isolation tests, and to publish the results.

1.4 Motivation

Our motivation came from watching small businesses around us struggle with their online presence. Template builders still expect the owner to write copy and understand SEO, freelance developers are expensive for a business that needs a five-page site, and AI builders produce a draft but stop there. At the same time, open-weight language models have become small enough to run on a single consumer GPU. This raised a practical question: how much of an AI website builder actually needs a cloud language model?

A second motivation was the gap between generating a page and making it good. Search visibility depends on measurable on-page factors [4], visitor behaviour can be modelled as a sequence [5], and design flaws can be detected as anomalies without labelled failures [6]. These techniques are well studied individually but are rarely connected to a generator in one loop. Building Sitezy let us combine full-stack engineering, machine learning and systems evaluation in one product, and test the design with the rigour expected of a research paper.

1.5 Scope of the Work

In scope. The project covers the complete platform: the React single-page application (landing page, registration and login, dashboard, AI playground, visual builder, analytics, settings, subscription, deployments and activity pages, and the public-site renderer); the Node.js/Express API with 16 MongoDB collections, Socket.io collaboration, versioning, publishing, domains, forms, analytics, payments and administration; the Flask machine learning service with the four Tier-1 engines and their training scripts; local and cloud language-model integration; the agentic Firebase deployment pipeline; and an evaluation suite whose scripts and seeds are released with the source code.

Out of scope. Training or fine-tuning a language model, e-commerce checkout for tenants' own customers, native mobile applications, measuring real search rankings or conversion through online controlled experiments, and a formal penetration test. The learned engines are trained on synthetic data because no public dataset links page structure to search outcomes, visitor sequences or design-quality labels; Chapters 4 and 5 discuss the consequences.

1.6 Feasibility Study

Technical feasibility. All components use mature open-source technology: React, Node.js/Express, MongoDB, Flask, PyTorch and scikit-learn. The local language model (Qwen3.5-4B, 4-bit quantised) needs about 6.1 GB of GPU memory and runs on the RTX 4060 laptop GPU the team already owned. Firebase, Razorpay and Unsplash provide free tiers or test modes that are sufficient for development and evaluation.

Economic feasibility. Development used existing laptops and free or open-source software, so the cash cost of the project is negligible (Section 3.8). For operation, the measured marginal cost of local generation is about US$0.15 of GPU energy per 1,000 sites, against US$13.43 in API fees for the cloud model, which makes a free tier and low-price plans (₹499 to ₹2,499 per month) viable.

Operational feasibility. Users interact only through a browser. Generation needs a short brief, every AI suggestion is shown with an explanation and an explicit apply button, and an in-app help assistant answers how-to questions, so non-technical users can operate the system without training.

Schedule feasibility. A working prototype was completed in two days at the SPIT Hackathon 2026; the remaining features, the learned engines and the evaluation were planned across Semesters VI and VII (Fig. 3.3) and completed within the academic year.

Legal feasibility. All libraries and model weights are used under their open-source licences, stock images are taken from Unsplash under its licence, payments run through Razorpay, and personal fields are redacted before any visitor data is sent to the cloud model.

1.7 Organization of the report

- **Chapter 1, Introduction,** presents the background, problem definition, objectives, motivation, scope and feasibility of the project.
- **Chapter 2, Literature Survey,** reviews research on LLM routing, constrained decoding, website generation, SEO, session modelling, anomaly detection, zero-shot classification and multi-tenancy, and identifies the research gaps that Sitezy addresses.

- **Chapter 3, Proposed Methodology,** describes requirement gathering, functional and non-functional requirements, the proposed system and its algorithms, the project timeline, the UML, block, data-flow and timeline diagrams, the hardware and software requirements and the cost estimate.
- **Chapter 4, Implementation and Results,** explains how each module was implemented, presents screenshots of the working system, and analyses the quantitative evaluation.
- **Chapter 5, Conclusion,** summarises the limitations, conclusions, future scope and the published paper.
- The **References** and the **Appendix** (camera-ready paper, plagiarism and AI-writing reports, and IPR details) follow.

CHAPTER: 2

LITERATURE SURVEY

Chapter 2
Literature Survey

2.1 Introduction

Sitezy sits at the meeting point of several research areas: cost-aware serving of large language models (LLMs), structured generation, automatic website generation, search-engine optimisation, behavioural modelling of website visitors, unsupervised anomaly detection, zero-shot text classification and multi-tenant SaaS engineering. Before designing the platform we studied how each area approaches its problem, which techniques are mature enough to reuse, and where existing work stops short of what a small-business website platform needs.

The sources reviewed include peer-reviewed conference and journal papers (NeurIPS, ICLR, EMNLP, IEEE ICDM, Future Internet), widely cited arXiv preprints, standards documents such as WCAG 2.1, and existing commercial systems such as Wix ADI and Webflow [1, 2]. For each work we examined the approach, the technology used, the reported results and the limitations, with a view to three questions: can the technique replace a language-model call, can it make generated output more reliable, and can it measure or improve a page after generation? The answers define the research gaps in Section 2.3 and justify the design choices of Chapter 3.

2.2 Review of Literature Survey

Nine research papers were reviewed in detail. Each review gives the title and link, the authors and year, a summary in our own words, our observations and the inference drawn for Sitezy. Table 2.1 summarises the review.

2.2.1 FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance

Title of the paper (link): FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance (https://arxiv.org/abs/2305.05176)

Author(s) and year of publication: L. Chen, M. Zaharia and J. Zou, 2023 [7]

Abstract/Summary: The paper studies how an application can answer queries with commercial LLM APIs at lower cost. It proposes three strategies: prompt adaptation, LLM approximation (for example, caching) and an LLM cascade, in which a query is sent to a cheap model first and a learned scoring model decides whether to accept the answer or escalate to a more expensive model.

Observations: The cascade, trained per task on labelled examples, matched the accuracy of the strongest single API at a large fraction of its cost on several question-answering and classification

datasets. All candidate models are paid cloud APIs, escalation depends on a learned confidence score, and the tasks studied have short answers.

Inference drawn: Cost can be cut substantially by not sending every query to the strongest model. For Sitezy, however, a learned scorer would need per-tenant calibration data that a new tenant does not have, and the outputs are long JSON documents, so we escalate on deterministic, observable events instead and keep bounded tasks away from LLMs altogether.

2.2.2 RouteLLM: Learning to Route LLMs with Preference Data

Title of the paper (link): RouteLLM: Learning to Route LLMs with Preference Data (https://arxiv.org/abs/2406.18665)

Author(s) and year of publication: I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous and I. Stoica, 2024 [8]

Abstract/Summary: RouteLLM learns a router that sends each query either to a strong, expensive model or to a weak, cheap one. The routers (similarity-weighted ranking, matrix factorisation, a BERT classifier and a causal-LLM classifier) are trained on human preference data from Chatbot Arena, augmented with judged and labelled data.

Observations: Trained routers reduced the share of queries sent to the strong model considerably while keeping most of its quality on MT-Bench, MMLU and GSM8K, and transferred to model pairs not seen in training. Routing is always between two language models and requires large preference datasets; general chat is the target workload.

Inference drawn: Routing is an effective cost lever, and a survey of routing strategies [9] confirms that routing among LLMs is the dominant approach. Sitezy routes by task type instead, so recommendation, scoring, auditing and prediction never reach a language model, and routing decisions remain auditable rules.

2.2.3 Efficient Guided Generation for Large Language Models

Title of the paper (link): Efficient Guided Generation for Large Language Models (https://arxiv.org/abs/2307.09702)

Author(s) and year of publication: B. T. Willard and R. Louf, 2023 [10]

Abstract/Summary: The paper reformulates constrained text generation as transitions of a finite-state machine. A regular expression or context-free grammar is compiled into an index over the model's vocabulary, so that at each decoding step the tokens that would violate the grammar can be masked cheaply. The method is implemented in the open-source Outlines library.

Observations: Guided generation guarantees output that matches a schema such as JSON, with very small per-token overhead. The guarantee holds only if decoding finishes: the paper does not

consider token budgets or reasoning modes that consume output tokens before the structured answer starts.

Inference drawn: Grammar-constrained decoding is the right tool for layout JSON, and Sitezy uses Ollama's JSON grammar. Our experiments add the missing piece: for long outputs from a reasoning model, validity is decided by whether reasoning tokens exhaust the budget, so the reasoning mode must be disabled and a schema check and fallback kept.

2.2.4 Design2Code: How Far Are We From Automating Front-End Engineering?

Title of the paper (link): Design2Code: How Far Are We From Automating Front-End Engineering? (https://arxiv.org/abs/2403.03163)

Author(s) and year of publication: C. Si et al., 2024 [11]

Abstract/Summary: Design2Code introduces a benchmark of real-world web pages for converting a screenshot into HTML/CSS code, with automatic metrics for visual element recall, text, position and colour similarity, complemented by human evaluation. Several multimodal LLMs and prompting strategies, including self-revision, are compared.

Observations: Proprietary multimodal models performed best and open models lagged behind; models were good at reproducing text and colour but weaker at recalling all visual elements and laying them out correctly. The task is a single static page with no backend, no business content to write and no optimisation after generation.

Inference drawn: Benchmarks of web generation focus on the generation step. Sitezy targets what surrounds it: business-specific copy, multi-page structure, measured post-generation optimisation, working forms and deployment.

2.2.5 WebGen-Bench: Evaluating LLMs on Generating Interactive and Functional Websites from Scratch

Title of the paper (link): WebGen-Bench: Evaluating LLMs on Generating Interactive and Functional Websites from Scratch (https://arxiv.org/abs/2505.03733)

Author(s) and year of publication: Z. Lu et al., 2025 [12]

Abstract/Summary: WebGen-Bench evaluates LLM-based coding agents that must build a complete multi-file website codebase from a natural-language instruction. Test cases for each requested functionality, 647 in total, were generated and manually filtered, and a web-navigation agent executes them against the generated site. The authors also release WebGen-Instruct, a training set of 6,667 instructions.

Observations: Even the best agent and model combination passed only a minority of the functional test cases, and an open 32B coder model fine-tuned on agent trajectories reached 38.2%

accuracy, ahead of the best proprietary model. Evaluation covers functionality only, not search visibility, conversion, design quality, hosting or tenancy.

Inference drawn: Free-form code generation of whole websites is still unreliable. Generating a constrained layout JSON for a tested component library, as Sitezy does, trades flexibility for reliability and lets every page be validated against a schema.

2.2.6 Important Factors for Improving Google Search Rank

Title of the paper (link): Important Factors for Improving Google Search Rank (https://doi.org/10.3390/fi11020032)

Author(s) and year of publication: C. Ziakis, M. Vlachopoulou, T. Kyrkoudis and M. Karagkiozidou, 2019 [4]

Abstract/Summary: The paper investigates which factors influence the ranking of web pages in Google search. It combines a review of SEO literature with an empirical analysis of pages ranked for a set of keywords, measured with SEO auditing tools, to identify on-page and off-page factors associated with higher positions.

Observations: Ranking was associated with measurable page attributes such as relevant content and keyword use in titles, headings and body text, together with technical and off-page signals. Ranking algorithms are not public, so the findings are correlational and change over time.

Inference drawn: On-page factors that a builder controls can be extracted and scored automatically. Sitezy's SEO scorer uses 15 such features; because no dataset links them to real rankings, its labels come from a published-factor rubric and its scores are reported as a proxy, not as search performance.

2.2.7 Session-based Recommendations with Recurrent Neural Networks

Title of the paper (link): Session-based Recommendations with Recurrent Neural Networks (https://arxiv.org/abs/1511.06939)

Author(s) and year of publication: B. Hidasi, A. Karatzoglou, L. Baltrunas and D. Tikk, 2016 [5]

Abstract/Summary: GRU4Rec models a user session on a website as a sequence of events and uses a recurrent network with gated recurrent units [13] to predict the next item. It introduces session-parallel mini-batches and ranking losses suited to recommendation.

Observations: The GRU model clearly outperformed item-to-item baselines on e-commerce click-stream and video-watching data, showing that the order of events within a session carries predictive information. It requires real session logs and predicts the next item rather than an outcome such as a conversion.

Inference drawn: Visitor sessions on a generated site can be modelled with a GRU. Sitezy adapts the idea to predict conversion from the sequence of sections a visitor sees, which lets it answer the counterfactual question of how moving a section would change the predicted conversion; a GRU matches an LSTM [14] with fewer parameters [15].

2.2.8 Isolation Forest

Title of the paper (link): Isolation Forest (https://doi.org/10.1109/ICDM.2008.17)

Author(s) and year of publication: F. T. Liu, K. M. Ting and Z.-H. Zhou, 2008 [6]

Abstract/Summary: Isolation Forest detects anomalies by isolating them rather than profiling normal data. Random trees split the data on random features and thresholds; anomalies, being few and different, are isolated in fewer splits, so a short average path length indicates an anomaly.

Observations: The method runs in linear time with low memory, works with small sub-samples, needs no labelled anomalies and outperformed distance- and density-based detectors on benchmark datasets. Its axis-parallel splits can miss anomalies that differ only in combinations of features.

Inference drawn: Design flaws are rare and varied, and labelled examples of bad designs do not exist. An Isolation Forest, combined with a reconstruction-error autoencoder [16, 17] and trained only on clean templates, can flag unusual designs that a fixed rubric does not catalogue.

2.2.9 Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach

Title of the paper (link): Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach (https://arxiv.org/abs/1909.00161)

Author(s) and year of publication: W. Yin, J. Hay and D. Roth, 2019 [18]

Abstract/Summary: The paper standardises the evaluation of zero-shot text classification across topic, emotion and situation datasets and proposes treating classification as textual entailment: each label is turned into a hypothesis such as “this text is about …” and a model trained on natural language inference decides whether the input entails it.

Observations: Entailment-based models classified text into unseen label sets better than earlier zero-shot baselines without task-specific training data. Results depend on the hypothesis template, and inference costs one model pass per candidate label.

Inference drawn: A business brief can be classified into business type, content type, goal and audience without labelled briefs. Sitezy uses an entailment cross-encoder (MiniLM [19]) in exactly this way and feeds the labels to random forests [20] that select blocks. Scoring the entailment class, not the neutral class, proved essential.

Table 2.1: Summary of the literature survey

Sr. No.	Author(s) & Year	Paper Title	Methodology / Technology Used	Key Observations	Inference Drawn
1	Chen, Zaharia, Zou (2023)	FrugalGPT	LLM cascade with a learned answer scorer; prompt adaptation; caching	Large cost savings at similar accuracy; needs labelled data per task; cloud APIs only	Avoid sending every task to the strongest model; escalate on observable events
2	Ong et al. (2024)	RouteLLM	Routers trained on human preference data (MF, BERT, causal LLM)	Fewer strong-model calls with most of the quality; routes only among LLMs	Route by task type so bounded tasks never reach an LLM
3	Willard, Louf (2023)	Efficient Guided Generation for LLMs	Finite-state machine index over the vocabulary for regex/CFG-constrained decoding	Guarantees schema-valid output if decoding completes; budgets not studied	Use JSON grammar, disable reasoning, keep schema check and fallback
4	Si et al. (2024)	Design2Code	Screenshot-to-code benchmark; automatic and human evaluation	Open models lag; static single pages only	Focus beyond generation: content, optimisation, deployment
5	Lu et al. (2025)	WebGen-Bench	Agent-built multi-file websites tested by a web-navigation agent	Low functional pass rates even for the best agents	Generate constrained layout JSON for a tested component library
6	Ziakis et al. (2019)	Important Factors for Improving Google Search Rank	Literature review plus empirical analysis of ranked pages with SEO tools	Measurable on-page factors associate with ranking; correlational	Score 15 on-page features; report scores as a proxy
7	Hidasi et al. (2016)	Session-based Recommendations with RNNs (GRU4Rec)	GRU over click sequences; session-parallel mini-batches; ranking losses	Clearly beats item-based baselines; needs real logs	Model visitor section sequences with a GRU for conversion
8	Liu, Ting, Zhou (2008)	Isolation Forest	Random isolation trees; anomaly score from path length	Fast, label-free, strong on benchmarks	Audit designs with IF plus autoencoder trained on clean templates
9	Yin, Hay, Roth (2019)	Benchmarking Zero-shot Text Classification	Classification as textual entailment with NLI models	Works on unseen labels; template-sensitive; one pass per label	Classify briefs zero-shot with MiniLM entailment before block selection

Note: the review also drew on supporting studies cited in the text, including work on gated recurrent networks [13, 15], autoencoder anomaly detection [16, 17], WCAG contrast requirements [21], online controlled experiments [22] and multi-tenant SaaS maintenance [3].

2.3 Research Gaps

Comparing the reviewed work with existing AI website builders reveals the gaps listed in Table 2.2.

Table 2.2: Research gap analysis

Sr. No.	Existing Approach / System	Identified Limitation / Research Gap	Proposed Project's Approach
1	Cloud AI website builders (e.g. Wix ADI) [1]	Every generation is a metered cloud call; tenant data leaves the platform; routing is undocumented	Local-first generation on the platform's GPU; cloud only as an explicit, logged exception with redaction of visitor data
2	LLM cascades and routers (FrugalGPT, RouteLLM) [7, 8]	Route only among language models; need labelled or preference data for calibration	Task-typed routing: bounded tasks go to small learned models; deterministic escalation needs no calibration data
3	Grammar-constrained decoding [10]	Validity guarantee ignores token budgets and reasoning modes	Measure the effect of reasoning on validity; disable it, validate against a schema and fall

Sr. No.	Existing Approach / System	Identified Limitation / Research Gap	Proposed Project's Approach
			back to a template
4	Web generation benchmarks (Design2Code, WebGen-Bench) [11, 12]	Evaluate generation only; no optimisation, forms, hosting or tenancy	Close the loop: generate, score, audit, improve, publish, collect forms and sessions
5	SEO factor studies [4]	Factors identified but not connected to an automatic, safe rewrite of generated pages	MLP SEO scorer with a gated LLM rewrite loop that keeps a change only if the score rises
6	Session models (GRU4Rec) [5]	Predict next items from real logs; no use for layout decisions before traffic exists	GRU conversion model with counterfactual section reordering, usable before a site has traffic
7	Rule-based design checkers and anomaly detectors [6]	Rules miss uncatalogued flaws; detectors are not applied to page designs	Rubric plus autoencoder and Isolation Forest trained on clean templates
8	Website builders' backends	Forms, storage and deployment recovery are left to the user	Auto-provisioned Firestore forms and an agentic deployment pipeline with diagnosis, safe retries and a circuit breaker

The most significant gap is the absence of a website platform that decides **per task** whether a language model is needed at all, runs the model it does need locally and reliably, and measures what happens after generation. Existing routers optimise which LLM to call, existing benchmarks stop at the generated code, and commercial builders neither publish their routing nor measure the pages they produce. Closing this gap matters for cost, for tenant privacy and for quality, and it justifies a project that combines a routed AI stack with post-generation engines and backend automation, evaluated end to end.

2.4 Conclusion

The literature shows that each building block of Sitezy is individually well understood: routing and cascades reduce LLM cost, grammar-constrained decoding produces structured output, on-page SEO factors are measurable, GRUs model sessions, Isolation Forests and autoencoders detect anomalies without labels, and entailment models classify text zero-shot. The common limitations are that these techniques are studied in isolation, assume cloud models or labelled data, and stop at generation. The research gaps in Section 2.3, namely task-typed local-first routing, reliable local structured generation, gated post-generation optimisation and automated backend provisioning in a multi-tenant setting, directly shaped the objectives in Section 1.3 and the choice of technologies. The next chapter describes the methodology and architecture designed to close these gaps.

CHAPTER: 3

PROPOSED METHODOLOGY

Chapter 3
Proposed Methodology

3.1 Introduction

Sitezy is a multi-tenant platform in which a business describes itself in plain language and receives a complete, editable, optimised and deployable multi-page website. The proposed system has three tiers: a React single-page application for tenants and visitors, a Node.js/Express API that holds the business logic, the AI router, collaboration, versioning and the deployment agents, and a Python Flask service with four small learned engines. A locally served language model handles open-ended generation, and a cloud model is an explicit exception.

Development followed an iterative, prototype-driven methodology in six phases (Fig. 3.1): requirement analysis from existing builders, user needs and the literature; system design of the three-tier architecture, data model and UML; iterative development of frontend, backend and generation tier; development of the machine-learning engines on synthetic data; testing and evaluation through repeated runs, ablations, fault injection and security tests; and deployment and reporting. Evaluation results fed back into development: two access-control defects, an integration defect in the block recommender and empty forms from the local model were all found by the evaluation and fixed. The methodology therefore serves the objectives directly: every objective in Section 1.3 has a corresponding module and a corresponding measurement.

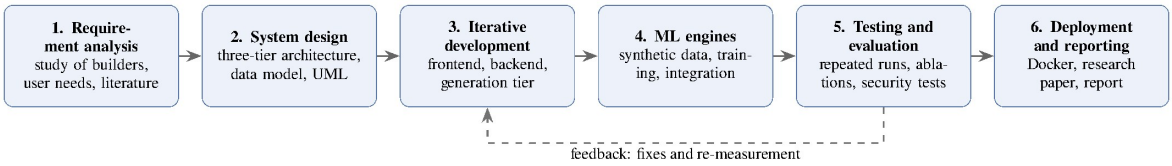


Figure 3.1: Development methodology with the evaluation feedback loop

3.2 Requirement Gathering

Requirements were collected from several complementary sources and refined at each iteration.

3.2.1 Requirement Gathering Approach

- **Study of existing systems.** We used template and AI website builders [1, 2] as small-business owners would, noting where a non-technical user gets stuck: writing copy, SEO settings, connecting forms, domains and hosting.
- **Literature survey.** The reviewed research (Chapter 2) defined the technical requirements for routing, structured generation and post-generation engines.

- **Prototype feedback.** The first prototype, built at the SPIT Hackathon 2026, was demonstrated and discussed with mentors, peers and our project guide; each later iteration was reviewed with the guide.
- **Observation of usage.** Usage logs from development tenants (251 generation requests from 19 tenants, February to September 2026) showed, for example, that 38% of cloud-only requests failed on rate limits and credentials, which motivated local serving.
- **Evaluation-driven requirements.** Experiments revealed new requirements, such as validating and regenerating incomplete forms and gating every automatic rewrite.

3.2.2 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Description
FR1	Tenant registration and login	Register an organisation (tenant) with a unique slug and an owner account; sign in with e-mail, password and tenant; JWT session.
FR2	Team and roles	Invite members and assign one of four roles (Owner, Admin, Editor, Developer) that gate billing, domains, publishing and editing.
FR3	Website management	Create, list, rename and delete websites within the plan's website limit.
FR4	AI website generation	Generate a complete multi-page site (Home, About, Contact or similar) from a brief, with tone, audience, theme, purpose, colours, base template and required blocks; Basic (local) or Pro (cloud) AI.
FR5	Multilingual and voice input	Accept briefs in English, Hindi or Marathi, typed or spoken, translated before generation.
FR6	AI auto-configure	Infer template, tone, audience, theme, purpose and brand colours from the description.
FR7	Visual builder	Edit sections inline, reorder by drag and drop, style colours and fonts, add and delete sections and pages, AI “Add Page” with automatic navbar linking.
FR8	Dynamic forms	Build forms with text, e-mail, phone, number, date, text area, drop-down, radio and checkbox fields; store submissions in Firestore.
FR9	Collaboration and versions	Show live collaborators and cursors; create commits (snapshots) of a page and restore earlier versions.
FR10	SEO analysis and auto-improve	Score a page on 15 factors, explain weak factors, rewrite weak copy without lowering the score, and generate meta description and JSON-LD.
FR11	Engagement and design insights	Predict engagement readiness and conversion, suggest section moves with predicted gains, and audit design health with one-click fixes.
FR12	Publishing and rollback	Publish a website as an immutable numbered snapshot on a subdomain or custom domain; roll back to an earlier snapshot.
FR13	Custom domains and export	Add, verify, rename and open custom domains; export the site as an exact static code bundle.
FR14	Agentic deployment	Deploy to Firebase Hosting with automatic provisioning of the form backend, error diagnosis, safe retry and a circuit breaker.
FR15	Analytics and insights	Show live and total visitors, the form-submission inbox and AI-generated business insights from form data.
FR16	Public site and chatbot	Serve published pages to visitors with working forms and an AI assistant grounded in the site's content.
FR17	Subscriptions	Offer FREE, BASIC, PRO and ENTERPRISE plans, take payment through Razorpay and enforce plan limits.
FR18	Help assistant and activity log	Answer how-to questions in the dashboard and builder; record tenant activity.

3.2.3 Non-Functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement
Security	JWT authentication with seven-day expiry; bcrypt password hashing (12 rounds); Helmet headers; rate limiting; role checks on mutating routes; every tenant-owned document carries a tenantId used in every query.
Privacy	Visitor and user-submitted data is redacted (names, e-mail addresses, phone numbers, addresses) before any cloud model call; local inference keeps data on the host.
Reliability	Every generated layout is validated against a schema with a deterministic template fallback; every learned engine degrades to a rule if its model artifact is missing; deployments are retried only when safe.
Performance	Tier-1 decisions in milliseconds (SEO score 0.32 ms, design audit 6.7 ms, suggestions 8 ms); a full three-page site generated locally in about 77 s on a laptop GPU.
Scalability	Stateless API behind JWT; tenant-scoped collections with indexes; separate ML service; Docker image; Prometheus metrics for monitoring.
Cost	Marginal generation cost close to the GPU's energy use; cloud usage only as an explicit exception.
Usability	Browser-only use; every AI suggestion explained and applied with one click; in-app help assistant; responsive previews for desktop, tablet and mobile.
Maintainability	Modular code (12 backend modules, feature folders in the frontend); training scripts regenerate model artifacts deterministically; evaluation scripts and seeds released.

3.2.4 User Requirements

Table 3.3: User requirements by user class

User class	Needs
Owner / Admin	Create a workspace, generate and publish sites quickly, manage team members, roles, domains and subscription, and see analytics and form submissions.
Editor / Developer	Edit pages visually, collaborate in real time without overwriting others, keep versions, improve SEO and design with guidance, and export code.
Site visitor	Fast, readable pages on any device, working forms and quick answers from the site's assistant.
Platform operator	Low marginal cost per site, observability, tenant isolation and safe automatic deployment.

3.3 Proposed Algorithm

The proposed system addresses the research gaps of Section 2.3 with a set of cooperating algorithms. Fig. 3.2 shows the architecture: bounded tasks go to Tier 1, open-ended generation to Tier 2 and the cloud Tier 3 only on the Pro tier or a local failure, while the orchestrator publishes sites and provisions their backends. The key features are listed below, followed by the algorithms.

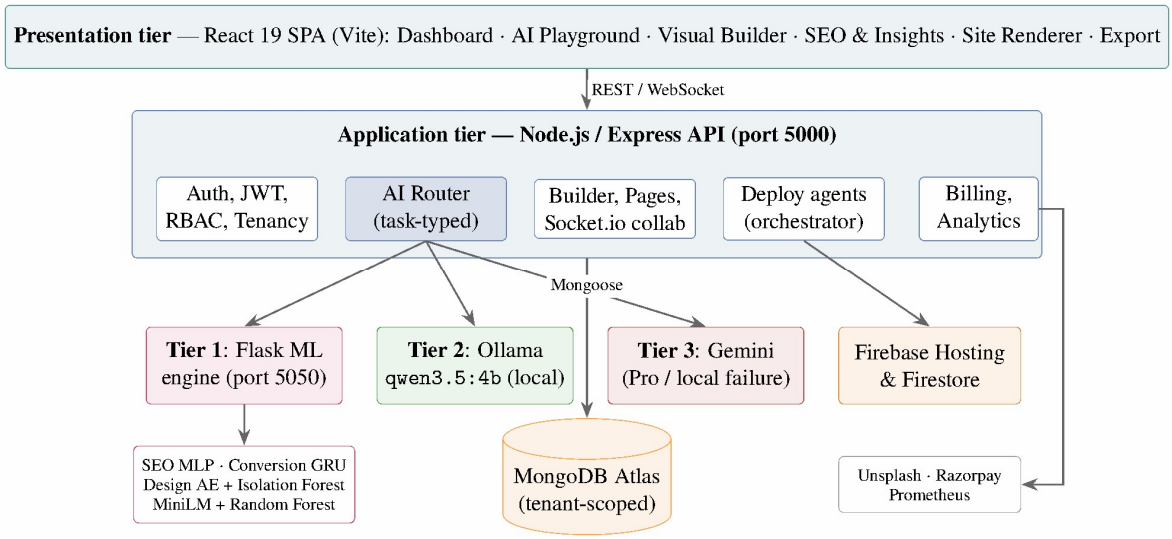


Figure 3.2: Three-tier architecture of Sitezy with the AI router and deployment orchestrator

- **Task-typed routing (gaps 1 and 2).** A rule-based router assigns each AI task by type; escalation follows observable events, never a learned confidence threshold, so it needs no calibration data and remains auditable.
- **Reliable local generation (gap 3).** Constrained JSON decoding with the reasoning mode disabled, schema validation, a deterministic template fallback and targeted regeneration of incomplete forms.
- **Post-generation engines (gaps 4 to 7).** An MLP SEO scorer with a gated rewrite loop, a GRU conversion model with counterfactual suggestions, an autoencoder plus Isolation Forest design auditor and a zero-shot block recommender.
- **Backend automation (gap 8).** Snapshot publishing with rollback, automatically provisioned Firestore forms and an agentic deployment pipeline.

Expected improvements over existing systems: near-zero marginal generation cost and no off-host data for local generation, schema-valid output in almost every case, page quality that can only rise under automatic rewriting, insights available in milliseconds, working forms without tenant-written backend code, and verified tenant isolation.

3.3.1 Task-Typed Routing with Deterministic Escalation

The router (Algorithm 3.1) decides where each request goes. Bounded tasks have a small, fixed output space and are answered by Tier-1 models; open-ended generation goes to the local model unless the tenant has selected the Pro tier. A runtime error of the local model (unreachable, timeout) escalates the same prompt to the cloud model. Invalid output from either model is replaced by a deterministic template layout and never triggers a second generation call, which bounds cloud spend.

Algorithm 3.1: Task-typed routing with deterministic escalation

```
Input : request r with task type T, tenant plan tier, prompt P
Output: response and the tier that produced it
1  if T in {recommend, score, audit, predict} then
2      if model artifact for T is loaded then return Tier1Model_T(r)
3      else return Rule_T(r)                      // deterministic fallback, never cloud
4  if tier = PRO then (out, tier) <- (Cloud(P), 3)
5  else
6      try   (out, tier) <- (Local(P, format=json, think=false), 2)
7      catch runtime error (unreachable, timeout) -> (out, tier) <- (Cloud(redact(P)), 3)
8  L <- extractJSON(out)
9  if L cannot be parsed or fails the layout schema then L <- TemplateLayout(business, tone,
     audience)
10 L <- RepairForms(L)                            // Algorithm 3.3
11 log (tenant, tier, success, tokens) to AIUsageLog
12 return (L, tier)
```

3.3.2 Constrained Local Generation

A three-page site needs about 4,000 output tokens. Tier-2 calls therefore set a JSON grammar (format: json), disable the reasoning mode (think: false) so that reasoning tokens cannot compete with the site for the same budget, and use a 5,000-token output budget, an 8,192-token context and temperature 0.1 (Algorithm 3.2). The prompt contains the brief, brand facts, recommended blocks and colour rules.

Algorithm 3.2: Constrained local generation of a site layout

```
Input : brief B (business, tone, audience, purpose, colours, language), recommended blocks K
Output: validated layout L = {theme, pages[ ] -> sections[ ]}
1  if language(B) != English then B <- translate(B)
2  P <- buildPrompt(B, K, brand facts, colour rules, section schema)
3  out <- Ollama.chat(model=qwen3.5:4b, system msg, P, format=json, think=false,
                      num_predict=5000, num_ctx=8192, temperature=0.1)
4  L <- JSON.parse(stripCodeFences(out))      // on failure: TemplateLayout
5  validate L with the Joi layout schema       // on failure: TemplateLayout
6  for each section s with an image query: s.image <- Unsplash(query, unique)
7  return L
```

Validity does not imply completeness: the local model sometimes returned a valid form with an empty field list. Algorithm 3.3 repairs such forms with a short, targeted Tier-2 call.

Algorithm 3.3: Field validation and targeted form regeneration

```
Input : layout L, business description D
1  for each form section F in L do
2      V <- { f in F.fields : f has name, label, allowed type,
                              >= 2 options if select, unique name }
3      if |V| < 3 then
4          G <- Local(short prompt: 5-8 fields specific to D, format=json, think=false)
5          V' <- valid fields of G
6          if |V'| >= 3 then F.fields <- V' else F.fields <- FixedFields(D)
7      else F.fields <- V
8  return L
```

3.3.3 MLP SEO Scorer and Gated Rewrite Loop

An extractor turns a page into 15 on-page features x (title and meta-description length, keyword density, word count, readability, image alt-text coverage, link and heading counts and binary flags such as a single H1 or structured data). An MLP maps them to a score:

$$s = 100 \cdot \sigma(W_3 \, \varphi(W_2 \, \varphi(W_1 x + b_1) + b_2) + b_3)$$

with hidden widths 64 and 32, ReLU activation φ and dropout 0.2, trained with Adam and mean-squared error. The improvement loop (Algorithm 3.4, Fig. 3.7) only keeps a language-model rewrite when the score rises.

Algorithm 3.4: Gated SEO rewrite loop

```
Input : page G, optional target keyword k, threshold tau = 80
1  s <- SEO(G); i <- 0
2  while s < tau and i < 3 do                    // stage 1: gated LLM rewrite
3      W <- up to 3 weakest text-fixable factors of G
4      patches <- Local(W, existing text fields of G, format=json, temperature=0.4)
5      G' <- apply patches to existing fields only; s' <- SEO(G')
6      if s' > s then (G, s) <- (G', s'); i <- i + 1
7      else discard patches; break
8  G' <- insert k once into a body field          // stage 2: keyword, same gate
9  if SEO(G') > s then (G, s) <- (G', SEO(G'))
10 if meta description or JSON-LD factors are weak then
11     G <- add meta description and JSON-LD      // stage 3: deterministic
12 return G, s
```

3.3.4 GRU Conversion Model and Counterfactual Suggestions

A visitor session is a sequence of up to 20 steps, each with 12 features (a 7-way section-type one-hot, dwell time, scroll depth, CTA click, form start and form completion). Two GRU layers [13] with 64 and 32 units, whose state update is

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

with update gate z_t, feed a small dense head and a sigmoid that outputs the conversion probability p. Before a site has traffic, a session is synthesised from its section order. Suggestions are counterfactual (Algorithm 3.5): the page is re-scored with a section moved and the predicted change Δp is reported.

Algorithm 3.5: Counterfactual section-reordering suggestions

```
Input : ordered section types S = (s1, ..., sn) of a page
1  p0 <- GRU(sessionFrom(S)); C <- {}
2  if a CTA sits more than one place below the Hero then
3      S' <- S with the CTA moved just after the Hero
4      C <- C + (move CTA, GRU(sessionFrom(S')) - p0)
5  for each Text or Gallery section s above the Footer do
6      S' <- S with s moved to just before the Footer
7      dp <- GRU(sessionFrom(S')) - p0
8      if dp > 0.012 then C <- C + (move s, dp)
9  if S has no CTA then C <- C + (info: add a call-to-action)
10 if S has no form then C <- C + (info: add a contact or order form)
11 return p0 and the top 5 of C by predicted gain, each with an Apply button
```

3.3.5 Design Auditor

A page is encoded as a 20-dimensional feature vector f in [0, 1] (section and CTA counts, heading hierarchy, WCAG contrast [21], font and colour diversity, alt coverage, duplicate-heading and empty-section ratios, structural order). A rubric of eleven checks with severity penalties (high 22, medium 11, low 5) yields the health score shown to users; a symmetric autoencoder (20-14-8-4, mirrored) and an Isolation Forest [6] with 200 trees, both trained only on clean templates, provide an informational anomaly score (Algorithm 3.6).

Algorithm 3.6: Design health audit

```
Input : page G
1  f <- designFeatures(G)                          // 20 values in [0,1]
2  flaws <- [c for c in 11 rubric checks if c fails on G]
3  health <- clip(100 - sum of severity penalties of flaws, 0, 100)
4  a1 <- clip((||f - AE(f)||^2 - mu_AE) / (4 sd_AE), 0, 1)
5  a2 <- clip((mu_IF - IF.score(f)) / (4 sd_IF), 0, 1)
6  anomaly <- max(a1, a2)                          // informational signal
7  return health, flaws with one-click fixes, anomaly
```

3.3.6 Zero-Shot Block Recommender

A MiniLM cross-encoder trained for natural language inference (nli-MiniLM2-L6-H768) [19] classifies the brief zero-shot [18] over four label sets: 13 business types, 11 content types, 9 goals and 10 audiences. The labels feed one random forest [20] per block type (Algorithm 3.7).

Algorithm 3.7: Block recommendation

```
Input : brief text B
1  for each label set Lset in {business, content, goal, audience} do
2      label[Lset] <- argmax over l in Lset of P_entail(B, "This website is about l")
3  for each of the 12 block types b do
4      if RF_b(label) > threshold(business) then select b     // threshold 0.45-0.60
5  add blocks whose keywords occur in B
6  pad the selection to at least 4 blocks
7  return selected blocks (the LLM then fills this skeleton with copy)
```

3.3.7 Agentic Deployment Orchestrator

Publishing copies a site's pages into an immutable, numbered snapshot served on the tenant's subdomain or custom domain. The optional export to Firebase Hosting is run by an orchestrator that coordinates two agents (Algorithm 3.8): a FirebaseAgent that deploys pages and assets, applies security rules and provisions the form backend, and a DiagnosingAgent that classifies failures and decides whether a retry is safe.

Algorithm 3.8: Agentic deployment with diagnosis and circuit breaker

```
Input : website W of tenant t
1  if failures(t, last 5 minutes) >= 5 then reject   // circuit breaker open
2  for attempt <- 1 to 2 do
3      result <- FirebaseAgent.deploy(W)             // pages, assets, rules, form backend
4      if result.success then record Deployment and logs; return success
5      d <- DiagnosingAgent.classify(result.error)
6           // quota, permission, network, credentials, isolation, timeout, storage
7      re-check isolation, rules and quota; severity <- d.severity
8      if severity in {critical, high} or attempt = 2 then break
9      wait 2 s (exponential back-off)               // medium severity: retry once
10 record failure for the circuit breaker; return failure with diagnosis
```

3.4 Gantt Chart for Timeline

Fig. 3.3 shows the project plan in half-month periods across Semesters VI and VII. Problem identification and the literature survey began in January 2026; the core multi-tenant prototype was built at the SPIT Hackathon in February; the local LLM, multilingual input, site chatbot, analytics, dynamic forms and the ML recommender followed in April; the deep-learning SEO, engagement and design engines, multi-page sites, code export and the UI redesign in June; the research paper was prepared and submitted between July and September; and the evaluation suite, security and deployment tests, camera-ready paper and this report were completed in October, before the paper presentation in November 2026.

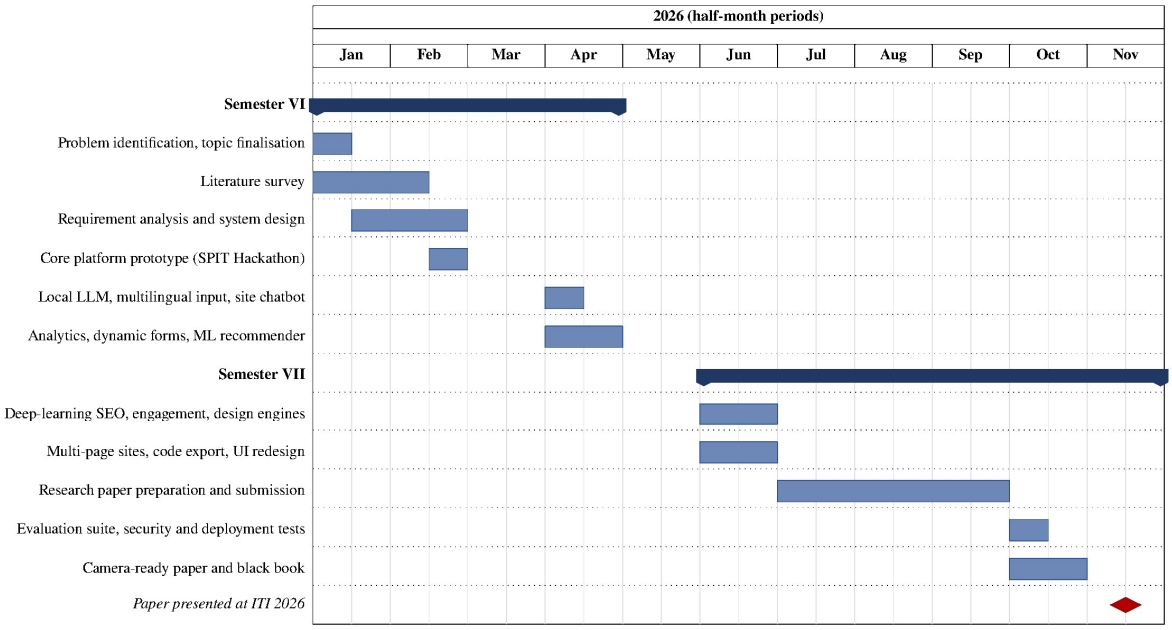


Figure 3.3: Gantt chart of the project timeline (2026)

3.5 Architectural Diagrams

3.5.1 UML Diagrams

Use case diagram. Fig. 3.4 shows the actors and their use cases. Owners and Admins manage the workspace, team, domains, subscription, publishing and analytics; team members (Editors and Developers) generate and edit sites, collaborate, analyse SEO, design and engagement and export code; site visitors browse published sites, submit forms and chat with the site assistant. The LLM tier, Firebase and Razorpay are external system actors.

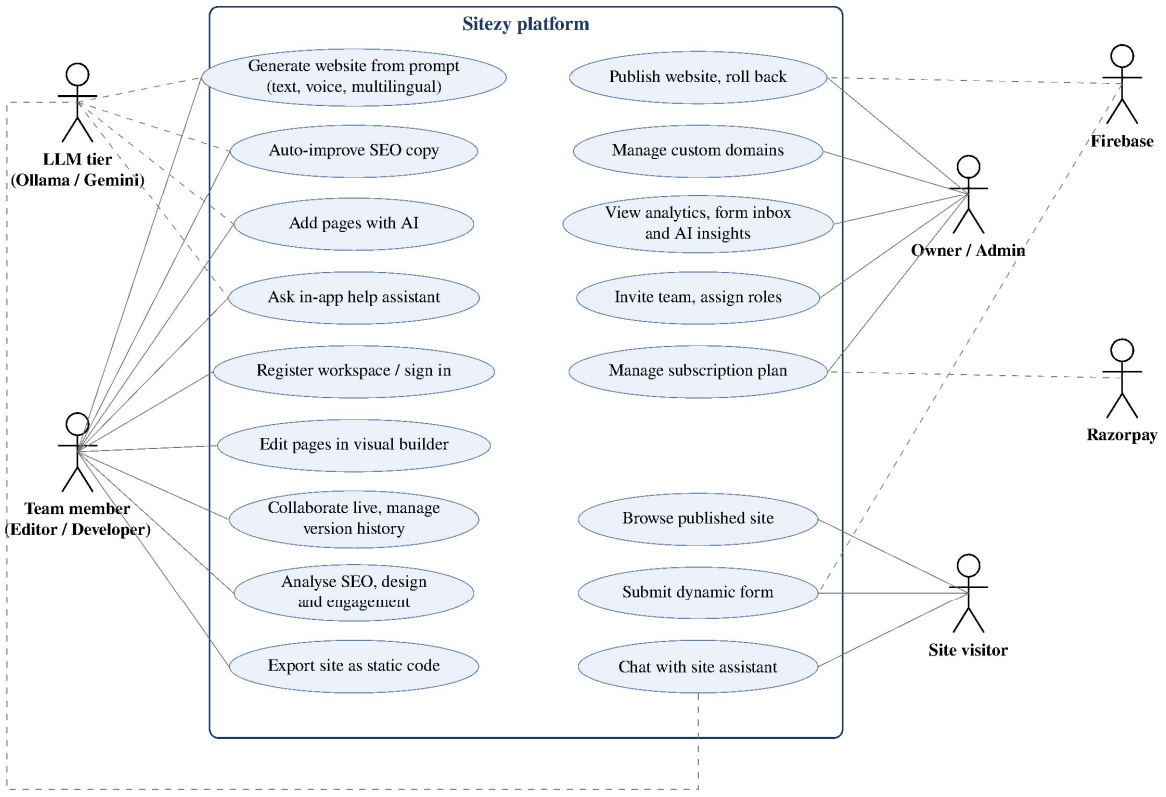


Figure 3.4: Use case diagram

Class diagram. Fig. 3.5 shows the main persistent classes and their multiplicities. A Tenant owns Users, Websites, Domains and a Subscription to a Plan; a Website has Pages, each Page has Commits (versions), and a Website has Deployments with DeploymentLogs and EngagementEvents; AIUsageLog records every AI call.

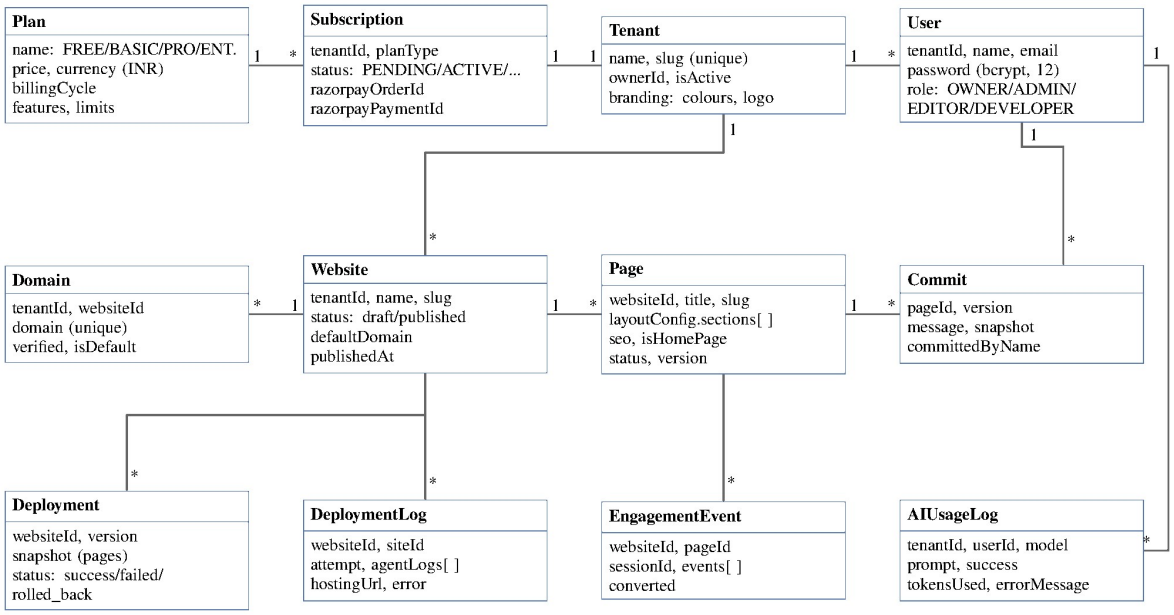


Figure 3.5: Class diagram of the main entities

Sequence diagram. Fig. 3.6 traces AI generation: the frontend posts the brief, the API checks the JWT, tenant scope and plan quota, obtains recommended blocks from the Flask engine, calls the local model (or, in the alternative fragment, the cloud model), resolves images, repairs forms if necessary, saves the website, pages and usage log, and returns the draft to the builder.

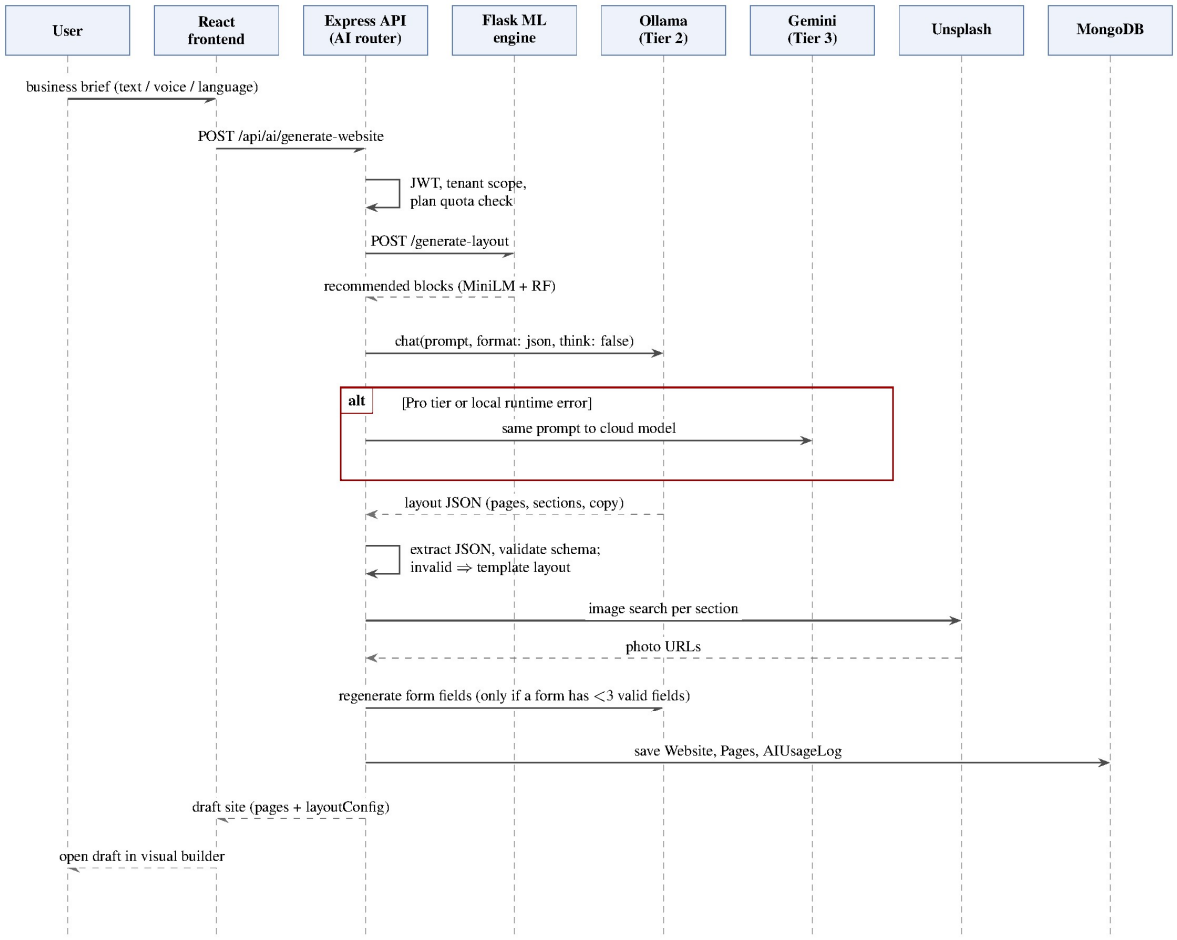


Figure 3.6: Sequence diagram of AI website generation

Activity diagram. Fig. 3.7 shows the gated SEO rewrite loop of Algorithm 3.4: a rewrite is kept only if the re-scored page improves, otherwise it is discarded and the loop stops.

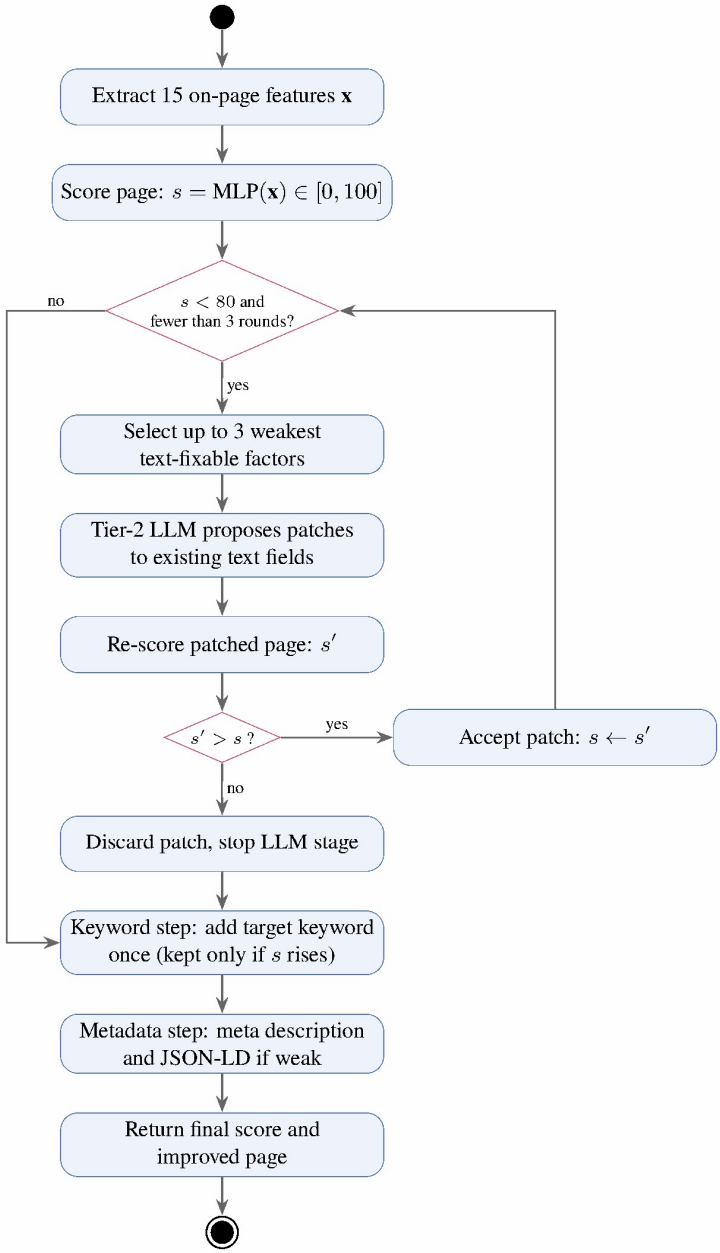


Figure 3.7: Activity diagram of the gated SEO auto-improve loop

3.5.2 Block Diagram

Fig. 3.8 is the Sitezy Workflow Diagram, the block diagram filed for copyright registration (diary number AT-40971/2026-CO, Appendix A.II). On the left, users (owners, editors, admins and visitors) reach the React 19 frontend, which talks over REST and WebSocket to the Node.js/Express backend API (JWT, RBAC, middleware and REST modules); the Socket.IO real-time layer, Razorpay billing and Prometheus/Grafana metrics hang off the backend. The **Generate** path leads into the AI generation subsystem: an AI generation controller drives the random-forest layout recommender, the tiered LLM (Ollama Qwen 3.5:4b with Gemini as cloud fallback), validation with a Joi schema and deterministic fallback, and an image resolver that fetches verified

Unsplash photos; the Flask ML engine on port 5050 supplies the SEO MLP, design autoencoder and Isolation Forest, engagement GRU, MiniLM zero-shot recommender and the closed-loop auto-improve. The **Publish** path leads into autonomous deployment: the orchestrator (bounded retry, exponential back-off, per-tenant circuit breaker, deploy log) coordinates the Firebase agent and the diagnosing agent and deploys to tenant-isolated Firebase (Firestore, Hosting, Storage). Tenant-scoped CRUD goes to MongoDB, and the published site is served back to the user.

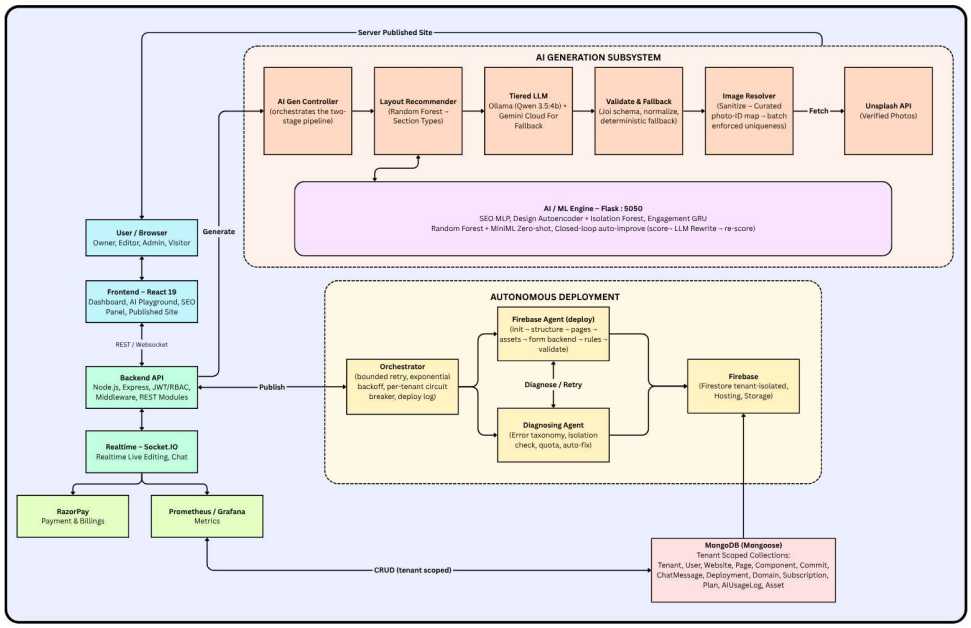


Figure 3.8: Block diagram: Sitezy Workflow Diagram (copyright application AT-40971/2026-CO)

3.5.3 Data Flow Diagram

The context diagram (Fig. 3.9) treats Sitezy as one process exchanging data with tenant teams, site visitors, the LLM services, Firebase and Razorpay. The level-1 diagram (Fig. 3.10) decomposes it into seven processes: authentication and tenant scoping, site generation, editing and versioning, analysis and optimisation by the Tier-1 engines, publishing and deployment by the agents, serving the public site with forms and chat, and subscription management; and five data stores: tenants and users, websites, pages and commits, AI usage and engagement logs, form submissions in Firestore, and deployments with their logs.

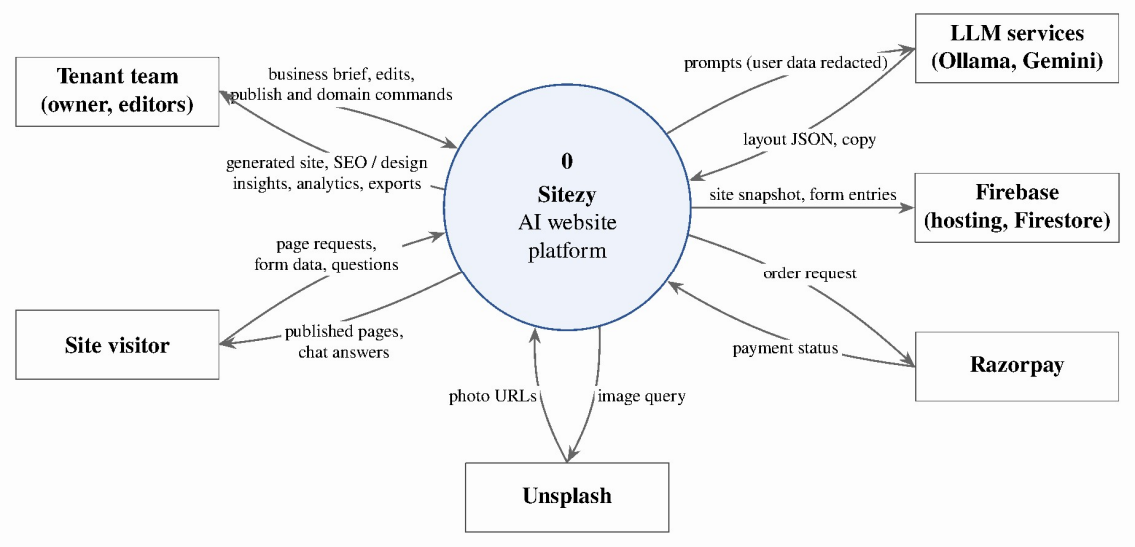


Figure 3.9: Data flow diagram, level 0 (context diagram)

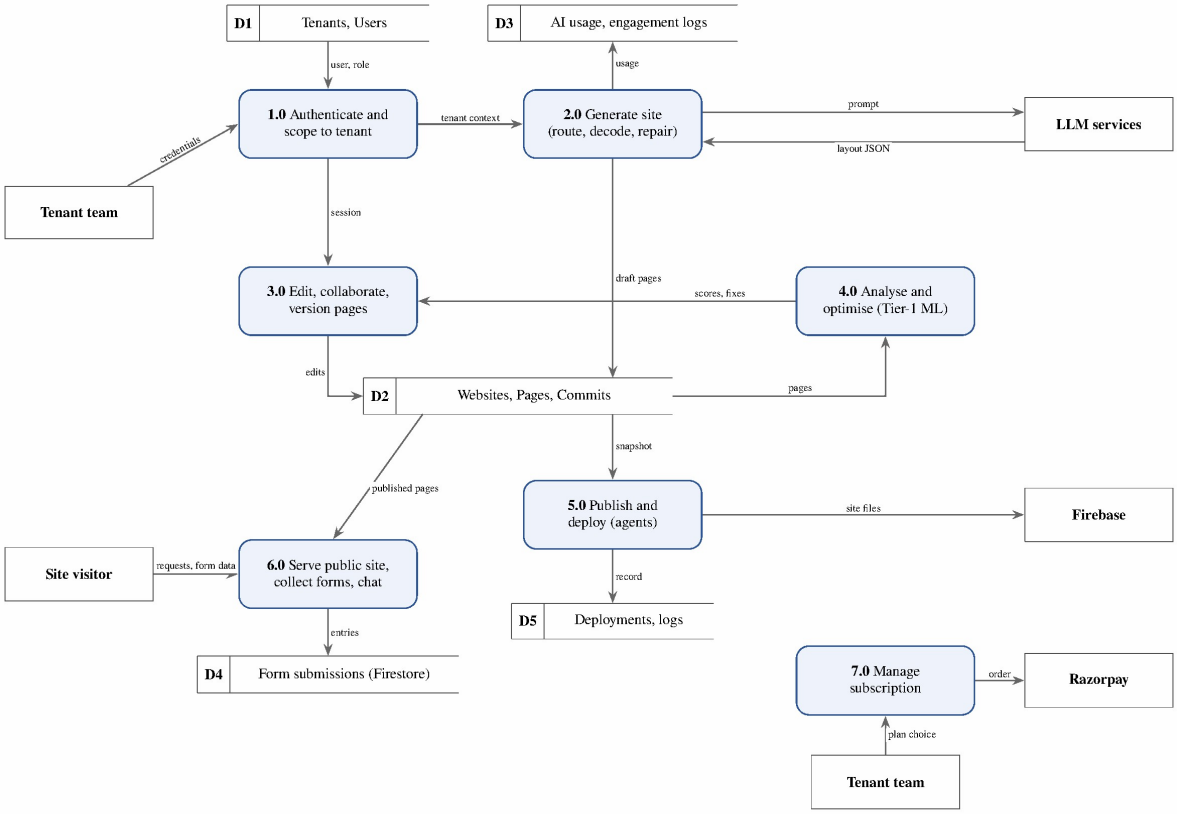


Figure 3.10: Data flow diagram, level 1

3.5.4 Timeline Chart

Fig. 3.11 summarises the project's milestones, from topic finalisation in January 2026 to the paper presentation at ITI 2026 in November 2026.

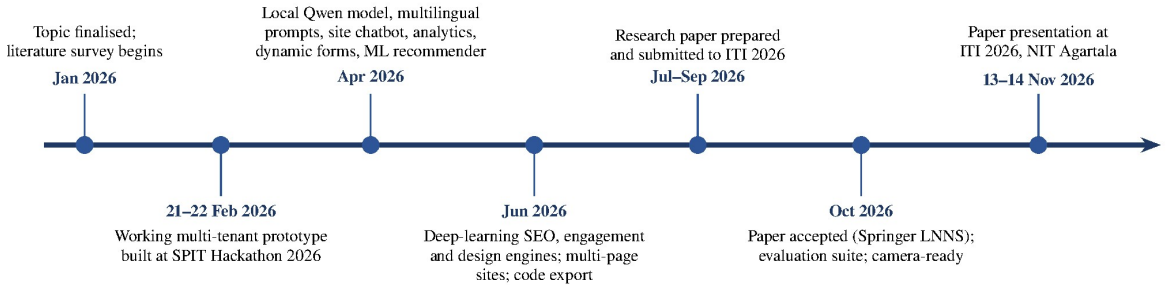


Figure 3.11: Timeline chart of project milestones

3.6 Hardware Requirements

Table 3.4: Hardware requirements

Component	Development and evaluation (used)	Recommended for deployment
Processor	AMD Ryzen 7 7840HS (8 cores)	4 or more cores (x86-64)
GPU	NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB VRAM	NVIDIA GPU with at least 8 GB VRAM (model uses about 6.1 GB)
Memory	16 GB RAM	16 GB RAM or more
Storage	SSD, about 20 GB for code, models, database and Docker images	SSD with at least 50 GB
Network	Broadband internet (APIs, Firebase, Unsplash)	Server-grade connection with a public IP / domain
Client device	Any laptop or desktop with a modern browser	Any device with a modern browser; microphone for voice input (optional)

3.7 Software Requirements

Table 3.5: Software requirements

Layer	Software and version
Operating system	Windows 11 (development); Linux or Docker for deployment
Frontend	React 19, Vite 7, Redux Toolkit 2, React Router 7, Tailwind CSS 4, @hello-pangea/dnd, socket.io-client, lucide-react, react-hot-toast
Backend	Node.js 22 (ES modules), Express 4, Mongoose 8, Socket.io 4, jsonwebtoken, bcryptjs, Joi, Helmet, express-rate-limit, prom-client, Multer, firebase-admin, Razorpay SDK
Database and storage	MongoDB (16 collections); Firebase Cloud Firestore (form submissions) and Firebase Hosting [23, 24]
Machine learning	Python 3.13, Flask 3, PyTorch 2.11, scikit-learn 1.7, Transformers 5.6, NumPy, Pandas [25, 26]
Language models	Ollama 0.21 serving qwen3.5:4b (Q4_K_M) [27, 28]; Google Gemini API, gemini-3-flash-preview [29]
External APIs	Unsplash (images), Razorpay (payments, test mode), Google Gemini
DevOps and tools	Docker, Prometheus, Git and GitHub, Visual Studio Code, Postman
Browser	Any current version of Chrome, Edge, Firefox or Safari

3.8 Cost Estimation

The cost estimate covers the development, testing and evaluation of the project from January to October 2026 (ten months). Costs are in Indian Rupees; resources already owned by the team are listed as existing resources at ₹0.

Table 3.6: Cost estimation

Sr. No.	Item / Resource	Description	Estimated Cost (₹)
1	Hardware	Four personal laptops, one with an RTX 4060 GPU used for local LLM inference and evaluation (existing resources)	0 (existing)
2	Software / Tools	VS Code, Git/GitHub, Node.js, Python, React, Express, PyTorch, scikit-learn, MongoDB Community, Ollama, Docker, Prometheus	0 (free/open-source)
3	AI models	Qwen3.5-4B open weights via Ollama; MiniLM NLI cross-encoder	0 (open weights)
4	Cloud / Hosting	Firebase Spark plan (Firestore, Hosting) and database during development, 10 months	0 (free tier)
5	APIs / Services	Gemini API for the cloud tier and baseline experiments (low volume, at list price); Unsplash API (free); Razorpay test mode (no fees)	500 (estimate)
6	Other Expenses	Electricity for local GPU inference and evaluation (assumed 150 GPU-hours at 60 W and ₹10/kWh, about 9 kWh); internet (existing)	100 (estimate)
	Total Estimated Cost		**≈ 600**

The overall cash cost of the project is therefore about ₹600, dominated by occasional cloud API calls and electricity, because the team used existing hardware and free or open-source software throughout. Conference registration for the research paper is a publication expense outside the project's development cost and is not included. For operation, the measured marginal cost is the more relevant figure: generating and optimising 1,000 sites with the full local pipeline costs about US$0.23 of GPU energy (about ₹20 at an assumed ₹88 per US$), against US$13.48 (about ₹1,190) with cloud generation (Section 4.4). Server rental, GPU amortisation and operations would be added for a production deployment.

3.9 Conclusion

This chapter translated the research gaps into a concrete methodology. Requirements were gathered from existing builders, the literature, prototype feedback and usage logs, and formalised as 18 functional and eight categories of non-functional requirements. The proposed system routes AI work by task type across three tiers and is realised through eight algorithms: deterministic routing, constrained local generation, form repair, a gated SEO rewrite loop, GRU counterfactual suggestions, a hybrid design auditor, a zero-shot block recommender and an agentic deployment orchestrator. UML, block, data-flow and timeline diagrams fix the architecture and development schedule, and the hardware, software and cost analysis shows that the system can be built on existing hardware with open-source tools for about ₹600. Together these provide the foundation for the implementation and evaluation in Chapter 4.

CHAPTER: 4

IMPLEMENTATION & RESULTS

Chapter 4
Implementation & Results

4.1 Introduction

This chapter describes how the methodology of Chapter 3 was turned into a working system and how well it works. The implementation consists of three services, a React frontend, a Node.js/Express backend and a Python Flask machine learning engine, together with a locally served language model, a cloud fallback and Firebase services. The major implementation activities were the multi-tenant core, the AI generation tier, the visual builder with collaboration and versioning, the four Tier-1 engines, publishing and the agentic deployment pipeline, and payments and monitoring.

The system was evaluated on every aspect the objectives name: validity, latency and cost of generation against cloud baselines; accuracy of the learned engines against stronger baselines, the deployed rules and the Bayes-optimal ceiling; behaviour on 66 real published tenant pages; robustness of the deployment pipeline under fault injection; tenant isolation; and an end-to-end ablation in which one component at a time is removed. All numbers were produced by executing the deployed code and are the same as those in our published paper [30].

4.2 Implementation

4.2.1 Project Structure and Technology Stack

The repository is organised into a frontend, a backend, an AI engine and an evaluation folder. Table 4.1 lists the three services; Docker packages them for production and Prometheus scrapes the backend's metrics.

Table 4.1: Services of the implemented system

Service	Stack	Port	Responsibility
Frontend	React 19, Vite 7, Redux Toolkit, React Router 7, Tailwind CSS 4, @hello-pangea/dnd, socket.io-client	5173	Landing page, authentication, dashboard, AI playground, visual builder, analytics, settings, subscription, published-site renderer, in-browser export
Backend API	Node.js 22, Express 4, Mongoose 8, Socket.io 4, JWT, Joi, Helmet, prom-client, Firebase Admin, Razorpay	5000	Authentication and tenancy, websites and pages, AI orchestration, domains, analytics, forms, publishing, deployment agents, payments, administration
AI engine	Python 3.13, Flask, PyTorch, scikit-learn, Transformers	5050	SEO MLP, GRU engagement model, design autoencoder and Isolation Forest, MiniLM + random-forest block recommender

4.2.2 Frontend

The frontend is a React 19 [31] single-page application built with Vite, using Redux Toolkit for global state (session, tenant, websites, builder state), React Router for navigation and Tailwind CSS for styling. It is organised by feature: *auth* (login and registration), *dashboard* (overview, websites, analytics, deployments, activity), *ai* (the AI playground), *builder* (page layers, section editors, live preview, SEO and insights panel, version history), *settings* (workspace, team members and custom domains), *subscription*, *publicSite* (the renderer that serves published pages, dynamic forms and the visitor chatbot) and *marketing/home* (the landing page). A shared section-component library renders every section type identically in the builder, the public site and the code export, which is how the export reproduces the live preview exactly. Drag-and-drop reordering uses @hello-pangea/dnd, and the Web Speech API provides voice input.

4.2.3 Backend Modules

The Express [32] backend is divided into twelve modules, each with its own routes, controller, validation and model: auth, tenant, website, builder (pages and commits), ai, domain, analytics, deployment, forms, payment, upload and admin. Middleware provides JWT authentication, tenant context and scope, role checks, subscription limits, logging and error handling; the agents folder holds the orchestrator, the FirebaseAgent, the DiagnosingAgent and the Firestore security rules. Listing 4.1 shows how the website routes combine the authentication chain with role checks.

Listing 4.1: Website routes: authentication chain and role checks (backend/src/modules/website)

```
router.get("/", authChain, getWebsites);
router.get("/:id", authChain, getWebsite);
router.post("/", authChain, requireRole("OWNER", "ADMIN"), createWebsite);
router.put("/:id", authChain, requireRole("OWNER", "ADMIN", "EDITOR"), updateWebsite);
router.delete("/:id", authChain, requireRole("OWNER", "ADMIN"), deleteWebsite);
router.post("/:id/publish", authChain, requireRole("OWNER", "ADMIN"), publishWebsite);
router.get("/:id/export", authChain, requireRole("OWNER", "ADMIN", "DEVELOPER"), exportWebsite);
router.post("/:id/rollback/:deploymentId", authChain, requireRole("OWNER"), rollback);
```

4.2.4 Data Model

MongoDB stores 16 collections through Mongoose schemas (Table 4.2); every tenant-owned document carries a tenantId. Form submissions are stored separately in Cloud Firestore under a tenant- and site-scoped path.

Table 4.2: MongoDB collections

Collection	Purpose
Tenant, User	Organisations (slug, branding, active flag) and their members (bcrypt password, role)
Plan, Subscription	FREE, BASIC, PRO and ENTERPRISE plans (₹0, 499, 999, 2,499 per month) and each tenant's Razorpay subscription
Website, Page, Component	Sites, their pages with layoutConfig sections and SEO data, and reusable components
Commit	Page versions (snapshot, message, author) for version history and restore
Deployment, DeploymentLog	Immutable published snapshots with status, and per-attempt agent logs
Domain	Subdomains and custom domains with verification status

Collection	Purpose
Asset	Uploaded images and files
ChatMessage	Team chat inside the builder
EngagementEvent	Visitor session events from published sites
AIUsageLog	Every AI call: tenant, user, model, success, tokens and errors
ActivityLog	Audit trail of tenant actions

4.2.5 AI Generation Tier

The AI controller implements Algorithms 3.1 to 3.3. A request first passes the authentication chain and the plan's AI quota, then obtains recommended blocks from the Flask engine, builds the prompt and calls the local model through the Ollama client (Listing 4.2). If the tenant selected Pro AI, or the local call throws, the same prompt goes to Gemini. The output is extracted, parsed and validated against a Joi layout schema, with a deterministic template layout as fallback; images are resolved from Unsplash, forms are repaired, and the website, pages and usage log are saved. The same pattern, a short JSON-mode call with reasoning disabled, is used for auto-configure, AI "Add Page", SEO rewrites, the visitor chatbot and the help assistant. Before any cloud call that includes user- or visitor-submitted data, personal fields are masked and e-mail addresses and phone numbers are removed from free text.

Listing 4.2: Local generation call with JSON grammar and reasoning disabled (ai.controller.js, abridged)

```
const r = await ollamaClient.chat({
    model: 'qwen3.5:4b',
    think: false,                       // reasoning tokens cannot exhaust the budget
    messages: [
        { role: 'system', content: 'You are an expert AI website generator ...' },
        { role: 'user', content: prompt }
    ],
    format: 'json',                     // JSON grammar-constrained decoding
    options: { num_predict: 5000, num_ctx: 8192, temperature: 0.1 }
});
text = r.message.content || '';
// on a runtime error: usedModel = 'gemini-3-flash-preview'; text = await cloud(prompt)
// then: extract JSON -> JSON.parse -> aiLayoutSchema.validate -> fallback layout if invalid
```

4.2.6 Machine Learning Engine

The Flask service exposes four endpoints: /generate-layout (block recommender), /seo-score, /design-health and /engagement-suggest. Each engine has a feature extractor, a training script that generates its synthetic training data deterministically and writes small artifacts (.pt, .pkl, .json), and an inference module that falls back to a rule when an artifact is missing. Listings 4.3 and 4.4 show the two PyTorch models.

Listing 4.3: SEO scorer: 15 normalised features to a score in [0, 100] (ai_engine/seo_engine.py)

```
class SeoMLP(nn.Module):
    def __init__(self, n_in=N_FEATURES):            # 15 features
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_in, 64), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 1), nn.Sigmoid(),
        )
    def forward(self, x):
        return self.net(x).squeeze(-1) * 100.0
```

Listing 4.4: Two-layer GRU conversion model (ai_engine/gru_engine.py)

```
class EngagementGRU(nn.Module):
    def __init__(self, dim=gf.DIM):                  # 12 features per step
        super().__init__()
        self.gru1 = nn.GRU(dim, 64, batch_first=True)
        self.gru2 = nn.GRU(64, 32, batch_first=True)
        self.head = nn.Sequential(nn.Linear(32, 32), nn.ReLU(), nn.Linear(32, 1), nn.Sigmoid())
    def forward(self, x, lengths):
        o1, _ = self.gru1(x)
        o2, _ = self.gru2(o1)
        last = o2[torch.arange(o2.size(0)), (lengths - 1).clamp(min=0)]
        return self.head(last).squeeze(-1)
```

The design engine combines the eleven-check rubric with a 20-14-8-4 autoencoder and a 200-tree Isolation Forest; the recommender loads the MiniLM NLI cross-encoder from Hugging Face Transformers and one scikit-learn random forest per block type. The training data are summarised in Table 4.3.

Table 4.3: Synthetic training data of the Tier-1 engines

Engine	Inputs	Label	Size
SEO scorer	15 uniformly sampled on-page features (title 0-90 characters, meta 0-220, keyword density 0-6%, 20-800 words, readability, alt coverage, link and heading counts, binary flags)	Weighted rubric of normalised features plus Gaussian noise (σ = 3), clipped to [0, 100]	20k train / 5k test per seed
Conversion GRU	Navbar, Hero, 2-6 shuffled sections, Footer; bounce 0.35; CTA click 0.55 if the CTA is in the first four sections, else 0.30; form start 0.5, completion 0.6; dwell time	Bernoulli outcome with a probability built from CTA click, early CTA, form start and completion, no bounce and section count	10k train / 4k test per seed
Design auditor	16 builder templates as 20-dimensional feature vectors, 130 Gaussian jitters each (σ = 0.04)	None (clean designs only)	2,096

4.2.7 Collaboration and Versioning

A Socket.io [33] server keeps one room per page and one per website. Clients join a page room and exchange content updates, typing indicators and cursor positions, autosave the page layout and chat with teammates. The socket is authenticated with the same JWT, and every room name is prefixed with the tenant ID taken from the verified token, so users of different tenants can never share a room. A commit stores a snapshot of a page's layout with a message and author, and restoring a commit writes the snapshot back as a new version.

4.2.8 Publishing, Forms and Agentic Deployment

Publishing copies every page of a website into a numbered Deployment snapshot that the public renderer serves on the tenant's subdomain or custom domain; rollback re-activates an earlier snapshot. Each home page carries a dynamic form of five to eight typed fields whose submissions (scalar values of at most 5,000 characters) are posted to the site's tenant-scoped Firestore path, so no tenant-written backend code is needed. The optional Firebase export is run by the orchestrator (Algorithm 3.8): a per-tenant circuit breaker opens after five failures in five minutes, the FirebaseAgent deploys pages and assets, applies the security rules and provisions the form backend, and after a failure the DiagnosingAgent classifies the error, re-checks isolation, rules and quota and allows a single retry after a 2 s back-off only for medium-severity errors.

4.2.9 Security, Payments and Monitoring

The authentication chain verifies the JSON Web Token [34] (seven-day expiry), rejects tokens whose tenant differs from the user's, checks that the tenant is active and attaches a tenant filter that every controller adds to its queries. Passwords are hashed with bcrypt (12 rounds), Helmet sets security headers and express-rate-limit throttles the API. Razorpay creates orders for plan upgrades and verifies payment signatures before a subscription becomes active, and the subscription middleware enforces each plan's limits on websites, AI usage and custom domains. prom-client exposes request and usage metrics for Prometheus.

4.3 Results of Implementation

The following screenshots were captured from the running system (local services, a throwaway database seeded with the project's demo tenant, and the local Qwen model). The demo brief describes an artisan coffee shop in Pune.

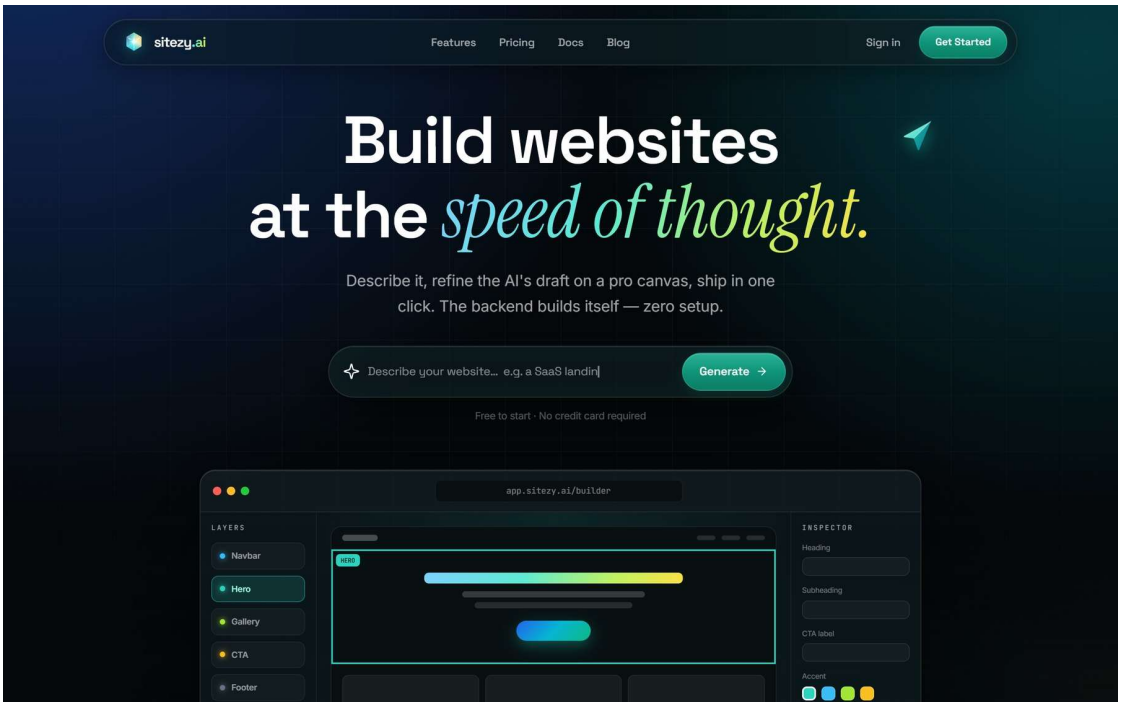


Figure 4.1: Landing page of Sitezy

The landing page (Fig. 4.1) introduces the platform and leads to registration. A new organisation registers a workspace with its own slug (Fig. 4.2) and signs in with e-mail, password and workspace.

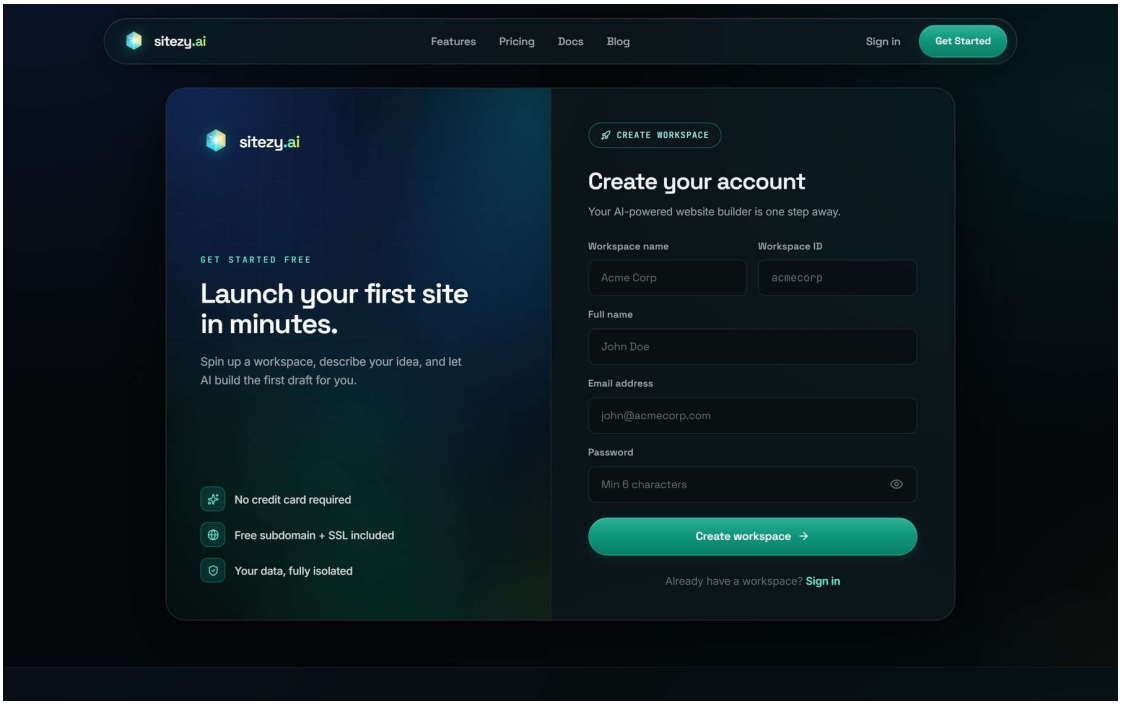


Figure 4.2: Workspace registration

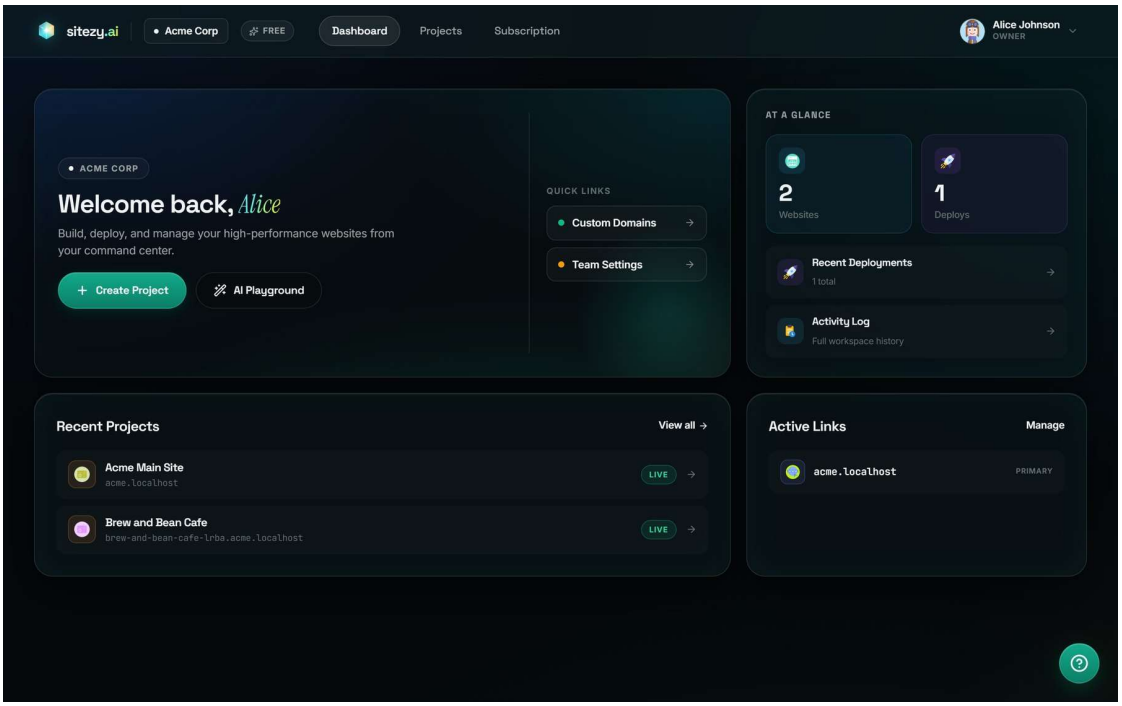


Figure 4.3: Tenant dashboard with usage, plan and quick actions

After sign-in the dashboard (Fig. 4.3) shows the workspace's websites, plan and usage, and the Projects page (Fig. 4.4) lists the tenant's websites with their status.

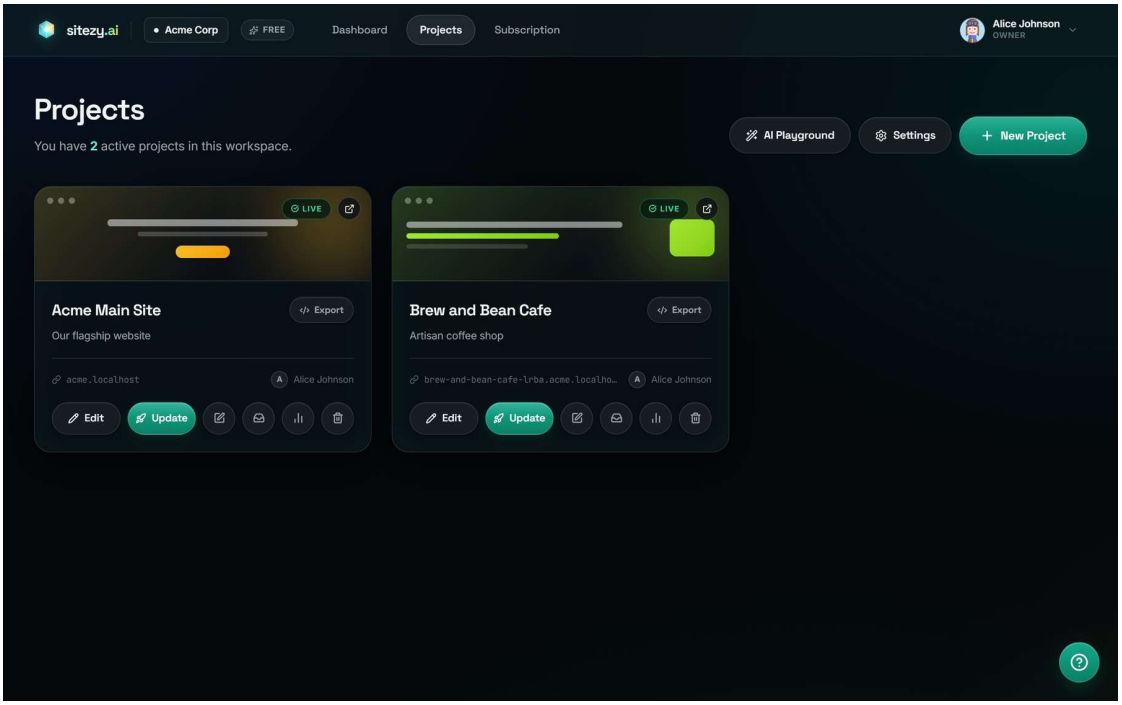


Figure 4.4: Projects page listing the tenant's websites

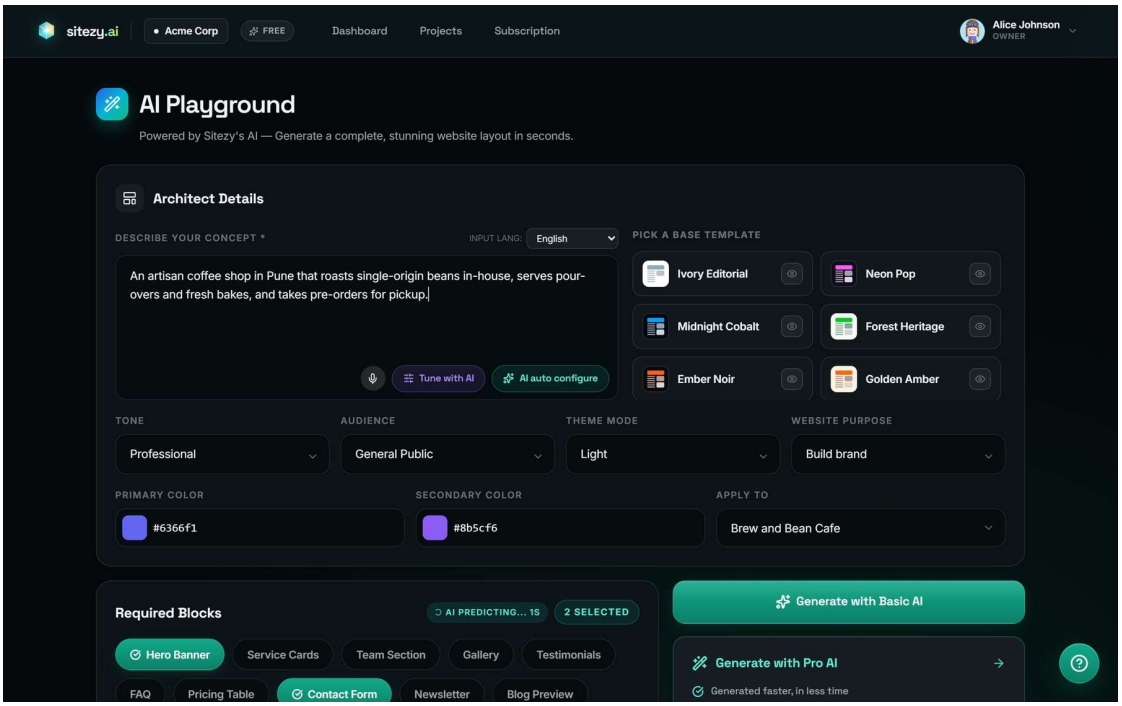


Figure 4.5: AI playground with the business brief, settings and target website

In the AI playground (Fig. 4.5) the user enters the brief, chooses Basic (local) or Pro (cloud) AI, tone, audience, theme, purpose, colours and required blocks, and selects the website to apply the result to. During generation the playground shows progress (Fig. 4.6); with the local model the three-page coffee-shop site was generated and saved, as confirmed in Fig. 4.7.

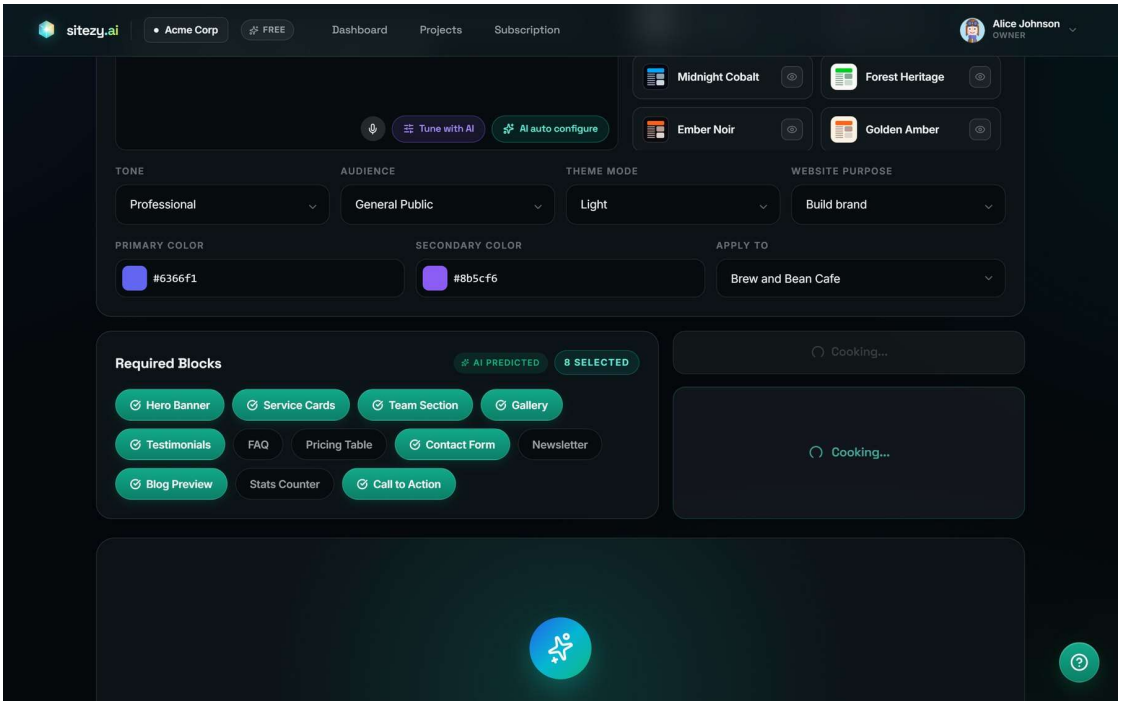


Figure 4.6: Generation in progress with the local model

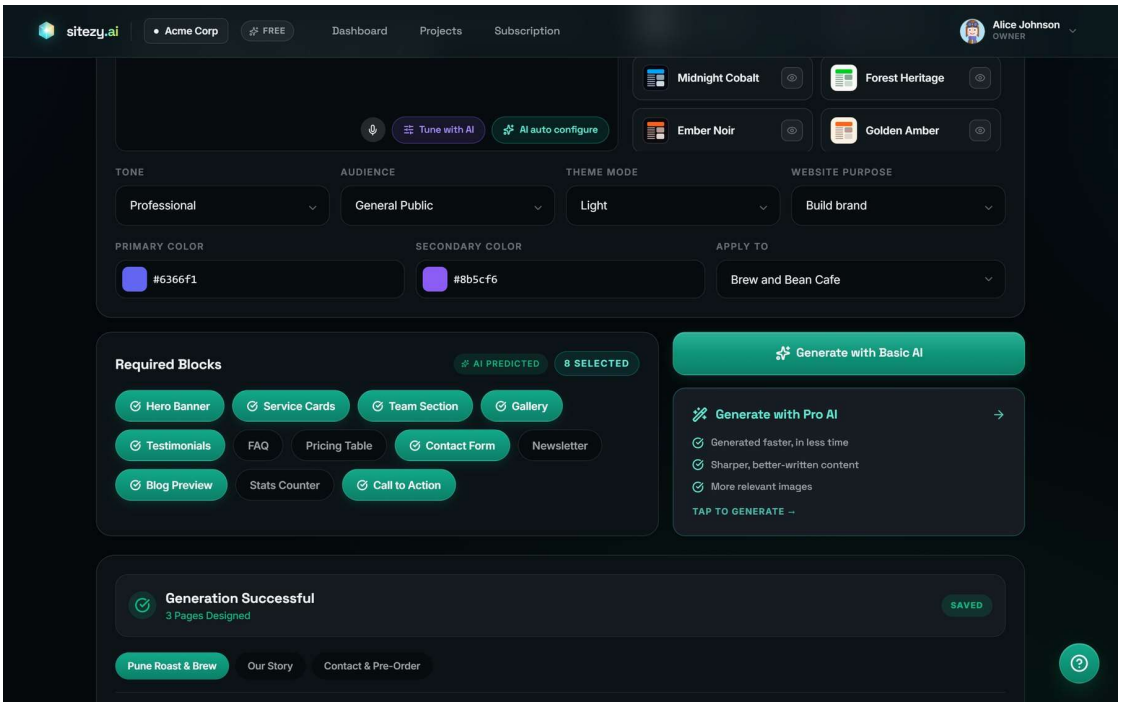


Figure 4.7: Generation result: three pages designed and saved

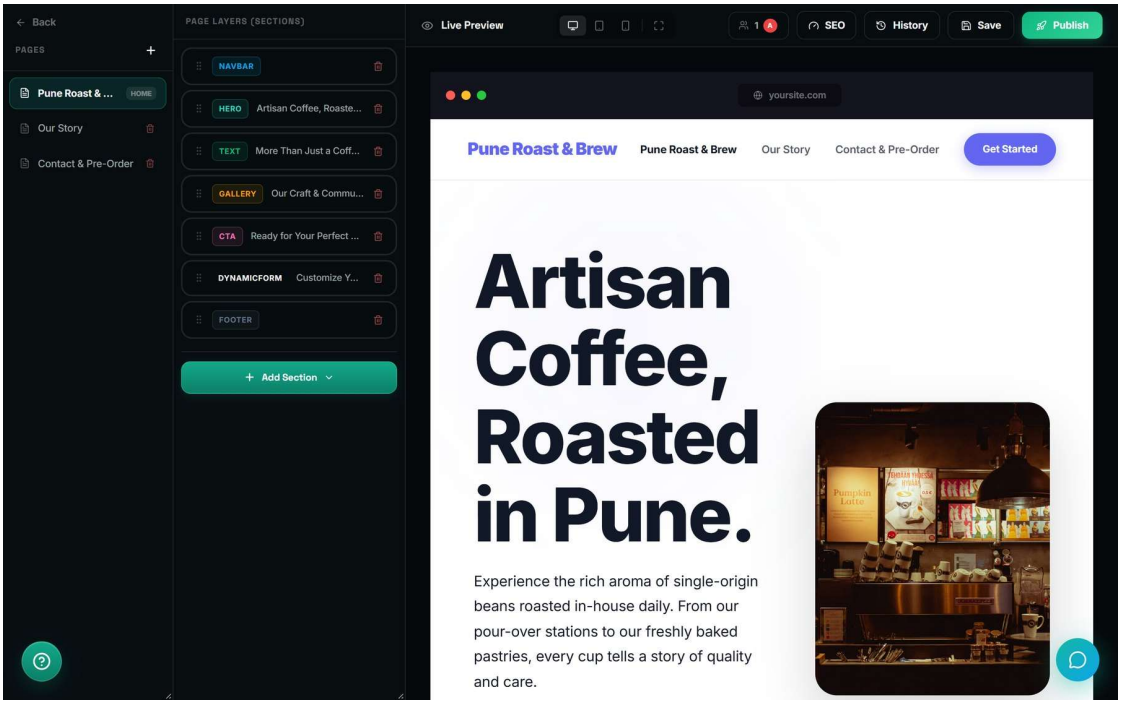


Figure 4.8: Visual builder with pages, section layers and live preview

The generated site opens in the visual builder (Fig. 4.8): pages on the left (Pune Roast & Brew home page, Our Story, Contact & Pre-Order), section layers (Navbar, Hero, Text, Gallery, CTA, DynamicForm, Footer) in the middle and a live, responsive preview, with collaborators, SEO, History, Save and Publish in the toolbar.

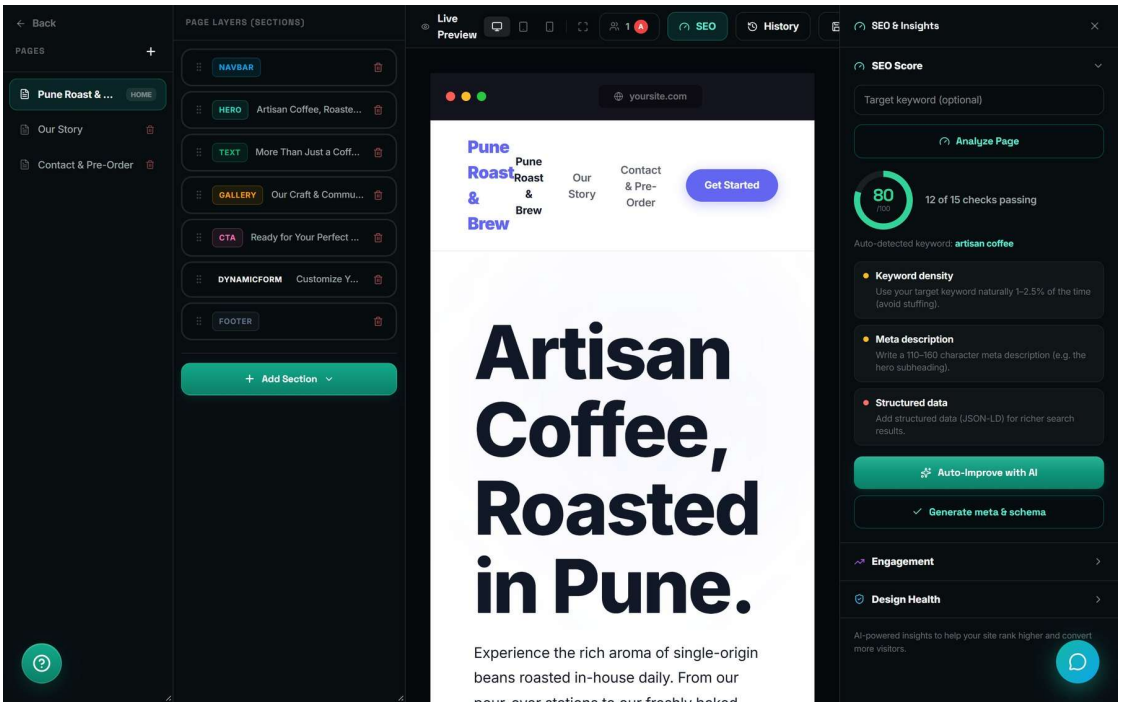


Figure 4.9: SEO & Insights panel: SEO score with factor diagnostics

The SEO & Insights panel (Fig. 4.9) scored the generated home page 80 out of 100, with 12 of 15 checks passing; the three weak factors (keyword density, meta description and structured data) are explained, and the Auto-improve with AI and Generate meta & schema buttons apply the gated rewrite and the deterministic metadata step. The engagement section (Fig. 4.10) rated the page's engagement readiness at 90, listed its strengths and proposed GRU-based reordering suggestions with their predicted gains, each with an Apply button. The design health scan (Fig. 4.11) found no design flaws and gave a health score of 100.

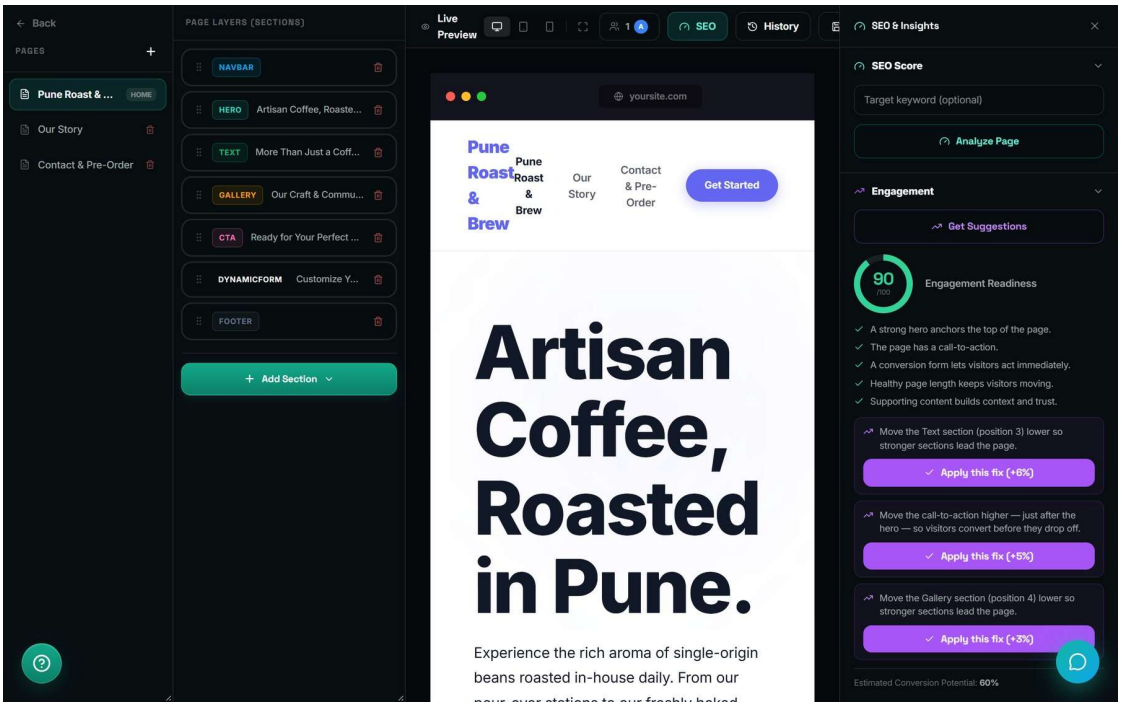


Figure 4.10: Engagement readiness and GRU reordering suggestions

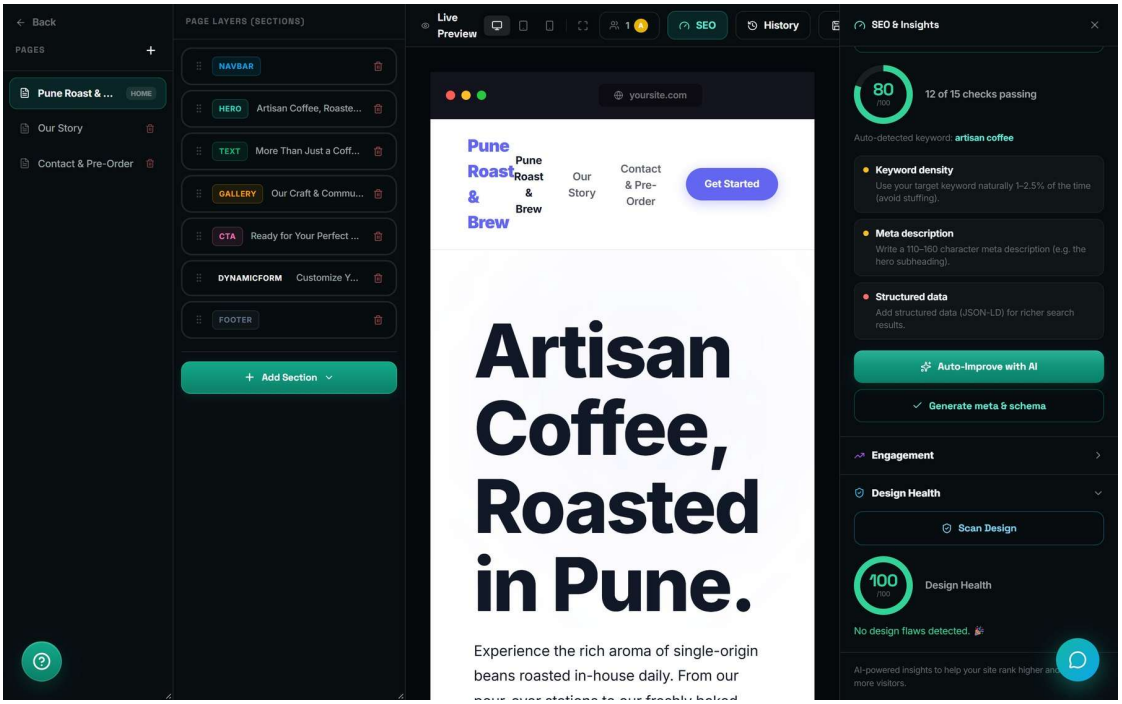


Figure 4.11: Design health scan

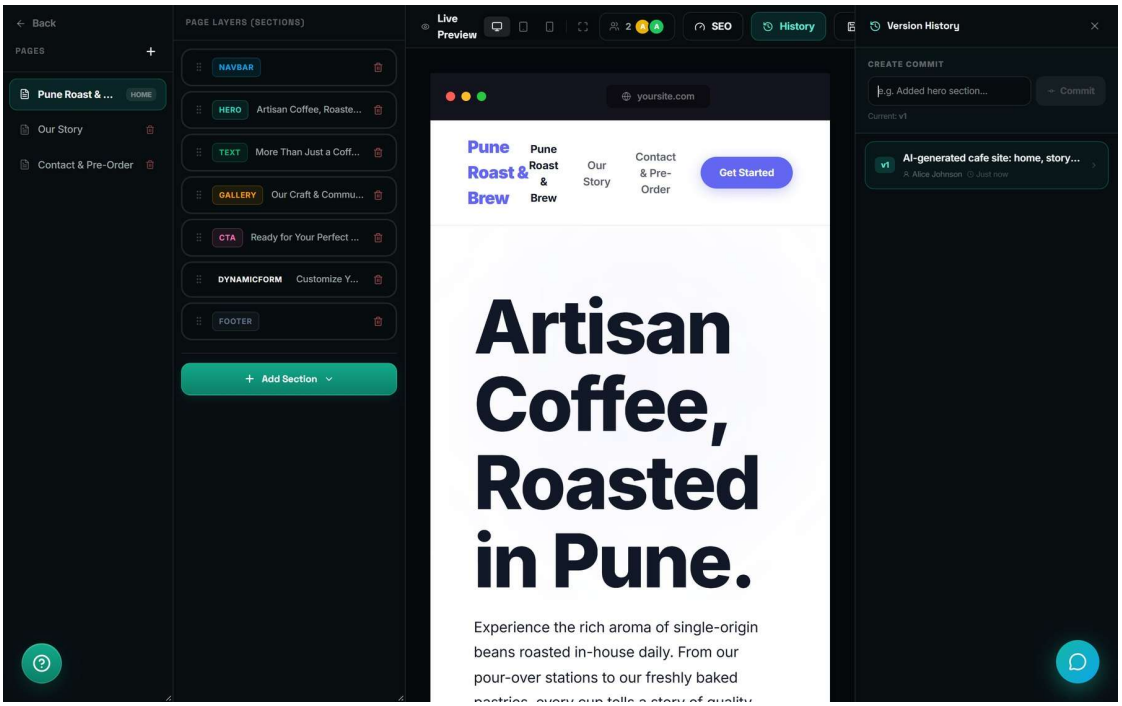


Figure 4.12: Version history after creating a commit

Version history (Fig. 4.12) records commits of a page, which can be restored later. Once published, the site is served to visitors by the public renderer (Fig. 4.13), with its pre-order form connected to Firestore and the site assistant in the corner.

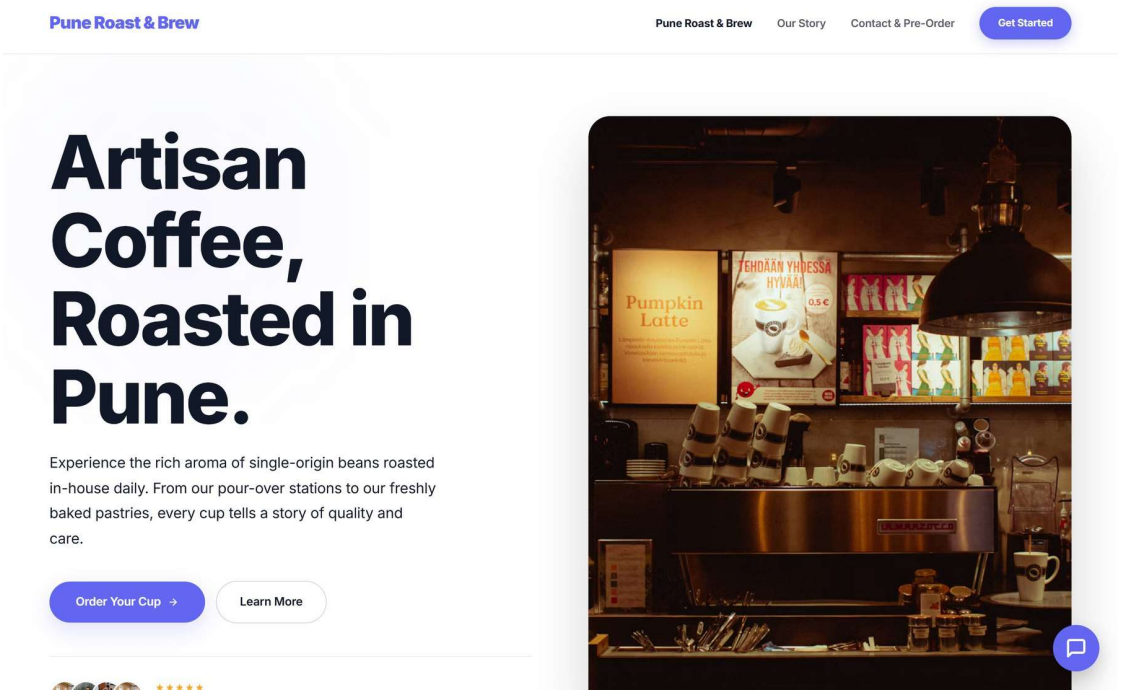


Figure 4.13: Published coffee-shop website served to visitors

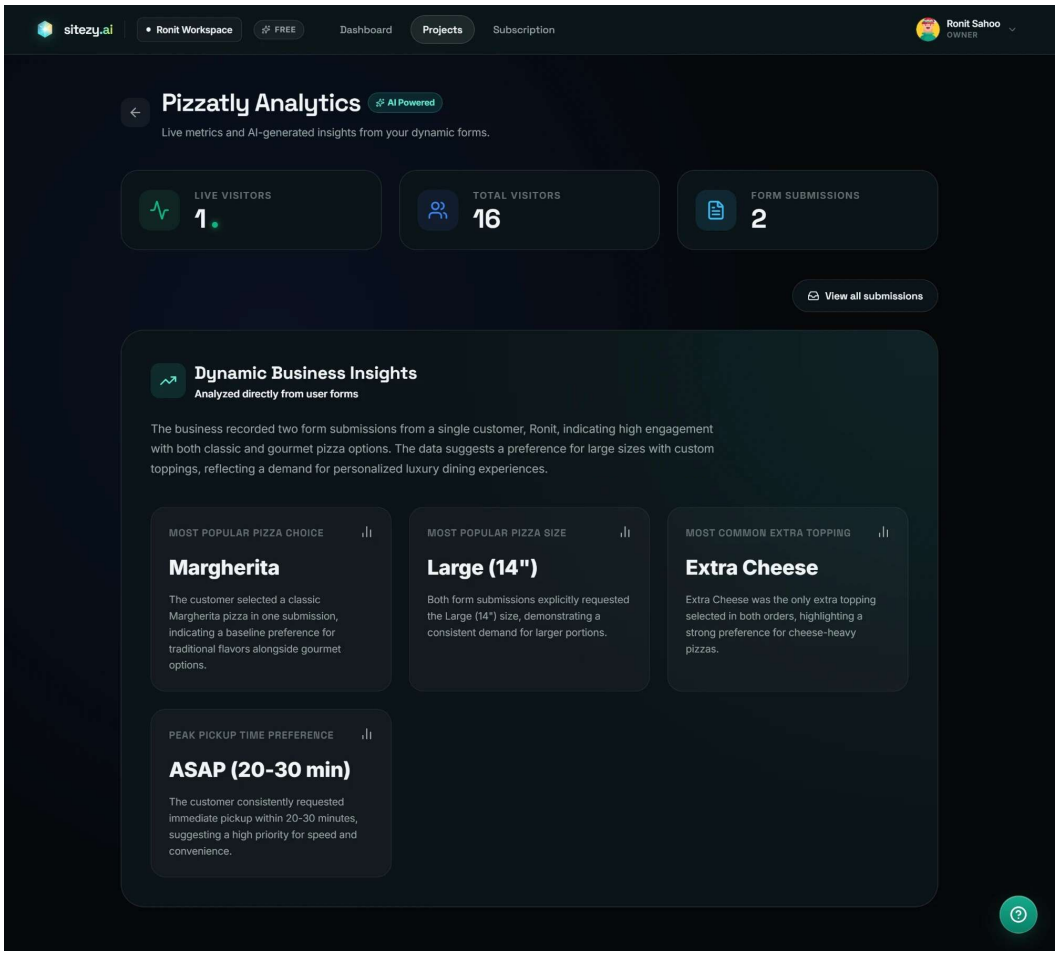


Figure 4.14: Project analytics: live visitors, total visitors, form submissions and AI-generated business insights

The analytics page (Fig. 4.14) is shown for Pizzatly, a pizza-ordering site in the team's own workspace. It combines live visitors counted through Socket.IO with total page views and form submissions stored in Firestore, and adds business insights that the language model derives from the submitted orders: a two-sentence summary and cards for the most popular pizza, the most popular size, the most common extra topping and the preferred pickup time. *View all submissions* opens the form inbox (Fig. 4.15), which lists every order with its typed fields and the tenant and site identifiers under which it is stored. The orders shown are test submissions made by the team.

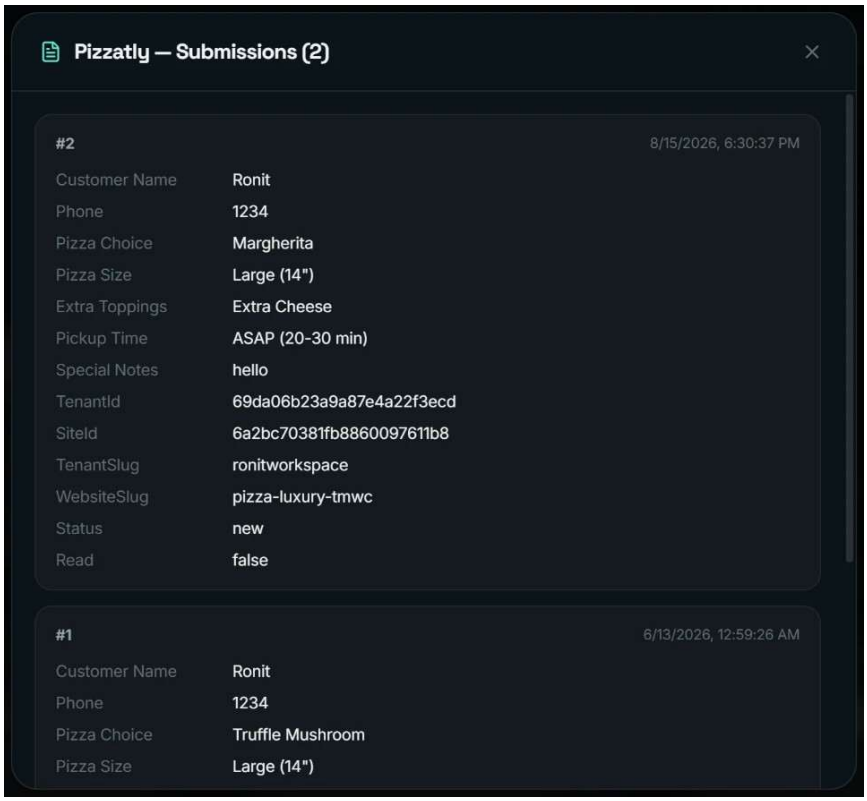


Figure 4.15: Form-submission inbox of the Pizzatly site

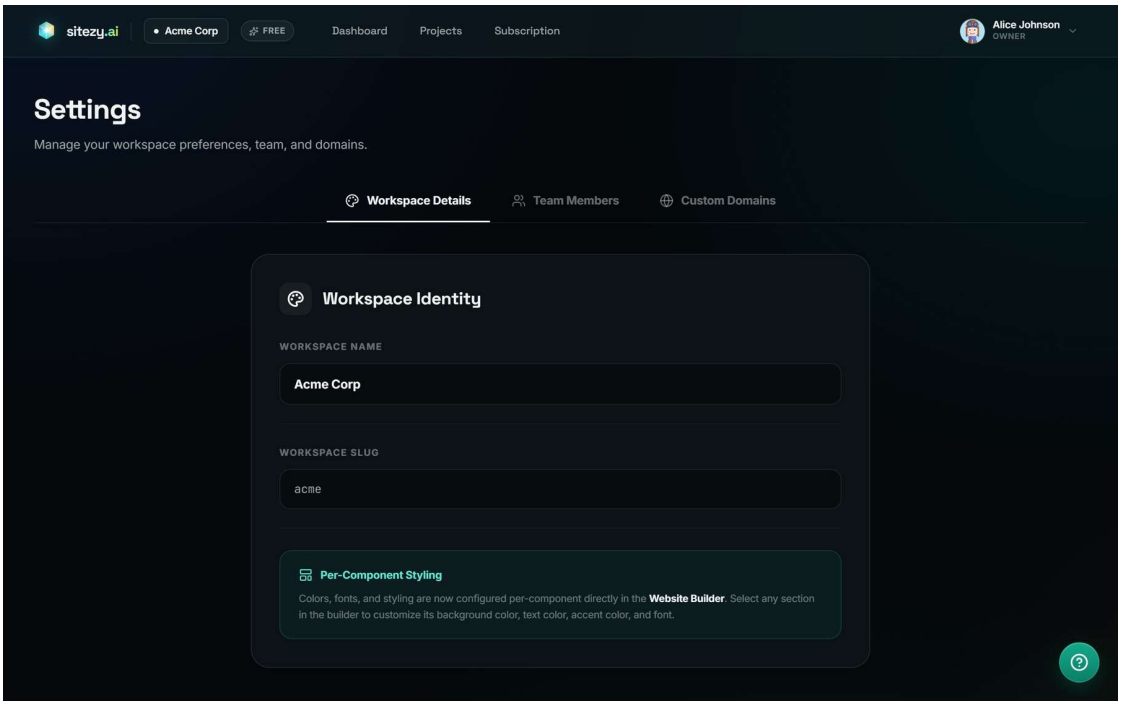


Figure 4.16: Workspace settings with Workspace Details, Team Members and Custom Domains tabs

Workspace settings (Fig. 4.16) manage team members, roles and custom domains; the subscription page (Fig. 4.17) offers the four plans with Razorpay checkout; and the deployments and activity pages (Figs. 4.18 and 4.19) show every published snapshot and the tenant's audit trail.

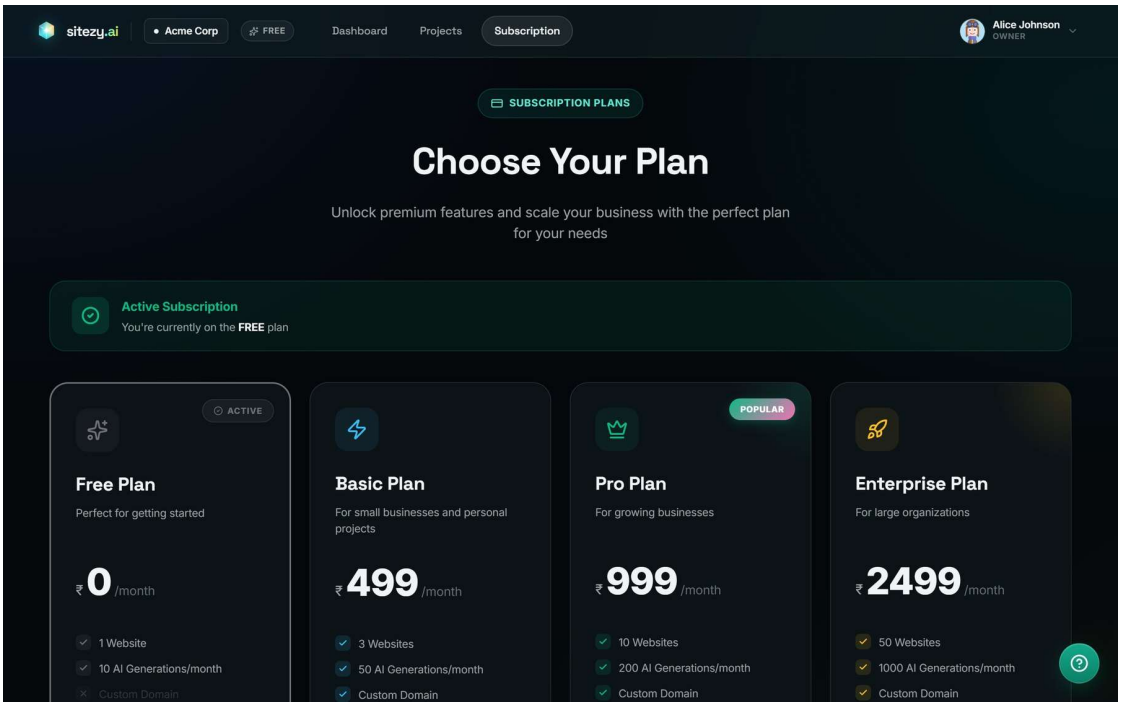


Figure 4.17: Subscription plans

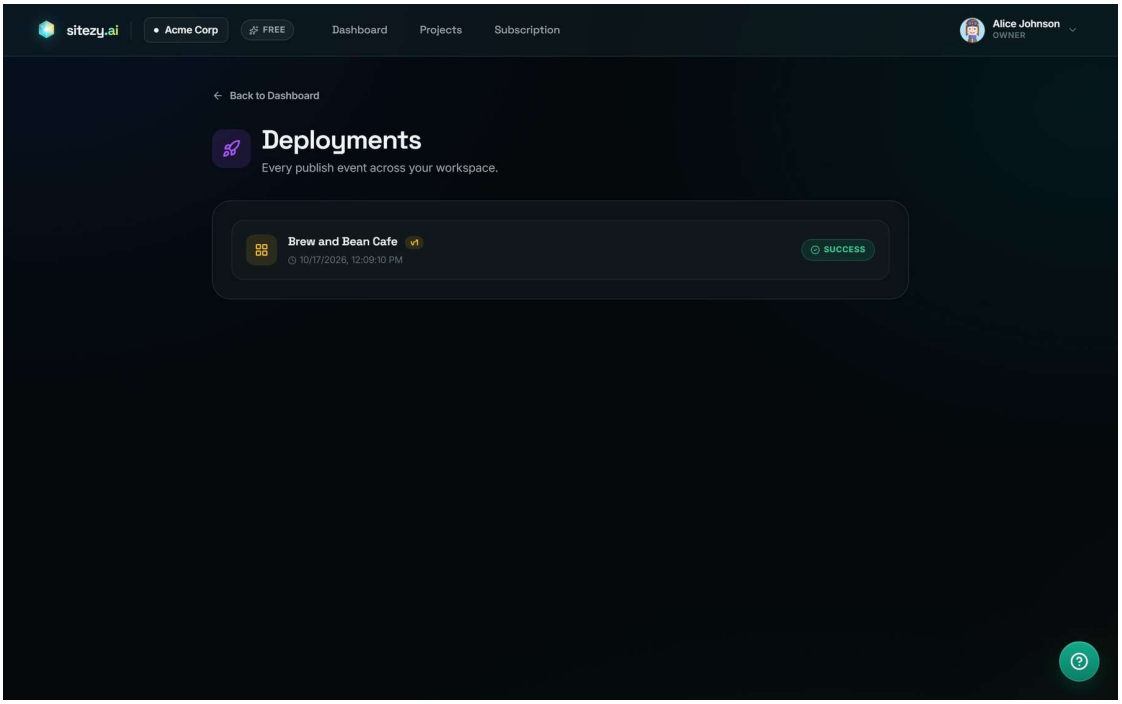


Figure 4.18: Deployment history

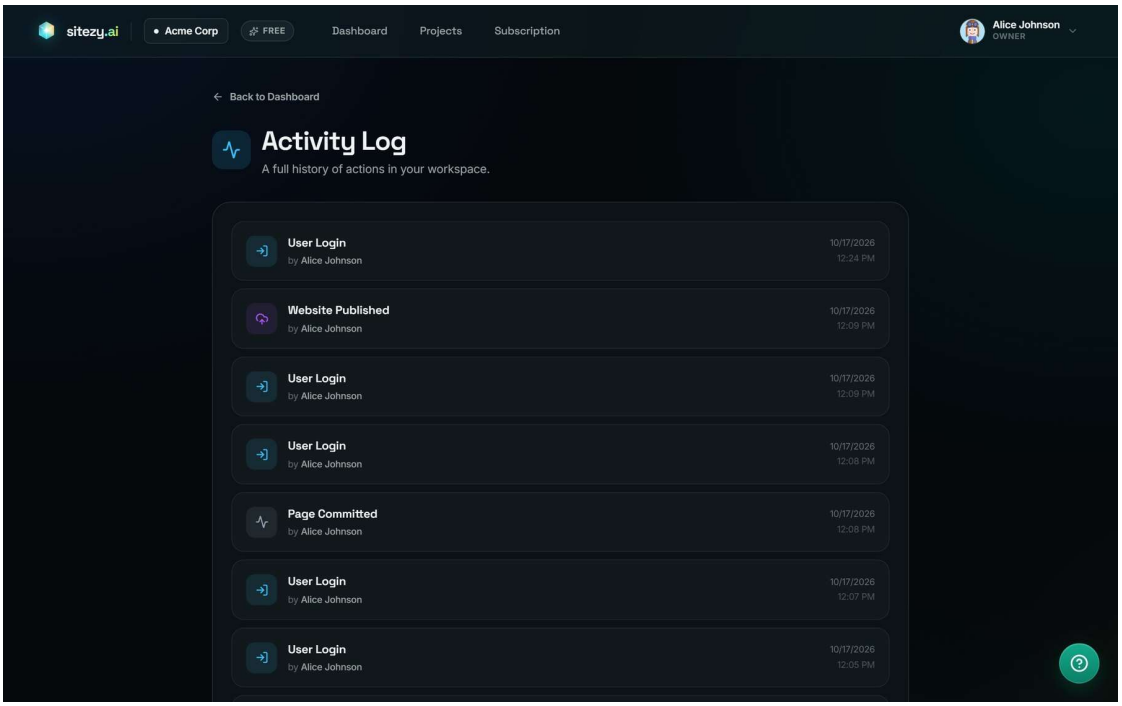


Figure 4.19: Activity log

4.4 Result Analysis

Unless stated otherwise, all experiments ran on one laptop (NVIDIA RTX 4060 Laptop GPU with 8 GB, AMD Ryzen 7 7840HS, 16 GB RAM; Ollama 0.21, Python 3.13, PyTorch 2.11 on the CPU, scikit-learn 1.7, Transformers 5.6, Node.js 22). Trained engines were evaluated over five seeds with fresh, seed-disjoint training and test sets (means with 95% t-intervals); proportions carry Wilson intervals [35], and paired comparisons on the same prompts use the exact McNemar test [36]. Generation experiments used the production prompt verbatim on 20 benchmark businesses.

4.4.1 Generation Validity, Latency and Cost

Table 4.4: Generation validity, latency and output length on the 20 production prompts

Tier	Decoding	Valid	Truncated	Median s	Output tokens
Local	Reasoning on, free-form	0/20 [0, 16]	20	99.1	5,000
Local	Reasoning off, free-form	20/20 [84, 100]	0	78.3	3,919
Local	Reasoning on, JSON grammar	29/60 [36, 61]	31	101.8	5,000
Local	**Reasoning off, JSON grammar (deployed)**	**59/60 [91, 100]**	**0**	**77.4**	**3,932**
Cloud	Gemini 3 Flash (production path)	40/40 [91, 100]	0	20.8	2,866 + 1,270 reasoning
Cloud	Gemini 3 Flash, matched constraints	12/20 [39, 78]	8	19.2	2,729 + 1,318 reasoning

Note: rows with 60 or 40 generations pool three or two independent runs; valid means the output passed the layout schema (Wilson 95% interval in brackets); truncated means generation stopped at the 5,000-token budget; times are medians and output tokens include reasoning tokens.

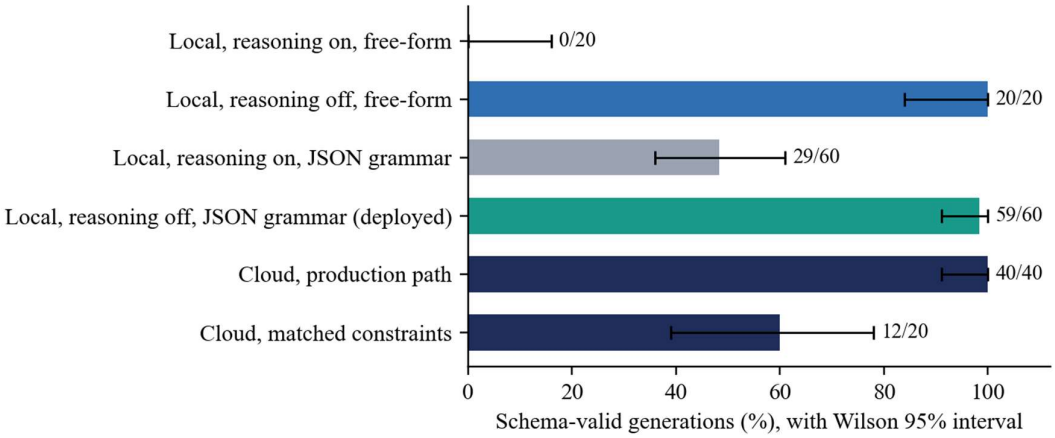


Figure 4.20: Schema-valid generations by decoding configuration

Table 4.4 and Fig. 4.20 show that the reasoning mode decides validity. With reasoning enabled and no grammar, every generation exhausted the budget before the JSON closed; the grammar recovered only 29 of 60. With reasoning disabled, 79 of 80 generations were valid with or without the grammar, and faster (median 77.4 s against 99 to 102 s). The one failure closed a brace too early, so JSON mode does not guarantee well-formed output and the schema check and template fallback remain necessary. Disabling reasoning is significant against free-form decoding (McNemar $p = 1.9 \times 10^{-6}$) and against reasoning with grammar ($p \leq 0.008$ in each run), and latency was stable across runs (Friedman $p = 0.70$). Under the same constraints the cloud model failed in the same way: on 8 of 20 sites its reasoning tokens exhausted the budget, so part of its advantage comes from an uncapped budget.

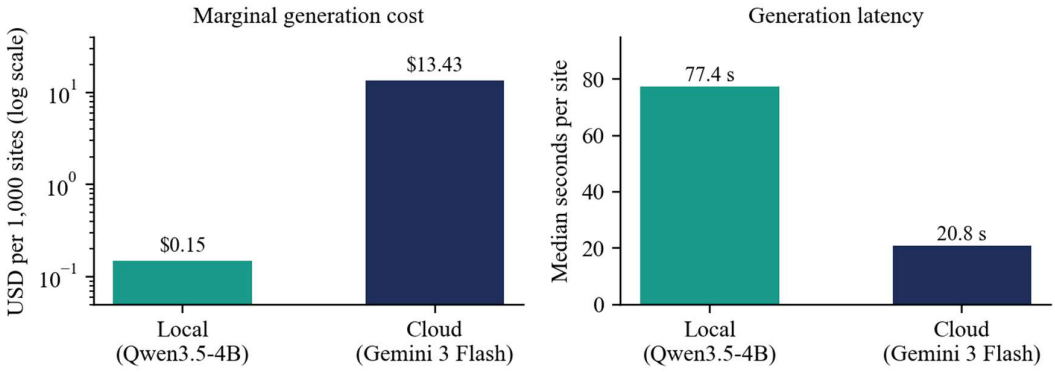


Figure 4.21: Marginal cost and latency of local and cloud generation

Cost. At list price (US$0.50 and US$3.00 per million input and output tokens [37]) the cloud tier costs US$13.43 per 1,000 sites. The local tier's marginal cost is energy: 60 W board power for a

median of 77.4 s at an assumed US$0.12 per kWh is 1.3 Wh, or US$0.15 per 1,000 sites, about 1.2% of the cloud fee, at 3.7 times the latency (Fig. 4.21). This excludes the rest of the system's power, GPU amortisation and operations. In the field, 19 development tenants logged 251 generation requests between February and September 2026: before local serving existed, all went to the cloud and 38% failed, mostly on rate limits and credentials; afterwards 56 of 153 were served locally, 53 of them successfully.

4.4.2 SEO Engine

All SEO figures are scores against our rubric, a proxy built from published ranking factors [4], not measured rankings. Because the labels are a linear function of the features plus noise, linear regression recovers the rubric almost exactly, and the deployed MLP configuration under-fits (Table 4.5). On rubric-derived labels the learned scorer therefore adds no accuracy; its value is as a trainable component that can be re-fitted to real search outcomes.

Table 4.5: SEO scorer against baselines (5 seeds, 5,000 test pages each; R^2 and MAE against the noise-free rubric)

Model	R^2	MAE (0-100)	Latency (ms)
Linear regression	1.000 ± 0.000	0.07 ± 0.02	0.04
Histogram gradient boosting	0.991 ± 0.000	0.90 ± 0.02	3.2
MLP, tuned (dropout 0.05, 900 epochs)	0.977 ± 0.009	1.62 ± 0.36	0.09
Random forest (200 trees)	0.956 ± 0.002	2.01 ± 0.04	8.2
MLP, deployed configuration	0.738 ± 0.105	5.93 ± 1.35	0.09

Gated rewrite loop. On the 60 pages of the 20 benchmark sites (51 below the threshold), the loop raised the rubric score from 63.3 ± 5.3 to 72.7 ± 4.7; pages at or above the threshold rose from 9 to 20 and none ended lower. The deterministic metadata step contributed more than the LLM (5.3 against 3.1 points). The gate is what makes the LLM stage safe: of 79 calls, 35 proposals were accepted, 16 had no applicable patch and 28 were rejected, 26 of which would have lowered the score (by up to 10.7 points). A rewrite took 7.5 s; scoring takes 0.32 ms.

4.4.3 Conversion Model and Design Auditor

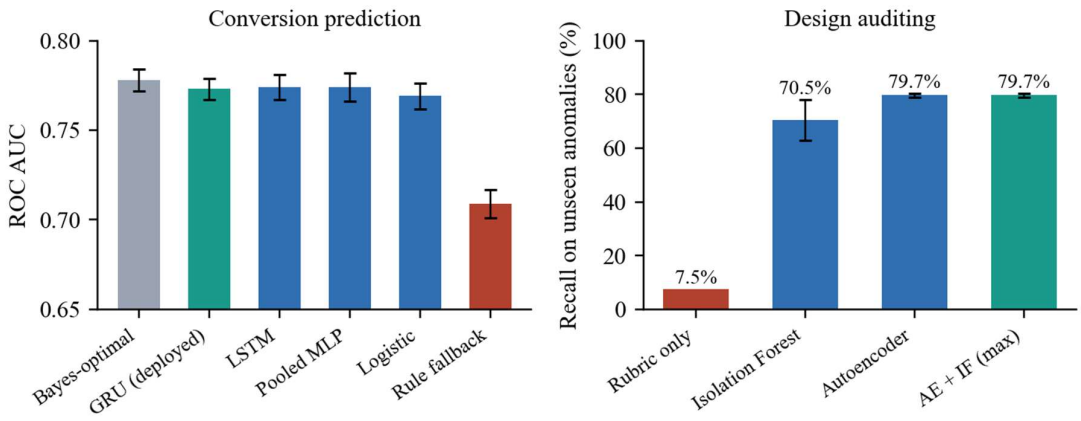


Figure 4.22: Conversion-model AUC and design-auditor recall on unseen anomalies

On 4,000 synthetic test sessions per seed, the Bayes-optimal AUC is 0.778 ± 0.006 and the deployed GRU reaches 0.773 ± 0.006, 99.4% of it, with 25,473 parameters, 24% fewer than an equally accurate LSTM (Fig. 4.22). Mean-pooled MLP and logistic models are equivalent, and all learned models beat the deployed rule fallback (0.709) by 0.061 to 0.065 AUC. Sequence modelling brings no accuracy gain on these labels; we keep the GRU because only it makes predictions depend on section order, which counterfactual suggestions need. Moving a buried CTA after the hero raised the *predicted* conversion by 0.061 ± 0.027 on all 15 templates that have one.

For the design auditor, section-level flaws were injected into the 16 templates: ten catalogued categories that match rubric checks and five out-of-catalogue anomalies (section bloat, gallery overload, sparse page, colour chaos, wall of text). The rubric is perfect on catalogued flaws by construction but catches only 7.5% of out-of-catalogue anomalies; the autoencoder plus Isolation Forest keeps catalogued F1 at 0.999 and catches 79.7% of them (AUC 0.897) at a 0.3% false-positive rate on clean pages. Auditing a page takes 6.7 ms.

4.4.4 Recommender and Forms

On 52 author-written prompts (four per business type), entailment scoring identified the business type in 43 (82.7%, Wilson [70.3, 90.6]; chance 7.7%); an earlier version that scored the neutral class managed 26 (50.0%; McNemar $p = 0.0009$). The evaluation also exposed an integration defect: the backend silently dropped 63 of the 152 blocks recommended for the benchmark businesses and never requested the order form; both were fixed. Batching the 43 cross-encoder passes cut a recommendation from 1.37 s to 0.37 s on the CPU.

Schema validity did not imply complete content: the local model left the form's field list empty on 10 to 12 of the valid sites in each run, while Gemini always filled it. After field validation and

targeted regeneration (Algorithm 3.3), all 32 empty forms on the 59 valid sites were regenerated by the model, giving 59 of 59 filled forms with six to eight valid fields, at 8.7 s per regeneration.

4.4.5 Real Tenant Pages

We ran the deployed engines read-only on all 66 published pages of 28 tenant websites. Rubric scores were 68.1 ± 4.2 with 58 pages below the threshold; the gated loop raised them to 75.1 ± 3.7 and the pages at or above the threshold from 8 to 15, lowering none. The rubric flagged 19 pages, mostly for a missing H1 or a buried CTA. The anomaly detector flagged 63 of 66 pages, including 44 with no rubric issue: trained on home-page templates, it flags page type and size rather than design quality and must be retrained before it transfers. All 30 forms had fields.

4.4.6 Deployment Pipeline

Against a Firebase test project, all 20 benchmark sites deployed on the first attempt (median 0.29 s), so six error classes and an unclassified error were injected into the deploy call, once (transient) or on every attempt (persistent), three times each, under three retry policies (Table 4.6).

Table 4.6: Deployment fault injection under three retry policies

Policy	Faults classified	Transient faults recovered	Extra attempts on persistent faults	Unsafe retries (permission, credentials)	Total time (s)
DiagnosingAgent (deployed)	42/42	9 (all medium severity)	9	0	51
No retry	-	0/21	0	0	-
Blind retry	-	21/21	21	12	99

The agent retries only what is safe, at the cost of six recoverable quota and storage faults that it does not retry; after five failures the circuit breaker rejected further deploys. These are fault-injection results on a test project, not production failure rates.

4.4.7 End-to-End Ablation

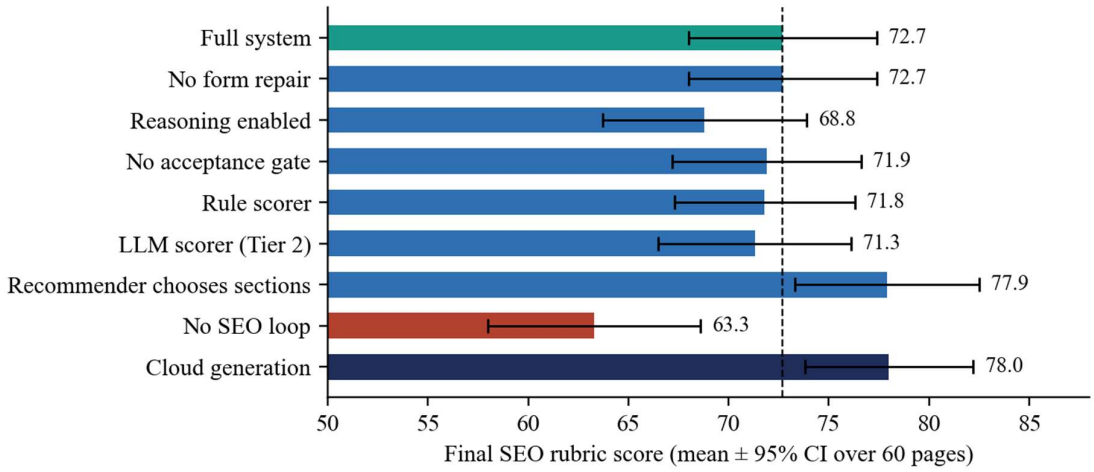


Figure 4.23: End-to-end ablation: final SEO rubric score per arm (dashed line: full system)

Table 4.7: End-to-end ablation on the 20 benchmark sites (60 pages), one factor changed per arm

Arm	Model layout	Forms filled	SEO (mean ± 95% CI)	Regressed	Time (s)	US$ per 1k sites
Full system	20/20	20/20	72.7 ± 4.7	0/60	113	0.23
No form repair	20/20	8/20	72.7 ± 4.7	0/60	107	0.21
Reasoning enabled	12/20	12/20	68.8 ± 5.1	0/60	123	0.25
No acceptance gate	20/20	20/20	71.9 ± 4.7	3/60	131	0.26
Rule scorer (no learned model)	20/20	20/20	71.8 ± 4.5	0/60	107	0.21
LLM scorer (Tier 2 instead of Tier 1)	20/20	20/20	71.3 ± 4.8	0/60	120	0.24
Block recommender chooses sections	18/20	18/20	77.9 ± 4.6	0/60	95	0.19
No SEO loop	20/20	20/20	63.3 ± 5.3	0/60	83	0.17
Cloud generation	20/20	20/20	78.0 ± 4.2	0/60	44	13.48

Removing the SEO loop costs 9.4 rubric points, the largest effect; removing form repair saves 6 s per site but leaves 12 of 20 forms empty; enabling reasoning replaces 8 of 20 sites with the template. Without the gate, 35 of 84 accepted edits lowered the score and three pages ended below their start: the gate prevents regressions rather than raising the average. Routing the scoring step to the Tier-2 model instead of the learned scorer loses 1.5 points ($p = 0.001$) and adds 7 s per site. Letting the recommender choose sections raises the score to 77.9, on a par with cloud generation (78.0), which is 2.6 times faster but costs US$13.48 instead of US$0.23 per 1,000 sites and sends all generation data off-host.

4.4.8 Security and Tenant Isolation

A static audit of all 163 database call sites found 118 with the tenant filter in the call; the other 45 are inherently cross-tenant (tenant or domain resolution, global uniqueness, administration and billing), scoped through a tenant-filtered parent, the published-pages chatbot, or offline scripts unreachable over HTTP. In dynamic tests on an empty in-memory database, tenant B's owner

token was sent with tenant A's identifiers to all 39 route-method pairs that take a resource identifier: all 39 were blocked, 10 list endpoints exposed nothing of A, seven tampered tokens (for example alg:none, a wrong secret or a foreign tenant claim) were rejected, and an editor was refused on eight owner and admin routes even with a forged OWNER claim. Two defects were found, fixed and re-tested (93 of 93 checks pass): a domain could be linked to another tenant's website, and the deployment route lacked a role check.

4.4.9 Comparison with Objectives

Table 4.8: Achievement of the objectives

Objective (Section 1.3)	Result
1. Multi-tenant platform with roles and plans	Implemented; 39/39 cross-tenant requests blocked; two defects found and fixed
2. Multi-page generation from a brief, multilingual and voice	Implemented; three-page sites with forms, images and theme from one brief
3. Task-typed routing	Bounded tasks answered in milliseconds by Tier 1; cloud used only on Pro tier or local failure
4. Reliable local generation	59/60 schema-valid; 59/59 forms filled after targeted regeneration
5. Post-generation engines	GRU at 99.4% of Bayes-optimal AUC; gated loop +9.4 rubric points with no regressions; auditor catches 79.7% of unseen anomalies
6. Builder, collaboration, versions, export	Implemented and demonstrated (Section 4.3)
7. Backend automation	Snapshot publishing and rollback; Firestore forms; agent classified 42/42 injected faults and never retried unsafe ones
8. Quantitative evaluation and publication	Full evaluation suite released; paper accepted at ITI 2026 (Springer LNNS)

4.5 Limitations

- **Synthetic labels and predicted lift.** The SEO labels are a linear rubric, so the scorer is a proxy until it is re-fitted to real search outcomes; visitor sessions are simulated and design flaws injected; all conversion gains are predictions, and the tracker has logged only a handful of real engagement events.
- **Transfer.** The design anomaly detector, trained on home-page templates, does not transfer to real inner pages and must be retrained on real page types.
- **Content completeness.** Only structure and form fields are validated; generated copy is not checked by a human or a factuality test.
- **Scope of evidence.** One laptop GPU; 251 development-period requests without recorded escalation reasons; deployments on a test project with injected faults; author-written test prompts for the recommender; no penetration test or external security audit.
- **Latency.** Local generation takes about 77 s per three-page site, 3.7 times slower than the cloud model.

4.6 Observation/Remarks

- Most decisions in a website builder do not need a language model: small learned models answer them in milliseconds, and routing by task type removes them from the LLM path entirely.
- A single decoding flag, disabling the reasoning mode, mattered more than the grammar itself: reasoning tokens competing for a fixed budget were the main cause of invalid output, for the cloud model as well.
- Automatic rewriting must be gated: about one third of the LLM's proposals would have lowered the score.
- Executing the deployed code found real defects that code review had missed: two access-control gaps, a recommender integration that silently dropped blocks, and empty forms from the local model. All were fixed and re-measured.
- Negative results are informative: a linear model fits the rubric, sequence modelling adds no accuracy on synthetic sessions, and the anomaly detector does not transfer. They show where real data is needed next.

4.7 Conclusion

The implementation delivers every feature planned in Chapter 3: a multi-tenant platform with roles and subscriptions, AI generation of complete multi-page sites from multilingual or spoken briefs, a collaborative visual builder with versions and exact export, four learned post-generation engines, snapshot publishing with working forms and an agentic deployment pipeline. The evaluation shows that a 4-billion-parameter local model produces schema-valid sites in 59 of 60 cases once reasoning is disabled, at about 1% of the cloud's marginal cost; that the gated loop raises SEO rubric scores without regressions; and that tenant isolation holds after two fixes. It also shows where learning does not yet help without real data. The next chapter draws the overall conclusions and outlines future work.

CHAPTER: 5

CONCLUSION

Chapter 5
Conclusion

5.1 Limitations

Sitezy meets its objectives, but the evidence and the system have clear limits:

- The Tier-1 engines are trained on synthetic data. The SEO scorer fits a rubric of published factors and is not yet a ranking model; conversion gains are predictions from simulated sessions, not measured lifts, which require online controlled experiments [22].
- The design anomaly detector, trained on home-page templates, flags page type and size on real inner pages and must be retrained on real page types before its signal is useful there.
- Generated copy is validated for structure and form fields only; there is no human or automatic check of factual accuracy or tone.
- Local generation takes about 77 s for a three-page site on a laptop GPU, 3.7 times slower than the cloud model, and serving many tenants concurrently would need a server GPU and a request queue.
- The evaluation used one GPU, development-period traffic, a Firebase test project with injected faults and author-written prompts; there has been no penetration test or external security audit.

5.2 Conclusion

This project designed, built and evaluated Sitezy, a multi-tenant platform that turns a plain-language brief into a complete, editable, optimised and deployable website. Its main lesson is that most decisions an AI website builder makes do not need a language model. Small learned models answer bounded questions, such as which blocks to use, how good the SEO is, whether the design has flaws and how a reordering would change conversion, in milliseconds; a 4-billion-parameter local model produces schema-valid sites in 59 of 60 cases once its reasoning mode is disabled, at about 1% of the cloud's marginal cost; targeted regeneration repairs the forms it leaves empty; and cloud calls become an explicit, auditable exception.

Around this stack, Sitezy delivers what small businesses need beyond a first draft: a real-time collaborative builder with version history and exact code export, a gated rewrite loop that improves SEO rubric scores without ever lowering them, design and engagement insights with one-click fixes, publishing with rollback, working forms without backend code, an agentic deployment pipeline that retries only what is safe, and tenant isolation verified by dynamic tests. Executing the deployed code also showed honestly where learning does not yet help: a linear model fits the rubric labels, sequence modelling adds no accuracy on synthetic sessions, and the

anomaly detector does not transfer to real pages. The work was accepted for publication at ITI 2026 in Springer LNNS.

5.3 Future Scope

- **Real data for the learned engines.** Collect search impressions and rankings (for example through Google Search Console), real visitor sessions and conversions, and re-fit the SEO scorer and the GRU to them.
- **Online experiments.** Add A/B testing to the publishing pipeline so that suggested changes can be validated as measured, not predicted, conversion lifts.
- **Retrained design auditor.** Train the anomaly detector on real pages of every page type and add learned fixes.
- **Faster and broader generation.** Batch and stream local generation, serve several tenants from a GPU queue, try newer small models and speculative decoding, and generate e-commerce, booking and blog sections.
- **Content quality checks.** Add factuality and tone checks against the brief, and human evaluation of generated copy.
- **Security and compliance.** Commission a penetration test, add audit exports and data-retention controls, and extend redaction to more personal-data types.
- **Product features.** Native mobile editing, a template marketplace, multi-language sites and integrations with payment and CRM tools for tenants' own customers.

5.4 Published Paper

Title: Sitezy: Hybrid Local-LLM Routing, Deep-Learning Post-Generation Optimisation, and Agentic Backend Automation for Multi-Tenant Website Generation

Authors: Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi, Shakti Sankpal and Kajal Joseph, Department of Information Technology, VESIT, Mumbai

Conference: ITI 2026, Paper ID 950; presentation at NIT Agartala, 13-14 November 2026

Publication: Accepted; camera-ready version submitted for the proceedings in Springer Lecture Notes in Networks and Systems (LNNS) [30]

Code and evaluation scripts: https://github.com/shaktisankpal/SitePilot/tree/ronit/evaluation

Originality: Turnitin similarity index 1% (13 October 2026); AI-writing score below Turnitin's 20% reporting threshold (Appendix A.I)

Presentation certificate: To be attached after the presentation.

The camera-ready paper is reproduced in Appendix A.I.

References

[1] Wix.com Ltd., "Wix ADI (Artificial Design Intelligence)," 2024. [Online]. Available: https://www.wix.com/adi

[2] Webflow, Inc., "Webflow: the visual web development platform," 2024. [Online]. Available: https://webflow.com

[3] C.-P. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: Maintenance dream or nightmare?," in Proc. Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution (IWPSE-EVOL), ACM, 2010, pp. 88-92.

[4] C. Ziakis, M. Vlachopoulou, T. Kyrkoudis and M. Karagkiozidou, "Important factors for improving Google search rank," Future Internet, vol. 11, no. 2, p. 32, 2019. doi: 10.3390/fi11020032

[5] B. Hidasi, A. Karatzoglou, L. Baltrunas and D. Tikk, "Session-based recommendations with recurrent neural networks," in Proc. International Conference on Learning Representations (ICLR), 2016. arXiv:1511.06939. [Online]. Available: https://arxiv.org/abs/1511.06939

[6] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008, pp. 413-422. doi: 10.1109/ICDM.2008.17

[7] L. Chen, M. Zaharia and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[8] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," arXiv:2406.18665, 2024. [Online]. Available: https://arxiv.org/abs/2406.18665

[9] C. Varangot-Reille et al., "Doing more with less: A survey on routing strategies for resource optimisation in large language model-based systems," Journal of Artificial Intelligence Research, vol. 86, Article 14, 2026.

[10] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv:2307.09702, 2023. [Online]. Available: https://arxiv.org/abs/2307.09702

[11] C. Si et al., "Design2Code: How far are we from automating front-end engineering?," arXiv:2403.03163, 2024. [Online]. Available: https://arxiv.org/abs/2403.03163

[12] Z. Lu et al., "WebGen-Bench: Evaluating LLMs on generating interactive and functional websites from scratch," in Advances in Neural Information Processing Systems (NeurIPS), 2025. arXiv:2505.03733. [Online]. Available: https://arxiv.org/abs/2505.03733

[13] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. EMNLP, 2014, pp. 1724-1734.

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[15] J. Chung, C. Gulcehre, K. Cho and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," arXiv:1412.3555, 2014.

[16] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," in Proc. MLSDA Workshop, ACM, 2014, pp. 4-11.

[17] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in Proc. International Conference on Learning Representations (ICLR), 2018.

[18] W. Yin, J. Hay and D. Roth, "Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach," in Proc. EMNLP-IJCNLP, 2019, pp. 3914-3923. arXiv:1909.00161. [Online]. Available: https://arxiv.org/abs/1909.00161

[19] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[20] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[21] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[22] R. Kohavi, R. Longbotham, D. Sommerfield and R. M. Henne, "Controlled experiments on the web: Survey and practical guide," Data Mining and Knowledge Discovery, vol. 18, no. 1, pp. 140-181, 2009.

[23] MongoDB, Inc., "MongoDB documentation," [Online]. Available: https://www.mongodb.com/docs/ (accessed Oct. 2026).

[24] Google, "Firebase documentation: Cloud Firestore and Firebase Hosting," [Online]. Available: https://firebase.google.com/docs (accessed Oct. 2026).

[25] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8024-8035.

[26] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[27] A. Yang et al. (Qwen Team), "Qwen3 technical report," arXiv:2505.09388, 2025.

[28] Ollama, "qwen3.5 model library entry," [Online]. Available: https://ollama.com/library/qwen3.5 (accessed Oct. 2026).

[29] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[30] R. Sahoo, A. Lotankar, M. Kabadi, S. Sankpal and K. Joseph, "Sitezy: Hybrid local-LLM routing, deep-learning post-generation optimisation, and agentic backend automation for multi-tenant website generation," accepted at ITI 2026 (Paper ID 950), to appear in Springer Lecture Notes in Networks and Systems (LNNS), 2026.

[31] Meta Open Source, "React: The library for web and native user interfaces," [Online]. Available: https://react.dev (accessed Oct. 2026).

[32] OpenJS Foundation, "Express: Fast, unopinionated, minimalist web framework for Node.js," [Online]. Available: https://expressjs.com (accessed Oct. 2026).

[33] Socket.IO, "Socket.IO documentation," [Online]. Available: https://socket.io/docs/v4/ (accessed Oct. 2026).

[34] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[35] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209-212, 1927.

[36] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153-157, 1947.

[37] Google, "Gemini Developer API pricing," [Online]. Available: https://ai.google.dev/gemini-api/docs/pricing (accessed Oct. 2026).

Appendix

A.I. Publication Details

Camera-Ready Paper

The camera-ready version of the paper accepted at ITI 2026 (Paper ID 950), as submitted for publication in Springer LNNS, is reproduced on the following pages.

Sitezy: Hybrid Local-LLM Routing, Deep-Learning Post-Generation Optimisation, and Agentic Backend Automation for Multi-Tenant Website Generation

Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi, Shakti Sankpal, and Kajal Joseph

Department of Information Technology, Vivekanand Education Society's Institute of Technology (VESIT), Mumbai, India
2023.ronit.sahoo@ves.ac.in, 2023.atharva.lotankar@ves.ac.in
2023.manaswi.kabadi@ves.ac.in, 2023.shakti.sankpal@ves.ac.in
kajal.jewani@ves.ac.in

Abstract. AI website builders send every generation request to a metered cloud language model and offer little measured optimisation once a site exists. We present *Sitezy*, a multi-tenant platform in which decisions with a bounded output space—block recommendation, SEO scoring, design auditing, conversion prediction—are made by small learned models, and only open-ended generation reaches a language model, served locally (Qwen3.5-4B) with a cloud model as an explicit exception. Executing the deployed code on 20 benchmark sites and 66 real published pages, we find that disabling the local model's reasoning mode is decisive: with it enabled, 8 to 20 of 20 generations exhaust the token budget; with it disabled, 59 of 60 are valid over three runs (McNemar $p < 10^{-4}$), at \$0.15 of GPU energy per 1,000 sites against \$13.43 for the 3.7× faster cloud model. On synthetic sessions, a GRU conversion model reaches 99.4% of the Bayes-optimal AUC, and a gated rewrite loop raises rubric-based SEO scores (a proxy, not search performance) without lowering any. Fault-injection and tenant-isolation tests found two access-control defects, since fixed. We also report negative results—a linear model fits the SEO rubric almost exactly, the local model leaves 32 of 59 forms empty until targeted regeneration fills them, and the anomaly detector does not transfer to real pages—and release all scripts and seeds.

Keywords: Local LLM · Inference routing · Constrained decoding · Multi-tenant SaaS · GRU · Anomaly detection · Ablation

1 Introduction

Building a small-business website couples layout, copywriting, search optimisation and a backend for customer input. AI website builders [1, 2] make layout tractable, but in a multi-tenant setting every content request is metered by a cloud model and sends tenant data off-platform; generated pages receive no

measured optimisation, although on-page attributes predict search visibility [14], sequence models capture visitor behaviour [15, 18], and anomaly detectors audit design without labelled failures [19, 21]; and a page stays inert until forms, storage and hosting are wired.

Primary contribution. We contribute a *local-first, task-typed* AI stack and its measured characterisation: bounded tasks are answered by small learned models (Tier 1) and only open-ended generation reaches a language model (Tier 2 local, Tier 3 cloud). The other components form one loop around it (Fig. 1): the language model generates a site, Tier-1 engines recommend, score and audit it, a gated rewrite step improves weak copy, backend agents publish it and provision form storage, and a tracker returns sessions to the conversion model. We contribute (i) the architecture, with exception-driven escalation and redaction before any cloud call (Sects. 3–6); (ii) a repeated decoding ablation, same-workload cloud baselines and a measured cost model (Sect. 7.1); (iii) an evaluation of the learned engines against stronger baselines, the deployed rules and the Bayes-optimal ceiling, on benchmarks and real pages (Sects. 7.2–7.6); and (iv) deployment fault injection, isolation tests and an end-to-end ablation (Sects. 7.7–8). All scripts and seeds are released.[1]

2 Related Work

Cost-aware LLM serving routes queries *among language models* [10]: FrugalGPT [8] cascades APIs and stops when a learned scorer accepts an answer, and RouteLLM [9] learns to send each query to a strong or a weak model. Sitezy instead routes by *task type*, so bounded tasks never reach a language model, and escalates on deterministic events rather than learned confidence. Grammar-constrained decoding [11] is designed to guarantee syntactically valid output; we show that for long outputs validity is decided by reasoning tokens competing for a fixed budget. Design2Code [12] benchmarks screenshot-to-code generation and WebGen-Bench [13] functional multi-file websites; we target what surrounds generation. Our engines build on gated recurrent units [15], which match LSTMs [16] with fewer parameters [17] and model sessions well [18]; on reconstruction-error [20, 21] and isolation-based [19] anomaly detection; and on entailment-based zero-shot classification [23] with MiniLM [24] feeding random forests [25]. Conversion effects require online controlled experiments [26] for validation, and shared-database isolation is a central multi-tenant risk [27]. Unlike these, and cloud AI builders whose routing is not documented, Sitezy routes by task type with deterministic escalation, runs local-first, constrains output, and adds four learned post-generation engines.

3 System Architecture

Sitezy (Fig. 1) comprises a React client (out of scope), a Node.js/Express API (AI router, collaboration, page versioning, deployment agents), a Python

[1] `https://github.com/shaktisankpal/SitePilot/tree/ronit/evaluation`

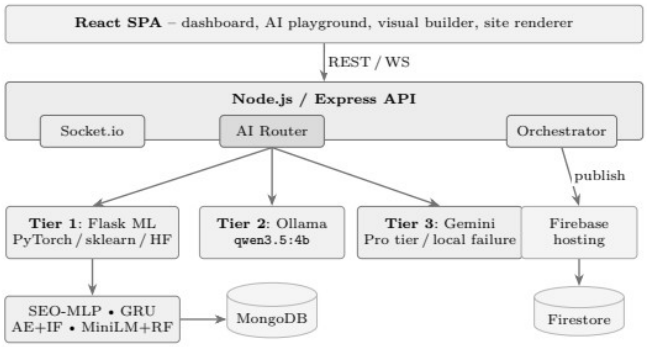


Fig. 1. Architecture. Bounded tasks go to Tier 1, open-ended generation to Tier 2 (local); Tier 3 (cloud) is used only on the Pro tier or a local failure. All stores are tenant-scoped.

Table 1. Routing and escalation as implemented. Apart from form repair, no condition re-prompts a model.

Condition	Action	Cloud
Bounded task (recommend, score, audit, predict)	Tier-1 learned model (rule if artifact missing)	never
Open-ended generation, Basic tier	Tier 2: `qwen3.5:4b`, JSON grammar, no reasoning	no
Open-ended generation, Pro tier selected	Tier 3 directly	yes
Tier-2 runtime error (unreachable, timeout)	Tier 3 on the same prompt	yes
Output unparseable or fails the layout schema	deterministic template layout	no
Form with <3 valid fields (Sect. 7.5)	Tier 2 regenerates that form only (fixed fields if it fails)	no

Flask service with the four Tier-1 engines, and an Ollama server running `qwen3.5:4b` [5, 6] (4.7 B parameters, Q4_K_M, 8,192-token context, 6.1 GB of VRAM); Tier 3 is `gemini-3-flash-preview` [3, 7]. MongoDB holds 16 collections and Firestore the per-tenant form submissions; sites are served by the platform renderer or exported to Firebase Hosting. Each Tier-1 endpoint degrades to a deterministic rule if its model artifact is missing.

4 Generation Tier: Routing, Decoding, and Data Handling

Routing. The router is rule-based, with no learned confidence threshold (Table 1): escalation follows observable events (a tenant's choice or a local runtime exception), since a validity check is exact and cheap while a threshold would need calibration data a new tenant lacks, and every assignment stays auditable. Invalid output from either model is replaced by a deterministic template layout, never by a second generation call, which bounds cloud spend.

Table 2. Synthetic training data. $\tilde{x}_k$: min–max normalised features; c, e, s, d, b: CTA click, early CTA, form start, form done, bounce; n: visible sections.

Engine	Inputs	Label	Size
SEO scorer	15 uniform on-page features (title 0–90 chars, meta 0–220, keyword density 0–6%, words 20–800, readability 0–20, alt coverage 0–1, link/heading counts, binary flags)	$100\sum_k w_k\tilde{x}_k + \mathcal{N}(0,3^2)$, clipped to $[0,100]$; hand-set w (heading hierarchy .16, keyword density .13, alt .12, ...)	20k/5k per seed
Conversion GRU	Navbar, Hero, 2–6 shuffled sections, Footer; bounce .35 truncates; CTA click .55 if the CTA is in the first four sections, else .30; form start .5, completion .6; dwell $\mathcal{N}(12,4)$ s (content) or $\mathcal{N}(6,4)$ s	$p = .06 + .30c + .10e + .12s + .30d + .08(1-b) + \min(.1, .015n)$, clipped; $y \sim \mathrm{Bern}(p)$	10k/4k per seed
Design auditor	16 builder templates as $\mathbf{f} \in [0,1]^{20}$, each with 130 Gaussian jitters ($\sigma = .04$)	none (clean designs only)	2,096

Constrained decoding. Tier-2 calls set `format:json`, a JSON grammar [11], and `think:false`, which disables the reasoning mode, with a 5,000-token budget, an 8,192-token context and temperature 0.1. A three-page site needs about 4,000 output tokens, so reasoning tokens compete with the site for the same budget (Sect. 7.1).

Data handling. On the cloud path, owner-authored content for publication (business description, tone, audience, brand facts, page copy) is sent as is; user- and visitor-submitted data (form submissions summarised by the analytics assistant, chatbot and help questions) is redacted first: personal fields (name, e-mail, phone, address, contact) are masked and e-mail addresses and phone numbers removed from free text, keeping dates, prices, quantities and product choices. The local model receives unredacted data, which stays on the host.

5 Post-Generation Engines (Tier 1)

Training data. No public dataset pairs page structure with search outcomes, visitor sequences, or design-quality labels, so the engines are trained on procedurally generated data (Table 2); Sect. 9 discusses the consequences.

SEO scorer and gated rewrite loop. An extractor turns a page into 15 features $\mathbf{x}$, and an MLP outputs $s = 100\,\sigma(W_3\phi(W_2\phi(W_1\mathbf{x} + b_1) + b_2) + b_3)$ with hidden widths 64 and 32, ReLU ϕ and dropout 0.2 (deployed: Adam, 160 epochs, MSE). Improvement has three stages. (1) While $s < 80$, up to three of the weakest text-fixable factors go to Tier 2 (JSON, no reasoning, temperature 0.4), which proposes patches to *existing* text fields; the page is re-scored and the patch set is kept only if $s' > s$, otherwise it is discarded and the stage stops ($\leq$3 iterations). (2) A deterministic step adds the target keyword once to a body field, under the same gate. (3) A deterministic step writes a meta description and JSON-LD when those factors are weak; it is not gated, so we measure it separately.

Conversion GRU. A visitor session is padded to 20 steps of 12 features (7-way section-type one-hot, dwell time, scroll depth, CTA click, form start, form completion). Two GRU layers [15] (64 and 32 units; state $h_t = (1 - z_t) \odot$

$h_{t-1} + z_t \odot \tilde{h}_t$ with update gate z_t) feed a dense layer and a sigmoid giving the conversion probability p (BCE, Adam, 14 epochs). Before a site has traffic, a session is synthesised from its section order. Suggestions are counterfactual: the page is re-scored with a section moved and the *predicted* change Δp is reported; since p depends on section order, a bag-of-features model cannot provide this.

Design auditor. A page is encoded as $\mathbf{f} \in [0,1]^{20}$ (section and CTA counts, heading hierarchy, WCAG contrast [22], font and colour diversity, alt coverage, duplicate-heading and empty-section ratios, structural order). A deterministic rubric of eleven checks with severity penalties (high 22, medium 11, low 5) yields the health score shown to users. A symmetric autoencoder (20–14–8–4, mirrored) and an Isolation Forest [19] (200 trees), both trained only on clean designs, give standardised anomaly scores whose maximum is shown as an informational signal.

Block recommender. A MiniLM cross-encoder trained for entailment [24] (`nli-MiniLM2-L6-H768`) classifies a prompt zero-shot [23] over four label sets (13 business types, 11 content types, 9 goals, 10 audiences), choosing the label with the highest entailment probability for "This website is about *label*". The labels feed one random forest [25] per block type (12 blocks); blocks above a business-specific threshold (0.45–0.60) are selected, keyword matches add blocks, and the set is padded to at least four. Tier 2 then fills this skeleton with copy.

6 Backend Automation

Publishing copies a site's pages into an immutable, numbered snapshot served on a tenant subdomain or custom domain; rollback restores a snapshot (11 tenants published 166 versions of 73 websites during the logging period). Every home page carries a form with five to eight typed fields (`text`, `email`, `tel`, `number`, `select`, `textarea`, `date`), validated and, if empty, regenerated (Sect. 7.5), which posts scalar values of at most 5,000 characters to the site's tenant-scoped Firestore path, so no tenant-written backend code is needed. An optional agentic pipeline exports a site to Firebase Hosting: behind a per-tenant circuit breaker (open after five failures in five minutes), a *FirebaseAgent* deploys pages and assets, applies security rules and provisions the form backend; after a failure a *DiagnosingAgent* classifies the error (quota, permission, network, credentials, isolation breach, timeout, storage), re-checks isolation, rules and quota, and assigns a severity: medium (network, timeout, unclassified) is retried once after a 2 s back-off, critical (permission, credentials, isolation) and high (quota, storage) are not.

7 Experimental Evaluation

Setup and protocol. All runs used one laptop: NVIDIA RTX 4060 Laptop GPU (8 GB), AMD Ryzen 7 7840HS, 16 GB RAM; Ollama 0.21, Python 3.13, PyTorch 2.11 (CPU), scikit-learn 1.7, Transformers 5.6, Node.js 22. Trained engines are evaluated over five seeds with freshly generated, seed-disjoint training

Table 3. Generation on the 20 production prompts; rows with 60 (40) generations pool three (two) independent runs. Valid = passed the layout schema (Wilson 95% interval); truncated = stopped at the 5,000-token budget; medians; output tokens include reasoning tokens.

Tier	Decoding	Valid	Truncated	Median s	Output tok.
Local	reasoning on, free-form	0/20 [0, 16]	20	99.1	5,000
Local	reasoning off, free-form	20/20 [84, 100]	0	78.3	3,919
Local	reasoning on, JSON grammar	29/60 [36, 61]	31	101.8	5,000
Local	**reasoning off, JSON grammar (deployed)**	**59/60 [91, 100]**	**0**	**77.4**	**3,932**
Cloud	Gemini 3 Flash (production path)	40/40 [91, 100]	0	20.8	2,866 + 1,270 reasoning
Cloud	Gemini 3 Flash, matched constraints	12/20 [39, 78]	8	19.2	2,729 + 1,318 reasoning

and test sets (means, 95% t-intervals, paired t-tests across seeds); proportions carry Wilson intervals [29], and paired comparisons on the same prompts use the exact McNemar test [28]. Generation experiments call the production prompt, system message and options verbatim on 20 businesses (coffee shop to co-working space), requesting all seven section types as when the recommender is unavailable, and judge validity with the production extraction and layout-schema check.

7.1 Generation Tier

Table 3 reports a paired four-way decoding ablation and a same-workload cloud baseline: the production cloud path (no system message, JSON mode or token cap) and a matched variant with the local call's system message, JSON mode, temperature and 5,000-token budget. The deployed configuration and reasoning with grammar were each run three times and the production cloud path twice (a third cloud run, stopped by the API quota, is excluded).

The reasoning mode decides validity. With it enabled and no grammar, every generation exhausts the budget before the JSON closes; the grammar recovers 12, 7 and 10 of 20 in the three runs (first run vs. free-form: McNemar p=0.0005), the rest truncating. With reasoning disabled, 79 of 80 generations are valid with or without the grammar, and faster (median 77.4 s vs. 99–102 s); the one failure closed a brace too early, so Ollama's JSON mode does not guarantee well-formed output and the schema check and template fallback remain necessary. `think:false` is thus decisive (vs. free-form $p=1.9 \times 10^{-6}$; vs. reasoning with grammar p=0.008, 0.002 and 0.002 per run), and latency is stable across runs (medians 79.0, 77.1, 76.8 s; Friedman p=0.70). Under the same constraints the cloud model fails alike: on 8 of 20 sites its reasoning tokens exhausted the 5,000-token budget (McNemar p=0.008 vs. the production path), so part of its advantage comes from an uncapped budget.

Cost and field usage. The cloud tier is billed $p_{\text{in}} t_{\text{in}} + p_{\text{out}}(t_{\text{out}} + t_{\text{reason}})$ per site at list price ($0.50/$3.00 per million input/output tokens [4]): $13.43 per 1,000 sites over both runs ($13.72 and $13.15; per site $11.36–15.68); failed requests (one transport error, retried) return no tokens and are not billed. The local tier's marginal cost is energy, $P\,\bar{t}\,c_e$, with board power P=60 W (`nvidia-smi`), $\bar{t}$=77.4 s and an assumed c_e=$0.12/kWh: 1.3 Wh or $0.15 per 1,000 sites, about 1.2% of the cloud fee, at 3.7× the latency (the cloud median moved from 26.0

Table 4. SEO scorer vs. baselines (5 seeds, 5,000 test pages each; mean ± 95% CI). R^2 and MAE against the noise-free rubric; latency per page.

Model	R^2	MAE (0–100)	Latency (ms)
Linear regression	1.000 ± 0.000	0.07 ± 0.02	0.04
Histogram gradient boosting	0.991 ± 0.000	0.90 ± 0.02	3.2
MLP, tuned (dropout .05, 900 epochs)	0.977 ± 0.009	1.62 ± 0.36	0.09
Random forest (200 trees)	0.956 ± 0.002	2.01 ± 0.04	8.2
MLP, deployed configuration	0.738 ± 0.105	5.93 ± 1.35	0.09

to 18.8 s between runs). This excludes rest-of-system power, GPU amortisation, operations and cloud free-tier allowances; "near-zero marginal cost" refers only to this energy figure. In the field, 19 tenants logged 251 generation requests (February–September 2026): before local serving existed all went to the cloud and 38% failed, mostly on rate limits and credentials; afterwards 56 of 153 (36.6%) were served locally, 53 successfully. The log covers site generation only, omits why a request went to the cloud, and reflects development-period, not steady-state, traffic.

7.2 SEO Engine

All SEO figures in this paper are scores against our rubric (Table 2), a proxy built from published ranking factors [14]; none is a measured ranking, impression or click-through rate. Because the labels are a linear function of the features plus noise (Table 2), linear regression recovers the noise-free rubric to within estimation error (R^2=0.99995, MAE 0.07; 0.943 against the noisy labels, the noise ceiling); the deployed MLP configuration under-fits (Table 4). On rubric-derived labels the learned scorer therefore adds no accuracy; its value is as a trainable component to be re-fitted to real search outcomes (Sect. 9).

Gated rewrite loop. On the 60 pages of the 20 sites (51 below the threshold), the loop raised the deployed score from 56.4 ± 4.8 to 65.2 ± 4.4 and the rubric score from 63.3 ± 5.3 to 72.7 ± 4.7; pages at or above the deployed scorer's threshold rose from 9 to 20 and none ended lower. The deterministic metadata step contributed more than the LLM (5.3 ± 0.7 vs. 3.1 ± 1.5 points; keyword step 0.3 ± 0.4). The gate makes the LLM stage safe: of 79 calls, 35 proposals were accepted (none lowered the rubric score), 16 had no applicable patch and 28 were rejected, 26 of which would have lowered the score (mean -2.0, worst -10.7). A rewrite took 7.5 s; scoring takes 0.32 ms.

7.3 Conversion Model

On 4,000 test sessions per seed (38% positive), the Bayes-optimal AUC (ranking by the true p) is 0.778 ± 0.006. The deployed GRU reaches 0.773 ± 0.006, 99.4% of it, with 25,473 parameters, 24% fewer than an equally accurate LSTM (0.774 ± 0.007, 33,601; p=0.24); mean-pooled MLP and logistic models (449 and 13 parameters) are equivalent (0.774 ± 0.008, 0.769 ± 0.007), and all learned models beat the deployed rule fallback (0.709 ± 0.008) by 0.061–0.065 AUC

(GRU: p=6 × 10^{-5}); F1 is 0.641–0.645 for all learned models and the Bayes ranking vs. 0.588 for the rule. Sequence modelling brings no accuracy gain on these aggregate-driven labels; we keep the GRU because only it makes predictions depend on section order, which counterfactual suggestions need. Moving a buried CTA after the hero raised the *predicted*—not measured—conversion by 0.061 ± 0.027 on all 15 of the 16 templates that have one; suggestions take 8 ms.

7.4 Design Auditor

We injected section-level flaws into the 16 templates, none of which triggers a rubric check: ten *catalogued* categories matching rubric checks (missing H1: high severity; multiple H1, no H2, buried CTA, empty sections, missing alt text: medium; no footer, too many fonts, duplicate headings, unusual order: low) and five *out-of-catalogue* anomalies (section bloat, gallery overload, sparse page, colour chaos, wall of text). Thresholds were set on even-numbered templates and results reported on odd-numbered ones (per seed: 160 clean, 160 catalogued, 80 out-of-catalogue pages). The rubric is perfect on catalogued flaws by construction but catches only 7.5% of out-of-catalogue anomalies (AUC 0.537); AE+IF (max) keeps catalogued F1 at 0.999 ± 0.003 and catches 79.7 ± 0.7% of them (AUC 0.897 ± 0.015; all but "wall of text", which the length feature cannot represent) at a 0.3% clean false-positive rate. The autoencoder alone matches it (F1 0.992 ± 0.007, recall 79.7 ± 0.7%, AUC 0.899 ± 0.010); the Isolation Forest alone is weaker (0.805 ± 0.031, 70.5 ± 7.5%, 0.860 ± 0.028). Auditing a page takes 6.7 ms.

7.5 Recommender and Form Schemas

No expert annotations of block sets exist, so we evaluate the zero-shot stage that feeds the forests on 52 prompts we wrote ourselves (four per business type). Entailment scoring identifies the business type in 43 (82.7%, Wilson [70.3, 90.6]; chance 7.7%); an earlier version that scored the neutral class managed 26 (50.0%; McNemar p=0.0009), and the fix changes the selected blocks for 98% of prompts. A recommendation takes 1.37 s on the CPU, almost all in 43 sequential cross-encoder passes; batching them cuts this to 0.37 s. The evaluation exposed an integration defect: the backend silently dropped 63 of the 152 blocks recommended for the 20 benchmark businesses (services, pricing, FAQ, team, statistics, newsletter, blog) and never requested the order form; both are fixed (closest section type plus a hint; order form always requested) and measured in Sect. 7.8.

Schema validity does not imply complete content. Gemini filled the required form on all 20 sites, but the local model left the field list empty on 12, 10 and 10 of the 20, 19 and 20 valid sites of its three runs, and post-processing added none. All 191 local and 268 cloud fields conformed to the schema; e-mail and phone fields always had the matching type, `select` fields offered 4.3–4.8 options, and 62% of local vs. 79% of cloud fields were required; 29% of local field names (first run) were copied from the prompt's examples (cloud: 7%). We therefore added *field validation and targeted regeneration*: a form with fewer than three

valid fields (name, label, allowed type, $\geq$2 `select` options, unique names) is regenerated alone by a short Tier-2 call for 5–8 business-specific fields, with fixed fields as fallback. On the 59 valid sites all 32 empty forms were regenerated by the model (fallback unused), giving 59/59 filled forms with 6–8 valid fields (all 220 conformant) and 7% example reuse, at 8.7 s per regeneration (4.8 s per site).

7.6 Real Tenant Pages

We ran the deployed engines on all 66 published pages of 28 tenant websites (read-only; only aggregate metrics stored). Rubric scores were 68.1 ± 4.2 (deployed scorer 60.6 ± 3.8), with 58 pages below the threshold; the loop raised them to 75.1 ± 3.7 and the pages at or above the threshold from 8 to 15, lowering none (33 proposals accepted, 41 rejected, 37 of them harmful). The rubric flagged 19 pages, mostly for a missing H1 or a buried CTA (18 each). The anomaly detector flagged 63 of 66, including 44 the rubric found nothing wrong with: trained on 16 home-page templates, it flags page type and size (all 24 pages with at most four sections, 57 of 58 inner pages) rather than design quality, and must be retrained before it transfers. Counterfactual reordering predicted a gain above 0.01 for 13 of the 57 pages that admit a move. All 30 forms had fields.

7.7 Deployment Pipeline

We ran the agentic pipeline against a Firebase test project under fresh tenant IDs (all test data deleted afterwards). All 20 benchmark sites deployed on the first attempt (median 0.29 s), so we injected six error classes and an unclassified error into the deploy call, once (transient) or on every attempt (persistent), three times each, under three retry policies. With the DiagnosingAgent, all 42 faults were classified correctly and the 9 transient medium-severity ones recovered; without retries none of the 21 transient faults recovered; blind retry recovered all 21 but spent 21 extra attempts on persistent faults (agent: 9), retried 12 permission and credential failures and took twice as long (99 vs. 51 s). The agent retries only what is safe, at the cost of 6 recoverable quota and storage faults; after five failures the circuit breaker rejected further deploys. These are fault-injection results on a test project, not production failure rates.

7.8 End-to-End Ablation

We ran the pipeline end to end on the same 20 sites, changing one factor per arm (Table 5; paired t-tests over sites). An independent re-run reproduced the full system (72.0 ± 4.8; p=0.10, 31 of 60 pages identical, none regressed). Removing the SEO loop costs 9.4 rubric points, the largest local effect; removing form repair saves 6 s per site but leaves 12 of 20 forms empty. Reasoning replaces 8 of 20 sites with the template and lowers the score by 3.9 points (not significant, p=0.07). Without the gate, 35 of 84 accepted edits lowered the score and 3 pages ended

Table 5. End-to-end ablation on the 20 benchmark sites (60 pages), one factor changed per arm. SEO: final rubric score, mean ± 95% CI over pages (text p-values are paired over sites; the re-run is in the text); forms filled: forms with ≥3 valid fields after repair; regressed: pages below their start; time and cost per site: generation (median over runs), rewriting, LLM scoring and form repair, as marginal GPU energy plus API fees.

Arm	Model layout	Forms filled	SEO	Regressed	Time (s)	$/1k sites
Full system (all seven sections)	20/20	20/20	72.7 ± 4.7	0/60	113	0.23
No form repair	20/20	8/20	72.7 ± 4.7	0/60	107	0.21
Reasoning enabled	12/20	12/20	68.8 ± 5.1	0/60	123	0.25
No acceptance gate	20/20	20/20	71.9 ± 4.7	3/60	131	0.26
Rule scorer (no learned model)	20/20	20/20	71.8 ± 4.5	0/60	107	0.21
LLM scorer (Tier 2 instead of Tier 1)	20/20	20/20	71.3 ± 4.8	0/60	120	0.24
Block recommender chooses sections	18/20	18/20	77.9 ± 4.6	0/60	95	0.19
No SEO loop	20/20	20/20	63.3 ± 5.3	0/60	83	0.17
Cloud generation	20/20	20/20	78.0 ± 4.2	0/60	44	13.48

below their start, yet the mean fell only 0.8 points (p=0.12): the gate prevents regressions rather than raising the average. *Routing* the scoring step matters: the rubric as scorer is indistinguishable from the learned model (−1.0, p=0.08), but the Tier-2 model loses 1.5 points (p=0.001) and adds 7 s per site, rejecting 51 of 67 proposals. With the recommender choosing sections, home pages average 8.4 sections (fallbacks included) instead of 7 and score 77.9 (+5.2, p=0.06), on a par with cloud generation and 18 s faster; two longer sites truncated and are scored as served, with the template (+7.5, p=0.004 on the other 18). Cloud generation scores 78.0 (p=0.005) at 44 s per site but costs $13.48 ($13.43 of API fees plus $0.05 of rewriting energy) instead of $0.23 ($0.15 of generation energy plus rewriting and repair) per 1,000 sites, and sends all generation data off-host. The top reorder suggestion changes nothing measurable on generated pages: all 20 home pages share the prompt-mandated order and get the same suggestion (predicted +0.062), with no design or SEO rubric gain (−0.2); its value lies with hand-built pages (Sect. 7.6). The anomaly detector is informational and cannot change a page.

8 Security and Multi-Tenancy

Every tenant-owned document carries a `tenantId`. An authentication chain verifies the JWT (seven-day expiry), rejects tokens whose tenant differs from the user's, checks that the tenant is active and injects a tenant filter into queries; four roles gate mutating routes, passwords use bcrypt (12 rounds), and the API is rate-limited. Public endpoints serve only published content. A static audit of all 163 database call sites found 118 with the tenant filter in the call; the other 45 are inherently cross-tenant (tenant or domain resolution, global uniqueness, administration and billing: 31), scoped through a tenant-filtered parent (2), the published-pages chatbot (1), or offline scripts unreachable over HTTP (11).

Dynamic isolation tests. On an empty in-memory database we registered two tenants through the API and sent tenant B's owner token with tenant A's identifiers to all 39 route–method pairs that take a resource identifier; a request counted as blocked only if it returned 401/403/404, left A's documents byte-

identical and echoed none of A's identifiers (12 positive controls with A's token succeeded). All 39 were blocked; 10 list endpoints exposed nothing of A; 7 tampered tokens (e.g. `alg:none`, wrong secret, a foreign tenant claim) were rejected, as was an editor on 8 owner/admin routes even with a forged `OWNER` claim; public endpoints served no drafts. Two defects were found, fixed and re-tested (93/93 checks pass): a domain could be linked to another tenant's website, and the deployment route lacked a role check. These are functional authorisation tests, not a penetration test; payment and LLM routes were excluded.

9 Limitations and Future Work

- **Synthetic labels, predicted lift.** The SEO labels are a linear rubric, so the scorer is not a ranking model until re-fitted to real search outcomes; sessions are simulated and flaws injected. All Δp are predictions; the tracker logged only six engagement events, so online controlled experiments [26] remain necessary.
- **Transfer and completeness.** The anomaly detector must be retrained on real page types (Sect. 7.6); of the generated content, only structure and form fields are validated.
- **Scope of evidence.** One laptop GPU; 251 development-period requests without escalation reasons; deployments on a test project with injected faults; no expert labels for block-level recommender accuracy, and author-written test prompts; no human evaluation of copy and no regression suite beyond the released scripts; no penetration test or external security audit.

10 Conclusion

Most decisions an AI website builder makes do not need a language model. Small learned models answer bounded questions in milliseconds; a 4-billion-parameter local model produces schema-valid sites in 59 of 60 cases once its reasoning mode is disabled, at about 1% of the cloud's marginal cost, with targeted regeneration repairing the forms it leaves empty; and cloud calls become an explicit, auditable exception. Executing the deployed code also showed where learning does not yet help: a linear model fits the rubric-derived SEO labels, sequence modelling adds no accuracy on synthetic sessions, and the design anomaly detector does not transfer to real pages. The most valuable next step is therefore real data—search outcomes, visitor sessions and controlled experiments—to which the Tier-1 engines can be re-fitted.

References

1. Wix.com Ltd.: Wix ADI. `https://www.wix.com/adi` (2024)
2. Webflow, Inc.: Webflow. `https://webflow.com` (2024)
3. Gemini Team, Google: Gemini: a family of highly capable multimodal models. arXiv:2312.11805 (2023)

4. Google: Gemini Developer API pricing. `https://ai.google.dev/gemini-api/docs/pricing` (accessed October 2026)
5. Yang, A., et al. (Qwen Team): Qwen3 technical report. arXiv:2505.09388 (2025)
6. Ollama: qwen3.5 model library entry. `https://ollama.com/library/qwen3.5` (accessed October 2026)
7. Google: Gemini API models. `https://ai.google.dev/gemini-api/docs/models` (accessed October 2026)
8. Chen, L., Zaharia, M., Zou, J.: FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176 (2023)
9. Ong, I., et al.: RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665 (2024)
10. Varangot-Reille, C., et al.: Doing more with less: a survey on routing strategies for resource optimisation in large language model-based systems. J. Artif. Intell. Res. **86**, Article 14 (2026)
11. Willard, B.T., Louf, R.: Efficient guided generation for large language models. arXiv:2307.09702 (2023)
12. Si, C., et al.: Design2Code: How far are we from automating front-end engineering? arXiv:2403.03163 (2024)
13. Lu, Z., et al.: WebGen-Bench: evaluating LLMs on generating interactive and functional websites from scratch. In: NeurIPS (2025)
14. Ziakis, C., et al.: Important factors for improving Google search rank. Future Internet **11**(2), 32 (2019)
15. Cho, K., et al.: Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: EMNLP, pp. 1724–1734 (2014)
16. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997)
17. Chung, J., et al.: Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv:1412.3555 (2014)
18. Hidasi, B., et al.: Session-based recommendations with recurrent neural networks. In: ICLR (2016)
19. Liu, F.T., Ting, K.M., Zhou, Z.-H.: Isolation forest. In: IEEE ICDM, pp. 413–422 (2008)
20. Sakurada, M., Yairi, T.: Anomaly detection using autoencoders with nonlinear dimensionality reduction. In: Proc. MLSDA Workshop, pp. 4–11. ACM (2014)
21. Zong, B., et al.: Deep autoencoding Gaussian mixture model for unsupervised anomaly detection. In: ICLR (2018)
22. W3C: Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation (2018)
23. Yin, W., Hay, J., Roth, D.: Benchmarking zero-shot text classification: datasets, evaluation and entailment approach. In: EMNLP-IJCNLP, pp. 3914–3923 (2019)
24. Wang, W., et al.: MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. In: NeurIPS (2020)
25. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)
26. Kohavi, R., et al.: Controlled experiments on the web: survey and practical guide. Data Mining and Knowledge Discovery **18**(1), 140–181 (2009)
27. Bezemer, C.-P., Zaidman, A.: Multi-tenant SaaS applications: maintenance dream or nightmare? In: Proc. IWPSE-EVOL, pp. 88–92. ACM (2010)
28. McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947)
29. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. J. Am. Stat. Assoc. **22**(158), 209–212 (1927)

Plagiarism Report

The paper (Sitezy.pdf, 12 pages, 5,915 words) was checked with Turnitin on 13 October 2026 (submission ID 3058403239), with quotes and the bibliography excluded. The originality report shows a similarity index of **1%** (internet sources 0%, publications 0%, student papers 0%), and no single source contributes more than 1%. The report is reproduced on the following pages.

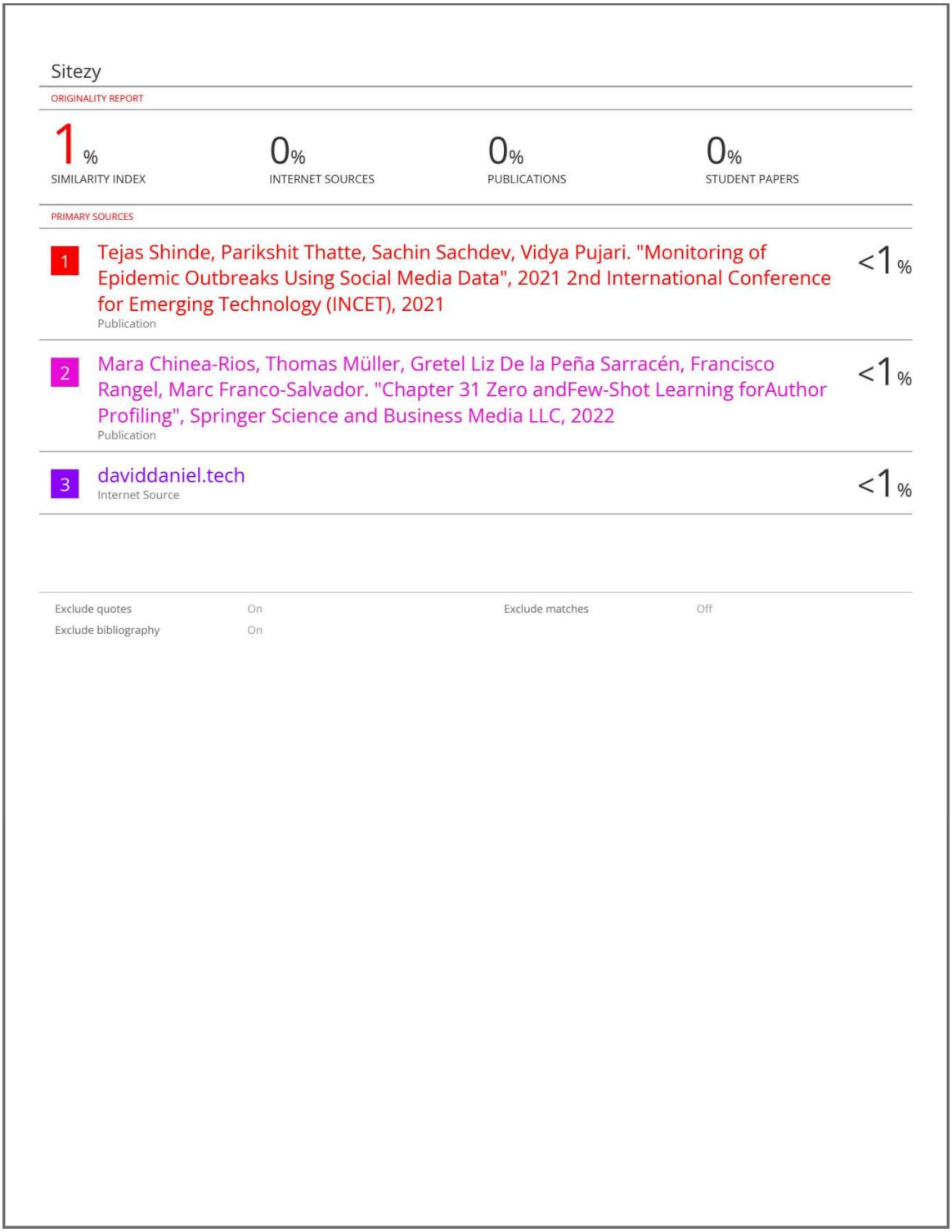
Table A.1: Summary of the originality and AI-writing checks

Check	Tool and date	Result
Similarity (plagiarism)	Turnitin originality report, 13 October 2026	1% similarity index; internet sources 0%, publications 0%, student papers 0%; each source < 1%
AI-writing detection	Turnitin AI writing overview, 13 October 2026	Score below the 20% reporting threshold (reported as *%)

Sitezy

by Shakti Sankpal

Submission date: 13-Oct-2026 08:36AM (UTC+0530)
Submission ID: 3058403239
File name: Sitezy.pdf (131.74K)
Word count: 5915
Character count: 29536



Sitezy

ORIGINALITY REPORT

1%	0%	0%	0%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

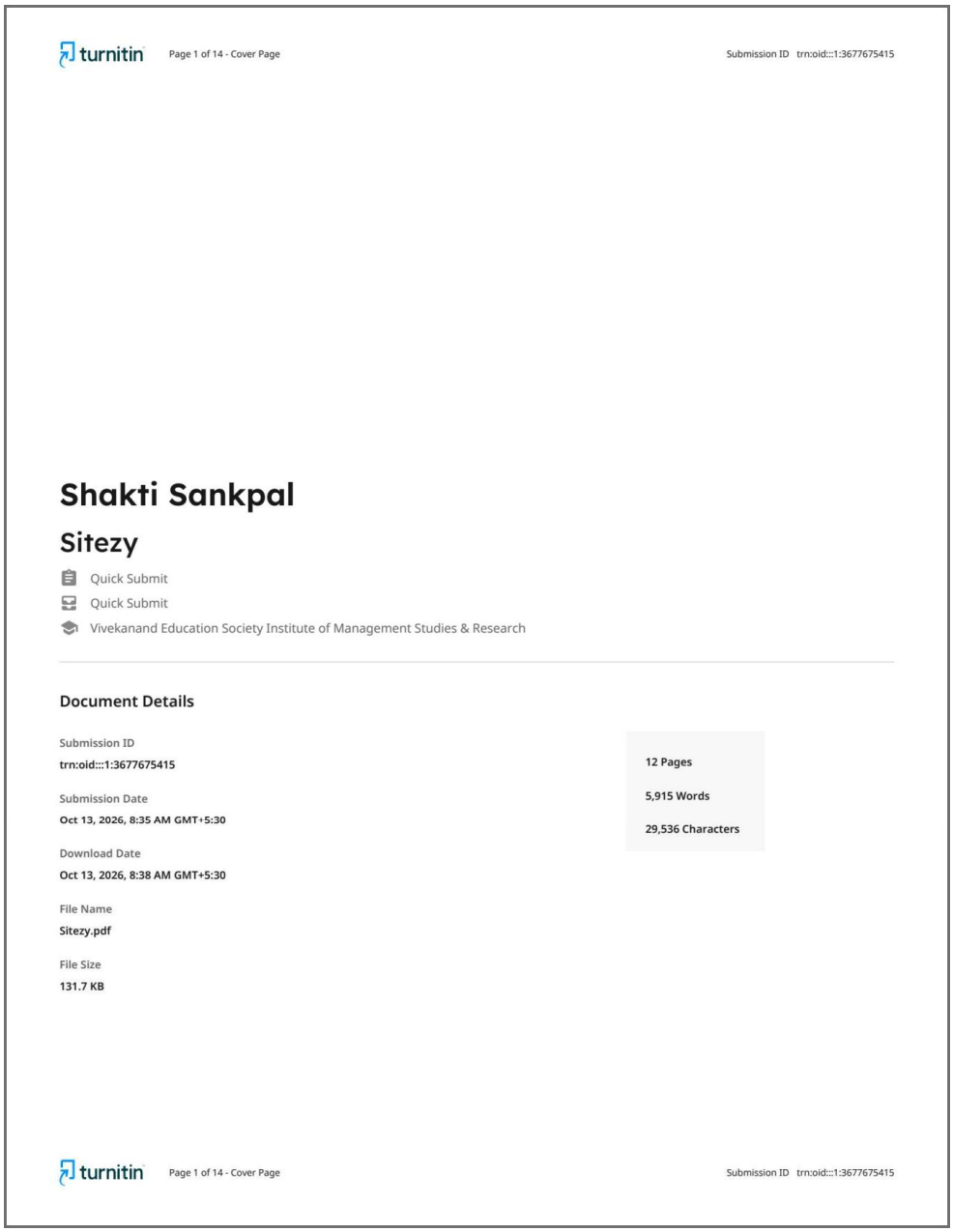
PRIMARY SOURCES

1	Tejas Shinde, Parikshit Thatte, Sachin Sachdev, Vidya Pujari. "Monitoring of Epidemic Outbreaks Using Social Media Data", 2021 2nd International Conference for Emerging Technology (INCET), 2021 Publication	<1%
2	Mara Chinea-Rios, Thomas Müller, Gretel Liz De la Peña Sarracén, Francisco Rangel, Marc Franco-Salvador. "Chapter 31 Zero andFew-Shot Learning forAuthor Profiling", Springer Science and Business Media LLC, 2022 Publication	<1%
3	daviddaniel.tech Internet Source	<1%

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		

AI Writing Check Report

The same document was checked with Turnitin's AI-writing detector on 13 October 2026 (submission ID trn:oid:::1:3677675415). Turnitin does not surface scores below 20% because of their higher likelihood of false positives, so the detected share is reported as *%, that is, below the threshold. The report is reproduced on the following pages.



turnitin Page 1 of 14 - Cover Page Submission ID trn:oid:::1:3677675415

Shakti Sankpal

Sitezy

Quick Submit

Quick Submit

Vivekanand Education Society Institute of Management Studies & Research

Document Details

Submission ID
trn:oid:::1:3677675415

Submission Date
Oct 13, 2026, 8:35 AM GMT+5:30

Download Date
Oct 13, 2026, 8:38 AM GMT+5:30

File Name
Sitezy.pdf

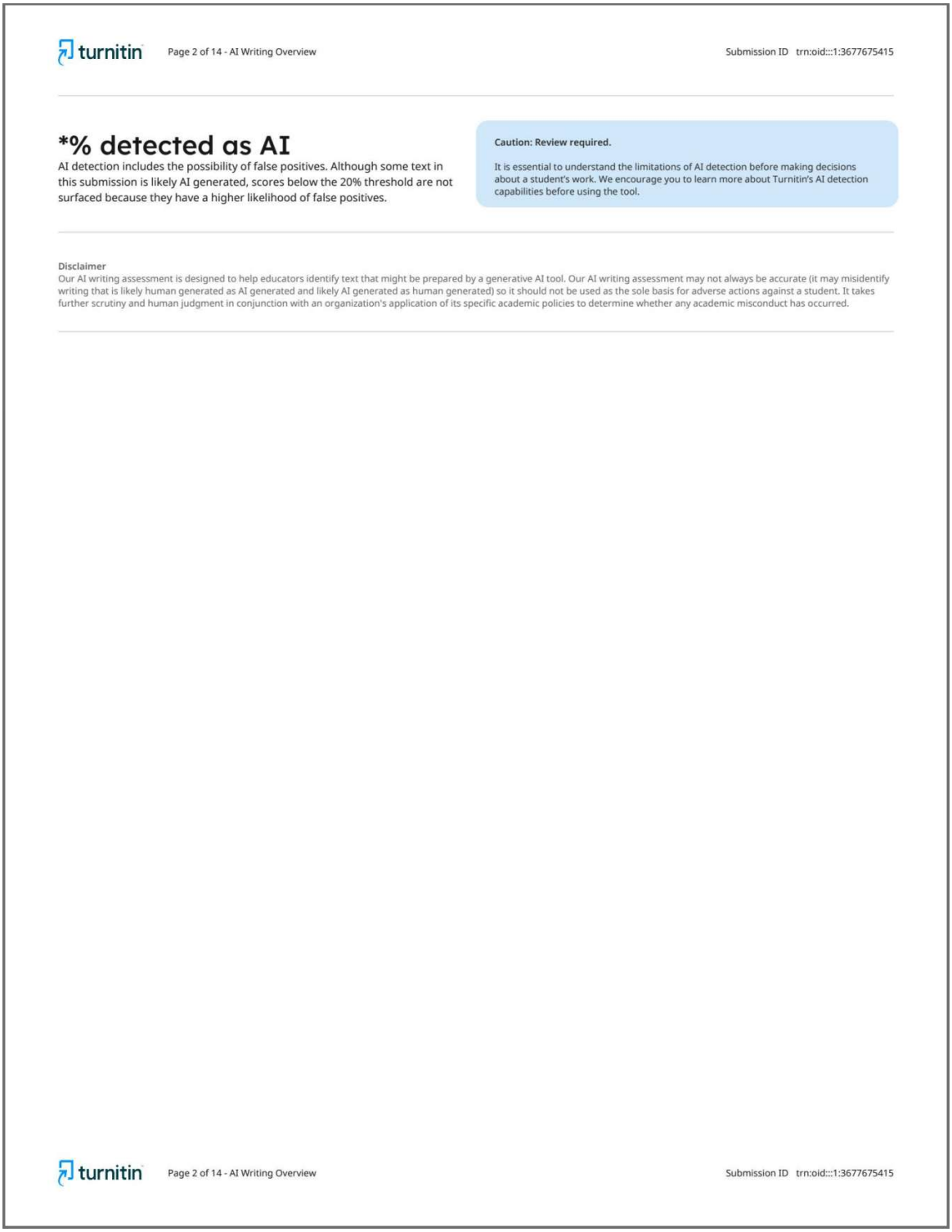
File Size
131.7 KB

12 Pages

5,915 Words

29,536 Characters

turnitin Page 1 of 14 - Cover Page Submission ID trn:oid:::1:3677675415



turnitin Page 2 of 14 - AI Writing Overview Submission ID trn:oid:::1:3677675415

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

turnitin Page 2 of 14 - AI Writing Overview Submission ID trn:oid:::1:3677675415

A.II. IPR: Copyright/Patent/etc. (if granted)

No intellectual property right had been granted at the time of submission of this report. An application for registration of copyright in the **Sitezy Workflow Diagram**, the block diagram reproduced as Fig. 3.8, was filed online with the Copyright Office, Government of India, on 31 August 2026 under Section 45 of the Copyright Act, 1957. The four students and the project guide are the joint authors and joint applicants. The application is under scrutiny; its particulars are summarised in Table A.2 and the portal records are shown in Figs. A.1 and A.2.

Table A.2: Copyright application filed for the project

Particular	Details
Diary number	AT-40971/2026-CO
Application	Form XIV with Statement of Particulars and Statement of Further Particulars (Rule 70); form received online
Title of the work	Sitezy Workflow Diagram
Class of work / language	Artistic work / English
Nature of the work	Original work; unpublished
Authors and applicants	Shakti Sankpal, Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi and Kajal Joseph (joint authors and joint owners; nature of interest: author)
Filed by / on	Shakti Sankpal (for all applicants), 31/08/2026
Fee	₹500, paid online on 31/08/2026 (payment ID 484543)
Supporting documents	Supporting document with the work and signed declaration (28/08/2026); No Objection Certificates of all five joint authors (31/08/2026)
Status	Scrutiny (registration awaited)

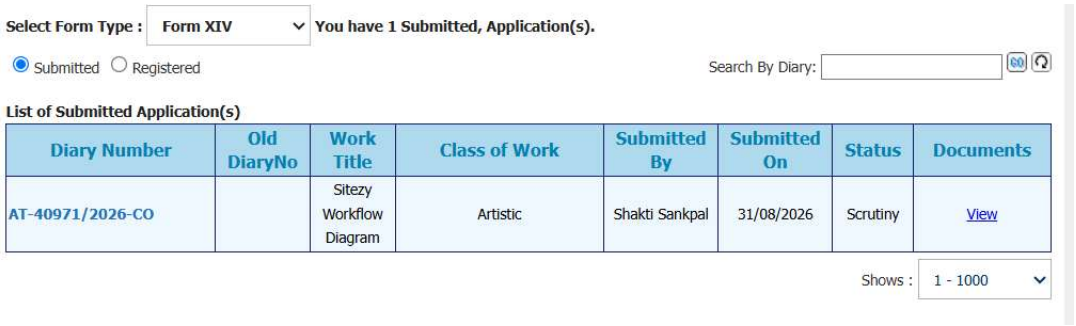


Figure A.1: Copyright Office portal: list of submitted applications (Form XIV)

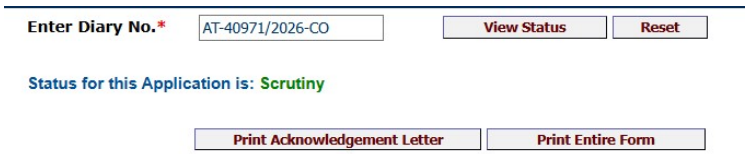
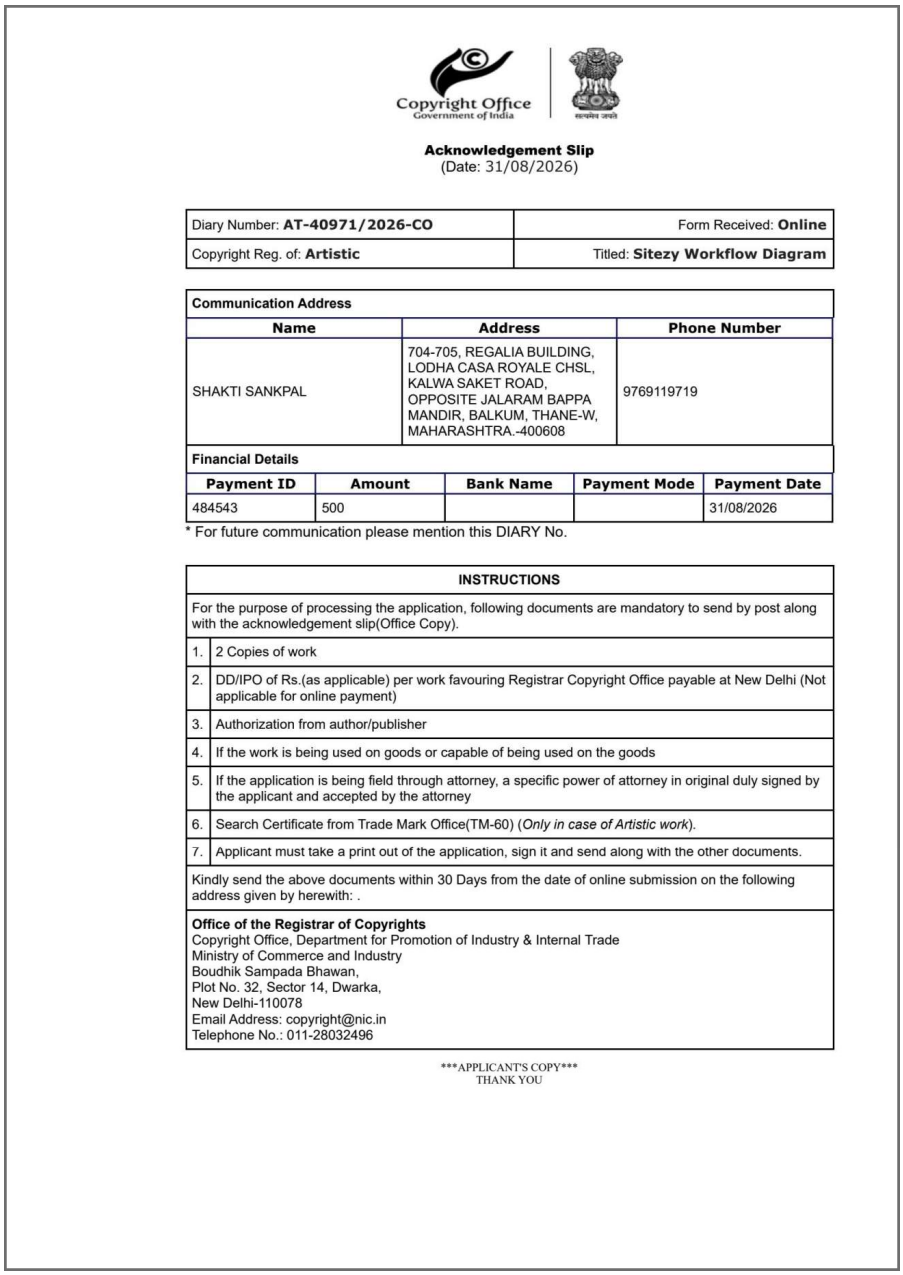


Figure A.2: Copyright Office portal: status of diary number AT-40971/2026-CO

The filed documents are reproduced on the following pages as submitted.

Acknowledgement Slip

Acknowledgement slip (applicant's copy) issued by the Copyright Office for diary number AT-40971/2026-CO.



Copyright Office
Government of India

Acknowledgement Slip
(Date: 31/08/2026)

Diary Number: **AT-40971/2026-CO**	Form Received: **Online**
Copyright Reg. of: **Artistic**	Titled: **Sitezy Workflow Diagram**

Communication Address

Name	Address	Phone Number
SHAKTI SANKPAL	704-705, REGALIA BUILDING, LODHA CASA ROYALE CHSL, KALWA SAKET ROAD, OPPOSITE JALARAM BAPPA MANDIR, BALKUM, THANE-W, MAHARASHTRA.-400608	9769119719

Financial Details

Payment ID	Amount	Bank Name	Payment Mode	Payment Date
484543	500			31/08/2026

* For future communication please mention this DIARY No.

INSTRUCTIONS

For the purpose of processing the application, following documents are mandatory to send by post along with the acknowledgement slip(Office Copy).

1. 2 Copies of work
2. DD/IPO of Rs.(as applicable) per work favouring Registrar Copyright Office payable at New Delhi (Not applicable for online payment)
3. Authorization from author/publisher
4. If the work is being used on goods or capable of being used on the goods
5. If the application is being field through attorney, a specific power of attorney in original duly signed by the applicant and accepted by the attorney
6. Search Certificate from Trade Mark Office(TM-60) (*Only in case of Artistic work*).
7. Applicant must take a print out of the application, sign it and send along with the other documents.

Kindly send the above documents within 30 Days from the date of online submission on the following address given by herewith: .

Office of the Registrar of Copyrights
Copyright Office, Department for Promotion of Industry & Internal Trade
Ministry of Commerce and Industry
Boudhik Sampada Bhawan,
Plot No. 32, Sector 14, Dwarka,
New Delhi-110078
Email Address: copyright@nic.in
Telephone No.: 011-28032496

APPLICANT'S COPY
THANK YOU

Form XIV with Statement of Particulars

Application for registration of copyright (Form XIV), with the Statement of Particulars and the Statement of Further Particulars for an artistic work.

FORM XIV
APPLICATION FOR REGISTERATION OF COPYRIGHT
[SEE RULE 70]

Diary Number: AT-40971/2026-CO

To

The Registrar of Copyrights,
Copyright Office,
Department of Industrial Policy & Promotion,
Ministry of Commerce and Industry,
Boudhik Sampada Bhawan,
Plot No. 32, Sector 14, Dwarka,
New Delhi-110075
Email Address: copyright@nic.in
Telephone No.: (Office) 011-28032496, 08929474194

Sir,

In Accordance with Section 45 of the Copright Act, 1957 (14 of 1957), I hereby apply for registration of Copyright and request that enteries may be made in the Register of Copyrights as in the enclosed Statement of Particulars.

1. I also send herewith duly completed the Statement of further Particulars relating to the work. (for Literary/Dramatic, Musical, Artistic works only) **Artistic works**

2. In accordance with rule 16 of the Copyright Rules, 1958, I have sent by prepaid registered post copies of this letter and of the Statement of Particulars and Statement of Further Particulars to other parties concerned as shown below:

[See columns 7,11,12, and 13 of the Statement of Particulars and party referred in col.2 (e) of the Statement of Further Particulars.]

3. The prescribed fee has been paid, as per details below: **500/-**

Payment ID	Payment Date	Amount	Bank Name	Payment Mode
484543	31/08/2026	500		

4. Communications on this subject may be addressed to:

SHAKTI SANKPAL
704-705, REGALIA BUILDING,
LODHA CASA ROYALE CHSL,
KALWA SAKET ROAD,
OPPOSITE JALARAM BAPPA
MANDIR, BALKUM, THANE-W,
MAHARASHTRA.-400608
9769119719

5. I hereby declare that to the best of my knowledge and belief, no person, other than to whom a notice has been sent as per paragraph 2 above any claim or interest or dispute to my copyright of this work or its use by me.

6. I hereby verify that the particulars given in this Form and the Statement of Particulars and Statment of Further Particulars are true to the best of my knowledge, belief and information and nothing has been concealed there from.

List of Enclosures:

1. 2 Copies of Work
2. DD/IPO of Rs.500 Per Work
3. Authorization from author
4. Search Certificate from Trade Mark Office (TM -60)
5. If the application is being filed through attorney , a specific Power of Attorney in original duly signed by the applicant and accepted by the attorney

Place:

Date: **31/08/2026**

For : SHAKTI SANKPAL



Proprietor

STATEMENT OF PARTICULARS

Diary Number: AT-40971/2026-CO

1.	Registration Number	
2.	Name, Address and Nationality of the Applicant	NAME: SHAKTI SANKPAL, ADDRESS: 704-705, REGALIA BUILDING, LODHA CASA ROYALE CHSL, KALWA SAKET ROAD, OPPOSITE JALARAM BAPPA MANDIR, BALKUM, THANE-W, MAHARASHTRA.-400608, Indian NAME: RONIT SAHOO, ADDRESS: FLAT NO. 902, ALLAMANDA B WING, PRIDE PARK, NEAR R-MALL, GHODBUNDER ROAD, THANE WEST-400607, Indian NAME: ATHARVA LOTANKAR, ADDRESS: 305/306, SERENA, SENROOFS, MULUND GOREGAON LINK ROAD, MULUND EAST, MUMBAI-400081, Indian NAME: MANASWI KABADI, ADDRESS: 303 , LIMAYE APARTMENT , 69 , BRAHMAN ALI , BEHIND VITTHAL MANDIR , BHIWANDI-421302, Indian NAME: KAJAL JOSEPH, ADDRESS: 240/1913, SHRIRANG SABDE MARG, ROAD NO.4, MOTILAL NAGAR-1, GOREGAON W-400104, Indian
3.	Nature of the Applicant's interest in the Copyright of the work	Author
4.	Class and description of the work	Artistic Work
5.	Title of the work	Sitezy Workflow Diagram
6.	Language of the work	English
7.	Name, Address and Nationality of the Author and if the Author is deceased, the date of decease.	NAME: SHAKTI SANKPAL, ADDRESS: 704-705, REGALIA BUILDING, LODHA CASA ROYALE CHSL, KALWA SAKET ROAD, OPPOSITE JALARAM BAPPA MANDIR, BALKUM, THANE-W, MAHARASHTRA.-400608, Indian, NAME: RONIT SAHOO, ADDRESS: FLAT NO. 902, ALLAMANDA B WING, PRIDE PARK, NEAR R-MALL, GHODBUNDER ROAD, THANE WEST-400607, Indian, NAME: ATHARVA LOTANKAR, ADDRESS: 305/306, SERENA, SENROOFS, MULUND GOREGAON LINK ROAD, MULUND EAST, MUMBAI-400081, Indian, NAME: MANASWI KABADI, ADDRESS: 303 , LIMAYE APARTMENT , 69 , BRAHMAN ALI , BEHIND VITTHAL MANDIR , BHIWANDI-421302, Indian, NAME: KAJAL JOSEPH, ADDRESS: 240/1913, SHRIRANG SABDE MARG, ROAD NO.4, MOTILAL NAGAR-1, GOREGAON W-400104, Indian,
8.	Whether the work is Published or Unpublished	Unpublished
9.	Year and Country of first publication, and Name, Address and Nationality of the publisher	N/A
10.	Year and Countries of subsequent publications, if any, and Name, Address and Nationality of the publisher	N/A
11.	Name, Address and Nationality of the Owners of the various rights comprising the copyright in the work and extent of rights held by each, together with particulars of assignments and licence. If any	NAME: SHAKTI SANKPAL, ADDRESS: 704-705, REGALIA BUILDING, LODHA CASA ROYALE CHSL, KALWA SAKET ROAD, OPPOSITE JALARAM BAPPA MANDIR, BALKUM, THANE-W, MAHARASHTRA.-400608, Indian NAME: RONIT SAHOO, ADDRESS: FLAT NO. 902, ALLAMANDA B WING, PRIDE PARK, NEAR R-MALL, GHODBUNDER ROAD, THANE WEST-400607, Indian NAME: ATHARVA LOTANKAR, ADDRESS: 305/306, SERENA, SENROOFS, MULUND GOREGAON LINK ROAD, MULUND EAST, MUMBAI-400081, Indian NAME: MANASWI KABADI, ADDRESS: 303 , LIMAYE APARTMENT , 69 , BRAHMAN ALI , BEHIND VITTHAL MANDIR , BHIWANDI-421302, Indian NAME: KAJAL JOSEPH, ADDRESS: 240/1913, SHRIRANG SABDE MARG, ROAD NO.4, MOTILAL NAGAR-1, GOREGAON W-400104, Indian

12.	Name and address and nationality of other persons, if any authorized to assign or licence the rights comprising the copyright	N/A
13.	If the work is an 'Artistic work', the location of the original work, including name, address and nationality of the person in possession of the work, (In the case of an architectural work, the year of completion of the work should also be shown)	LOCATION: THANE , NAME: SHAKTI SANKPAL, ADDRESS: 704-705, REGALIA BUILDING, LODHA CASA ROYALE CHSL, KALWA SAKET ROAD, OPPOSITE JALARAM BAPPA MANDIR, BALKUM, THANE-W, MAHARASHTRA.-400608, Indian LOCATION: THANE , NAME: RONIT SAHOO, ADDRESS: FLAT NO. 902, ALLAMANDA B WING, PRIDE PARK, NEAR R-MALL, GHODBUNDER ROAD, THANE WEST-400607, Indian LOCATION: MUMBAI , NAME: ATHARVA LOTANKAR, ADDRESS: 305/306, SERENA, SENROOFS, MULUND GOREGAON LINK ROAD, MULUND EAST, MUMBAI-400081, Indian LOCATION: THANE , NAME: MANASWI KABADI, ADDRESS: 303 , LIMAYE APARTMENT , 69 , BRAHMAN ALI , BEHIND VITTHAL MANDIR , BHIWANDI-421302, Indian LOCATION: MUMBAI , NAME: KAJAL JOSEPH, ADDRESS: 240/1913, SHRIRANG SABDE MARG, ROAD NO.4, MOTILAL NAGAR-1, GOREGAON W-400104, Indian
14.	If the work is an 'Artistic work' which is used or capable of being used in relation to any goods or services, the application should include a certification from the Registrar of Trade Marks in terms of the provision to Sub-Section (i) of Section 45 of the Copyright Act, 1957	N/A
15.	If the work is an 'Artistic work' whether it is registered under the Desings Act 2000 if yes give details.	N/A
16.	If the work is an 'Artistic work' capable of being registrar as a design under the Designs Act 2000, whether is has been applied to an article though an industrial process and,if yes ,then number of times it is reproduced	N/A
17.	Remarks, if any	

Place:
Date: **31/08/2026**

For : SHAKTI SANKPAL



Proprietor

STATEMENT OF FURTHER PARTICULARS

(For Literary/Dramatic, Musical and Artistic works only)

Diary Number: AT-40971/2026-CO

1. Is the work to be registered

(a) an orginal work? : Yes

(b) a translation of a work in the public domain? : N.A.

(c) a translation of a work in which Copyright subsists? : N.A.

(d) an adaptation of a work in the public domain? : N.A.

(e) an adaptation of a work in which Copyright subsists? : N.A.

2. If the work is a translation or adaptation of a work in which copyright subsists

(a) Title of the original work : N.A.

(b) Language of the original work : N.A.

(c) Name, address, and nationality of the author of the original work and if the author is deceased, the date of decease : N.A.

(d) Name, address, and nationality of the publisher, if any, of the original work : N.A.

(e) Name, address, and nationality of the publisher, or adaptation including the name, address and nationality of party authorizing : N.A.

3. Remarks, if any

Place:
Date: **31/08/2026**

For : SHAKTI SANKPAL



Proprietor

Supporting Document

Supporting document filed with Form XIV: title, authors, objectives of the work, the work itself (Sitezy Workflow Diagram) and the declaration signed by all five joint authors.

SUPPORTING DOCUMENT

Accompanying Form XIV — Application for Registration of Copyright (Artistic Work)

1. Title of the Work

Sitezy Workflow Diagram — a block diagram illustrating the AI-generation, deployment and data-flow architecture of the Sitezy AI website-builder platform across its Frontend, AI Generation, Deployment and Backend layers.

2. Author & Applicant Details

Name	Role	Address	Email
Ronit Sahoo	Author / Applicant	Flat No. 902, Allamanda B-Wing, Pride Park, Near R-Mall, Ghodbunder Road, Thane(West) 400607	ronitsahoo221@gmail.com
Shakti Sankpal	Author / Applicant	704-705, Regalia Building, Lodha Casa Royale CHSL, Kalwa-Saket Road, Balkum, Thane-W (400608)	shaktisankpal42@gmail.com
Atharva Lotankar	Author / Applicant	305/306, Serena, Senroofs, Mulund Goregaon Link Road, Mulund East, Mumbai - 400081	atharvalotankar11@gmail.com
Manaswi Kabadi	Author / Applicant	303 , Limaye Apartment , 69 , Brahman Ali , Behind Vitthal Mandir , Bhiwandi - 421302	manaswikabadi@gmail.com
Kajal Joseph	Author / Applicant	240/1913, Shrirang Sabde Marg, Road No.4, Motilal Nagar-1, Goregaon (W) - 400104	jewani.kajal@gmail.com

All five individuals named above are joint authors and joint applicants for this work, with equal and undivided ownership of the copyright.

3. Objectives - Theory Behind the Block Diagram

The diagram maps the complete system architecture of Sitezy — a multi-tenant, AI-driven website-builder platform — showing how a website is generated, deployed and continuously optimized end-to-end. Its objectives are to visually communicate:

- **The client and application layer:** a React 19 + Vite front end (Dashboard, AI Playground, SEO panel, published site with AI chatbot) and real-time Socket.IO collaboration, communicating over REST / WebSocket with a Node.js + Express Backend API (JWT / RBAC, middleware, REST modules), together with its Razorpay (billing) and Prometheus / Grafana (metrics) integrations.
- **The AI Generation subsystem:** a two-stage pipeline in which an AI Gen Controller drives a RandomForest layout recommender (section-type prediction), a tiered LLM (local Ollama qwen3.5:4b with automatic fallback to cloud Gemini) that authors the content, schema validation with a deterministic fallback, and an independent image resolver that maps sanitized keywords to verified Unsplash assets with enforced uniqueness.
- **The AI / ML engine (Flask :5050)** underpinning that subsystem: an MLP SEO scorer, an autoencoder + Isolation Forest design detector, a GRU engagement model, and the RandomForest + MiniLM zero-shot recommender, together with the closed-loop Auto-Improve (score → LLM rewrite → re-score).
- **The Autonomous Deployment subsystem:** an Orchestrator (bounded retry, exponential backoff, per-tenant circuit breaker, deploy log) coordinating a Firebase deployment agent and a diagnosing agent (isolation check, quota, auto-fix), deploying tenant-isolated sites to Firebase (Firestore, Hosting, Storage).
- **The data layer and the data-flow bridges that tie it together:** the tenant-scoped MongoDB (Mongoose) store; the *Generate* and *Publish* handoffs from the Backend API into the generation and deployment subsystems; the *CRUD (tenant-scoped)* path to MongoDB; and the *Serves Published Site* path returning the deployed website to end users — making explicit how generation, deployment and optimization are chained rather than independent.

4. The Artistic Work (Block Diagram)

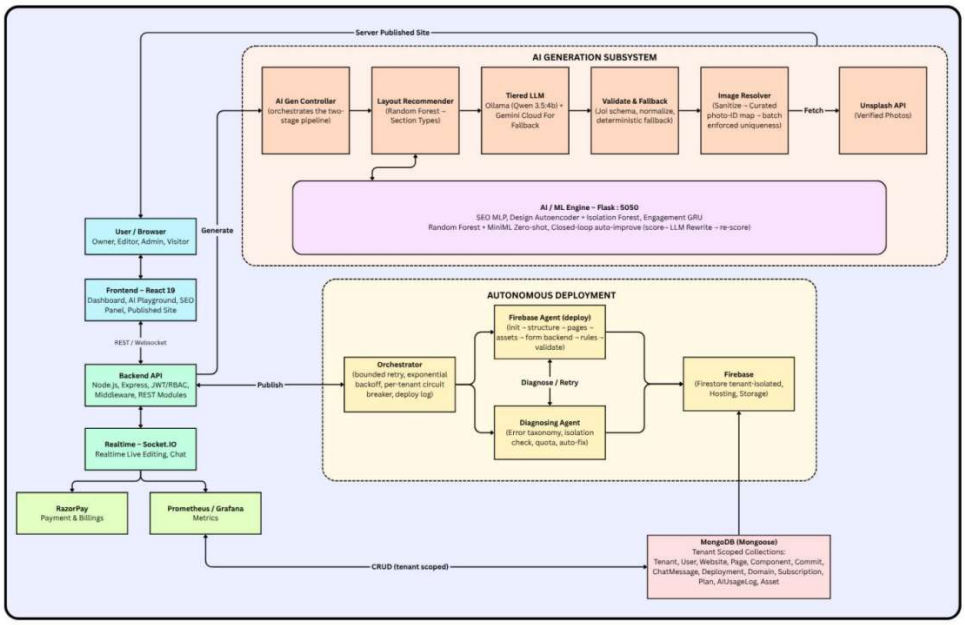


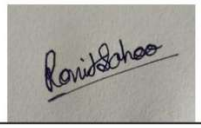
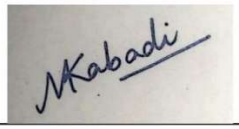
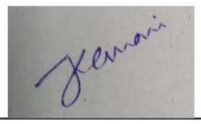
Fig. 1 — Sitezy Workflow Diagram

5. Declaration

We, the undersigned, being the joint authors and applicants of the artistic work titled "Sitezy Workflow Diagram", hereby declare that:

- The work is our own original creation, produced through our independent effort, skill, and judgment.
- No part of the work has been copied from any other artistic work, published or unpublished, in which copyright subsists.
- No person other than the undersigned has any claim, interest, or dispute over the copyright of this work or its use by us.
- All particulars furnished in this document and in the accompanying Form XIV, Statement of Particulars, and Statement of Further Particulars are true and correct to the best of our knowledge, belief, and information.

6. Signatures

 **Ronit Sahoo** Date: 28/08/2026	 **Shakti Sankpal** Date: 28/08/2026
 **Atharva Lotankar** Date: 28/08/2026	 **Manaswi Kabadi** Date: 28/08/2026
 **Kajal Joseph** Date: 28/08/2026	

Each signature above must be the individual's own — either signed by hand and scanned, or a scanned image of a wet-ink signature pasted in. A single person cannot sign on behalf of the group (Rule 70(3), Copyright Rules 2013).

No Objection Certificates

No Objection Certificates for joint registration, one from each of the five joint authors.

NO OBJECTION CERTIFICATE

To,

Head of Office,

Copyright Office, Department for Promotion of Industry and Internal Trade,

Boudhik Sampada Bhawan, Plot No. 32, Sector 14, Dwarka, New Delhi – 110078

Subject: No Objection for Joint Registration of Artistic Work titled "Sitezy Workflow Diagram"

Dear Sir/Madam,

I, Shakti Sankpal, residing at 704-705, Regalia Building, Lodha Casa Royale CHSL, Kalwa Saket Road, Opposite Jalaram Bappa Mandir, Balkum, Thane-W, holding Indian Nationality, do hereby solemnly affirm and declare as follows:

1. That I am one of the joint authors and joint first owners of the artistic work titled "Sitezy Workflow Diagram" (hereinafter "the Work"), created jointly by myself along with Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi, and Kajal Joseph.

2. That the Work was created jointly by all of us through our combined effort, skill, and judgment, as part of our academic research project, and no part of it has been copied from any pre-existing work in which copyright subsists.

3. That no rights in the Work have been assigned, transferred, or licensed by me to any person other than my continuing joint ownership alongside my co-authors named above.

4. I hereby confirm that I have no objection to this application for registration of copyright in the said Work being filed jointly in the names of all of us — Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi, Kajal Joseph and myself — as joint applicants and joint owners before your esteemed office.

5. I further confirm that I shall not raise any claim, dispute, or objection regarding the joint ownership or joint registration of the copyright in the said Work at any point in the future.

6. This No Objection Certificate is being issued voluntarily and for the purpose of facilitating the smooth joint registration of the Work.

I declare that the information provided above is true and correct to the best of my knowledge and belief.

Sincerely,



Shakti Sankpal

Email: shaktisankpal42@gmail.com

Date: 31/08/2026

NO OBJECTION CERTIFICATE

To,

Head of Office,

Copyright Office, Department for Promotion of Industry and Internal Trade,

Boudhik Sampada Bhawan, Plot No. 32, Sector 14, Dwarka, New Delhi – 110078

Subject: No Objection for Joint Registration of Artistic Work titled "Sitezy Workflow Diagram"

Dear Sir/Madam,

I, Ronit Sahoo, residing at Flat No.902, Allamanda B-Wing, Pride Park, Thane - West - 400607, holding Indian Nationality, do hereby solemnly affirm and declare as follows:

1. That I am one of the joint authors and joint first owners of the artistic work titled "Sitezy Workflow Diagram" (hereinafter "the Work"), created jointly by myself along with Shakti Sankpal, Atharva Lotankar, Manaswi Kabadi and Kajal Joseph.

2. That the Work was created jointly by all of us through our combined effort, skill, and judgment, as part of our academic research project, and no part of it has been copied from any pre-existing work in which copyright subsists.

3. That no rights in the Work have been assigned, transferred, or licensed by me to any person other than my continuing joint ownership alongside my co-authors named above.

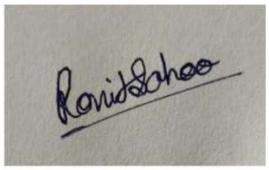
4. I hereby confirm that I have no objection to this application for registration of copyright in the said Work being filed jointly in the names of all of us — Shakti Sankpal, Atharva Lotankar, Manaswi Kabadi, Kajal Joseph and myself — as joint applicants and joint owners before your esteemed office.

5. I further confirm that I shall not raise any claim, dispute, or objection regarding the joint ownership or joint registration of the copyright in the said Work at any point in the future.

6. This No Objection Certificate is being issued voluntarily and for the purpose of facilitating the smooth joint registration of the Work.

I declare that the information provided above is true and correct to the best of my knowledge and belief.

Sincerely,



Ronit Sahoo

Email: ronitsahoo221@gmail.com

Date: 31/08/2026

NO OBJECTION CERTIFICATE

To,

Head of Office,

Copyright Office, Department for Promotion of Industry and Internal Trade,

Boudhik Sampada Bhawan, Plot No. 32, Sector 14, Dwarka, New Delhi – 110078

Subject: No Objection for Joint Registration of Artistic Work titled "Sitezy Workflow Diagram"

Dear Sir/Madam,

I, Atharva Lotankar, residing at 305/306, Serena, Senroofs, Mulund Goregaon Link Road, Mulund East, Mumbai - 400081, holding Indian Nationality, do hereby solemnly affirm and declare as follows:

1. That I am one of the joint authors and joint first owners of the artistic work titled "Sitezy Workflow Diagram" (hereinafter "the Work"), created jointly by myself along with Shakti Sankpal, Ronit Sahoo, Manaswi Kabadi, and Kajal Joseph.

2. That the Work was created jointly by all of us through our combined effort, skill, and judgment, as part of our academic research project, and no part of it has been copied from any pre-existing work in which copyright subsists.

3. That no rights in the Work have been assigned, transferred, or licensed by me to any person other than my continuing joint ownership alongside my co-authors named above.

4. I hereby confirm that I have no objection to this application for registration of copyright in the said Work being filed jointly in the names of all of us — Shakti Sankpal, Ronit Sahoo, Manaswi Kabadi, Kajal Joseph and myself — as joint applicants and joint owners before your esteemed office.

5. I further confirm that I shall not raise any claim, dispute, or objection regarding the joint ownership or joint registration of the copyright in the said Work at any point in the future.

6. This No Objection Certificate is being issued voluntarily and for the purpose of facilitating the smooth joint registration of the Work.

I declare that the information provided above is true and correct to the best of my knowledge and belief.

Sincerely,



Atharva Lotankar

Email: atharvalotankar11@gmail.com

Date: 31/08/2026

NO OBJECTION CERTIFICATE

To,

Head of Office,

Copyright Office, Department for Promotion of Industry and Internal Trade,

Boudhik Sampada Bhawan, Plot No. 32, Sector 14, Dwarka, New Delhi – 110078

Subject: No Objection for Joint Registration of Artistic Work titled "Sitezy Workflow Diagram"

Dear Sir/Madam,

I, Manaswi Kabadi, residing at 303, Limaye Apartment, 69, Brahman Ali, Behind Vitthal Mandir, Bhiwandi - 421302, holding Indian Nationality, do hereby solemnly affirm and declare as follows:

1. That I am one of the joint authors and joint first owners of the artistic work titled "Sitezy Workflow Diagram" (hereinafter "the Work"), created jointly by myself along with Shakti Sankpal, Ronit Sahoo, Atharva Lotankar, and Kajal Joseph.

2. That the Work was created jointly by all of us through our combined effort, skill, and judgment, as part of our academic research project, and no part of it has been copied from any pre-existing work in which copyright subsists.

3. That no rights in the Work have been assigned, transferred, or licensed by me to any person other than my continuing joint ownership alongside my co-authors named above.

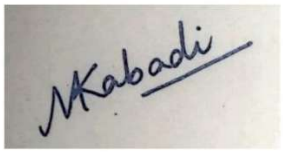
4. I hereby confirm that I have no objection to this application for registration of copyright in the said Work being filed jointly in the names of all of us — Shakti Sankpal, Ronit Sahoo, Atharva Lotankar, Kajal Joseph and myself — as joint applicants and joint owners before your esteemed office.

5. I further confirm that I shall not raise any claim, dispute, or objection regarding the joint ownership or joint registration of the copyright in the said Work at any point in the future.

6. This No Objection Certificate is being issued voluntarily and for the purpose of facilitating the smooth joint registration of the Work.

I declare that the information provided above is true and correct to the best of my knowledge and belief.

Sincerely,



Manaswi Kabadi

Email: manaswikabadi@gmail.com

Date: 31/08/2026

NO OBJECTION CERTIFICATE

To,

Head of Office,

Copyright Office, Department for Promotion of Industry and Internal Trade,

Boudhik Sampada Bhawan, Plot No. 32, Sector 14, Dwarka, New Delhi – 110078

Subject: No Objection for Joint Registration of Artistic Work titled "Sitezy Workflow Diagram"

Dear Sir/Madam,

I, Kajal Joseph, residing at 240/1913, Shrirang Sabde Marg, Road No. 4, Motilal Nagar-1, Goregaon West, Mumbai, 400104, holding Indian Nationality, do hereby solemnly affirm and declare as follows:

1. That I am one of the joint authors and joint first owners of the artistic work titled "Sitezy Workflow Diagram" (hereinafter "the Work"), created jointly by myself along with Shakti Sankpal, Ronit Sahoo, Atharva Lotankar and Manaswi Kabadi.

2. That the Work was created jointly by all of us through our combined effort, skill, and judgment, as part of our academic research project, and no part of it has been copied from any pre-existing work in which copyright subsists.

3. That no rights in the Work have been assigned, transferred, or licensed by me to any person other than my continuing joint ownership alongside my co-authors named above.

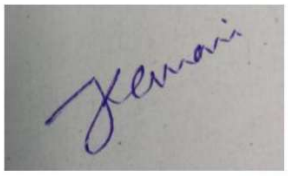
4. I hereby confirm that I have no objection to this application for registration of copyright in the said Work being filed jointly in the names of all of us — Shakti Sankpal, Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi and myself — as joint applicants and joint owners before your esteemed office.

5. I further confirm that I shall not raise any claim, dispute, or objection regarding the joint ownership or joint registration of the copyright in the said Work at any point in the future.

6. This No Objection Certificate is being issued voluntarily and for the purpose of facilitating the smooth joint registration of the Work.

I declare that the information provided above is true and correct to the best of my knowledge and belief.

Sincerely,



Kajal Joseph

Email: jewani.kajal@gmail.com

Date: 31/08/2026